Data Structures, Algorithms and Applications in C++ (Sartaj Sahni)

Pages selected from the course reference book to match the DSA lecture curriculum.
Book page + 22 = PDF page. Original is a scan (text not searchable).

Contents (book page range : topic) in printed order:

pp. 33-43 Ch1 Recursion
pp. 55-94 Ch2 Performance analysis: space and time complexity, operation and step counts, simple sorts and searches
pp. 95-120 Ch3 Asymptotic notation (Big-O, Omega, Theta), binary search
pp. 139-146 Ch5 Array representation of linear lists
pp. 172-192 Ch6 Singly linked lists and chains
pp. 269-289 Ch8 Stacks (array and linked, parenthesis matching, Towers of Hanoi)
pp. 317-333 Ch9 Queues (array and linked)
pp. 418-442 Ch11 Binary trees, representations, traversal
pp. 529-545 Ch14 Binary search trees
pp. 704-713 Ch18 Divide and conquer (introduction)

16. Extend the class `currency` of Program 1.13 by adding the following `public` member functions:

    (a) `input()` inputs a currency value from the standard input stream and assign it to the invoking object.

    (b) `subtract(x)` subtracts the value of the currency object `x` from that of the invoking object and returns the result.

    (c) `percent(x)` returns a currency object whose value is `x` percent of the value of the invoking object. The data type of `x` is `double`.

    (d) `multiply(x)` returns the currency object that results from multiplying the invoking object and the number `x`, which is of type `double`.

    (e) `divide(x)` returns the currency object that results from dividing the invoking object by the number `x`, which is of type `double`.

    Implement all member functions and test their correctness using suitable test data.

17. Do Exercise 16 using the implementation of Program 1.19.

18. (a) Do Exercise 16 using the implementation of Program 1.22. Overload the operators `>>`, `-`, `%`, `*`, and `/`. When overloading `>>`, declare it as a friend function and do not define a `public` input function to facilitate the input.

    (b) Replace the two `setValue` functions by overloading the assignment operator `=`. An overload of the type `operator=(int x)` that assigns an integer to an object of type `currency` should replace the three-parameter `setValue` function. `x` represents the sign, dollar amount, and cents rolled into a single integer. An overload of the type `operator=(double x)` should replace the single-parameter `setValue` function.

1.6 THE EXCEPTION CLASS `illegalParameterValue`

Program 1.26 shows a user-defined class `illegalParameterValue` that may be used when signaling errors in which the value of an actual parameter to a function is improper. Program 1.27 is a version of Program 1.8 in which an exception of type `illegalParameterValue` is thrown instead of an exception of type `char*`. Program 1.28 shows how to catch an exception of type `illegalParameterValue`.

1.7 RECURSION

A **recursive function** or method invokes itself. In **direct recursion** the code for function `f` contains a statement that invokes `f`, whereas in **indirect recursion**

Copyrighted material

```
class illegalParameterValue
{
   public:
      illegalParameterValue() :
            message("Illegal parameter value") {}
      illegalParameterValue(char * theMessage)
            {message = theMessage;}
      void outputMessage() {cout << message << endl;}
   private:
      char * message;
};
```

Program 1.26 Defining an exception class

```
int abc(int a, int b, int c)
{
   if (a <= 0 || b <= 0 || c <= 0)
       throw illegalParameterValue ("All parameters should be > 0");
   return a + b * c;
}
```

Program 1.27 Throwing an exception of type `illegalParameterValue`

```
int main()
{
   try {cout << abc(2,0,4) << endl;}
   catch (illegalParameterValue e)
   {
      cout << "The parameters to abc were 2, 0, and 4" << endl;
      cout << "illegalParameterValue exception thrown" << endl;
      e.outputMessage();
      return 1;
   }
   return 0;
}
```

Program 1.28 Catching an exception of type `illegalParameterValue`

Copyrighted material

the function `f` invokes some other function `g`, which invokes yet another function `h`, and so on until function `f` is again invoked. Before delving into recursive C++ functions, we examine two related concepts from mathematics—recursive definitions of mathematical functions and proofs by induction.

1.7.1 Recursive Mathematical Functions

In mathematics we often define a function in terms of itself. For example, the factorial function $f(n) = n!$, for n an integer, is defined as follows:

$$f(n) = \begin{cases} 1 & n \leq 1 \\ nf(n-1) & n > 1 \end{cases} \tag{1.1}$$

This definition states that $f(n)$ equals 1 whenever n is less than or equal to 1; for example, $f(-3) = f(0) = f(1) = 1$. However, when n is more than 1, $f(n)$ is defined recursively, as the definition of f now contains an occurrence of f on the right side. This use of f on the right side does not result in a circular definition, as the parameter of f on the right side is smaller than that on the left side. For example, from Equation 1.1 we obtain $f(2) = 2f(1)$. From Equation 1.1 we also obtain $f(1) = 1$, and substituting for $f(1)$ in $f(2) = 2f(1)$, we obtain $f(2) = 2$. Similarly, from Equation 1.1 we obtain $f(3) = 3f(2)$. We have already seen that Equation 1.1 yields $f(2) = 2$. So $f(3) = 3 * 2 = 6$.

For a recursive definition of $f(n)$ (we assume direct recursion) to be a complete specification of f, it must meet the following requirements:

- The definition must include a **base** component in which $f(n)$ is defined directly (i.e., nonrecursively) for one or more values of n. For simplicity, we assume that the domain of f is the nonnegative integers and that the base covers the case $0 \leq n \leq k$ for some constant k. (It is possible to have recursive definitions in which the base covers the case $n \geq k$ instead, but we encounter these definitions less frequently.)

- In the **recursive component** all occurrences of f on the right side should have a parameter smaller than n so that repeated application of the recursive component transforms all occurrences of f on the right side to occurrences of f in the base.

In Equation 1.1 the base is $f(n) = 1$ for $n \leq 1$; in the recursive component $f(n) = nf(n-1)$, the parameter of f on the right side is $n - 1$, which is smaller than n. Repeated application of the recursive component transforms $f(n-1)$ to $f(n-2)$, $f(n-3)$, $\cdots$, and finally to $f(1)$ which is included in the base. For example, repeated application of the recursive component gives the following:

$$f(5) = 5f(4) = 20f(3) = 60f(2) = 120f(1)$$

Copyrighted material

Notice that each application of the recursive component gets us closer to the base. Finally, an application of the base gives $f(5) = 120$. From the example, we see that $f(n) = n(n-1)(n-2)\cdots 1$ for $n \geq 1$.

As another example of a recursive definition, consider the Fibonacci numbers that are defined recursively as below:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ for } n > 1 \tag{1.2}$$

In this definition, $F_0 = 0$ and $F_1 = 1$ make up the base component, and $F_n = F_{n-1} + F_{n-2}$ is the recursive component. The function parameters on the right side are smaller than n. For Equation 1.2 to be a complete recursive specification of F, repeated application of the recursive component beginning with any value of $n > 1$ should transform all occurrences of F on the right side to occurrences in the base. Since repeated subtraction of 1 or 2 from an integer $n > 1$ reduces it to either 0 or 1, right-side occurrences of F are always transformed to base occurrences. For example, $F_4 = F_3 + F_2 = F_2 + F_1 + F_1 + F_0 = 3F_1 + 2F_0 = 3$.

1.7.2 Induction

Now, we turn our attention to the second concept related to recursive computer functions—proofs by induction. In a proof by induction, we establish the validity of a claim such as

$$\sum_{i=0}^{n} i = n(n+1)/2, n \geq 0 \tag{1.3}$$

by showing that the claim is true for one or more base values of n (generally, $n = 0$ suffices); we assume the claim is true for values of n from 0 through m where m is an arbitrary integer greater than or equal to the largest n covered in the base; and finally using this assumption, we show the claim is true for the next value of n (i.e., $m+1$). This methodology leads to a proof that has three components—**induction base**, **induction hypothesis**, and **induction step**.

Suppose we are to prove Equation 1.3 by induction on n. In the induction base we establish correctness for $n = 0$. At this time the left side is $\sum_{i=0}^{0} i = 0$, and the right side is also 0. So Equation 1.3 is valid when $n = 0$. In the induction hypothesis we assume the equation is valid for $n \leq m$ where m is an arbitrary integer ≥ 0. (For the ensuing induction step proof, it is sufficient to assume that Equation 1.3 is valid only for $n = m$.) In the induction step we show that the equation is valid for $n = m+1$. For this value of n, the left side is $\sum_{i=0}^{m+1} i$, which equals $m+1+\sum_{i=0}^{m} i$. From the induction hypothesis we get $\sum_{i=0}^{m} i = m(m+1)/2$. So when $n = m+1$, the left side becomes $m+1+m(m+1)/2 = (m+1)(m+2)/2$, which equals the right side.

At first glance, a proof by induction appears to be a circular proof—we establish a result assuming it is correct. However, a proof by induction is not a circular proof

Copyrighted material

for the same reasons that a recursive definition is not circular. A correct proof by induction has an induction base similar to the base component of a recursive definition, and the induction step proves correctness using correctness for smaller values of n. Repeated application of the induction step reduces the proof to one that is solely in terms of the base.

1.7.3 Recursive C++ Functions

C++ allows us to write recursive functions. A proper recursive function must include a base component. The recursive component of the function should use smaller values of the function parameters so that repeated invocation of the function results in parameters equal to those included in the base component.

Example 1.1 [Factorial] Program 1.29 gives a C++ recursive function that uses Equation 1.1 to compute `n!`. The base component covers the cases when $n \leq 1$. Consider the invocation `factorial(2)`. To compute `2*factorial(1)` in the `else` statement, the computation of `factorial(2)` is suspended and `factorial` invoked with `n` = 1. When the computation of `factorial(2)` is suspended, the program state (i.e., values of local variables and value formal parameters, bindings of reference formal parameters, location in code, etc.) is saved in a recursion stack. This state is restored when the computation of `factorial(1)` completes. The invocation `factorial(1)` returns the value 1. The computation of `factorial(2)` resumes, and the expression $2 * 1$ is computed.

```
int factorial(int n)
{// Compute n!
   if (n <= 1) return 1;
   else return n * factorial(n - 1);
}
```

Program 1.29 Recursive method to compute `n!`

When computing `factorial(3)`, the computation is suspended when the `else` statement is reached so that `factorial(2)` may be computed. We have already seen how the invocation `factorial(2)` works to produce the result 2. When the computation of `factorial(2)` completes, the computation of `factorial(3)` resumes and the expression $3 * 2$ is computed.

Because of the similarity between the code of Program 1.29 and Equation 1.1, the correctness of the code follows from the correctness of the equation. ■

Example 1.2 The template function `sum` (Program 1.30) computes the sum of elements `a[0]` through `a[n-1]` (abbreviated `a[0:n-1]`). When `n` is 0, the method returns the value `0`.

Copyrighted material

```
template<class T>
T sum(T a[], int n)
{// Return sum of the numbers a[0:n-1].
   T theSum = 0;
   for (int i = 0; i < n; i++)
      theSum += a[i];
   return theSum;
}
```

Program 1.30 Add `a[0:n-1]`

Program 1.31 is a recursive method to compute the sum of the elements `a[0:n-1]`. The code for `rSum` results from a recursive formulation of the problem—when `n` is 0, the sum is 0; when `n` is greater than 0, the sum of `n` elements is the sum of the first `n` − 1 elements plus the last element. ■

```
template<class T>
T rSum(T a[], int n)
{// Return sum of numbers a[0:n - 1].
   if (n > 0)
      return rSum(a, n-1) + a[n-1];
   return 0;
}
```

Program 1.31 Recursive code to add `a[0:n-1]`

Example 1.3 [Permutations] Often we wish to examine all permutations of n distinct elements to determine the best one. For example, the permutations of the elements a, b, and c are abc, acb, bac, bca, cba, and cab. The number of permutations of n elements is $n!$.

Although developing a nonrecursive C++ function to output all permutations of n elements is quite difficult, we can develop a recursive one with modest effort. Let $E = \{e_1, \cdots, e_n\}$ denote the set of n elements whose permutations are to be generated; let E_i be the set obtained by removing element i from E; let $perm(X)$ denote the permutations of the elements in set X; and let $e_i.perm(X)$ denote the permutation list obtained by prefixing each permutation in $perm(X)$ with element e_i. For example, if $E = \{a, b, c\}$, then $E_1 = \{b, c\}$, $perm(E_1) = (bc, cb)$, and $e_1.perm(E_1) = (abc, acb)$.

For the recursion base, we use $n = 1$. Since only one permutation is possible when we have only one element, $perm(E) = (e)$ where e is the lone element in E.

Copyrighted material

When $n > 1$, $perm(E)$ is the list $e_1.perm(E_1)$ followed by $e_2.perm(E_2)$ followed by $e_3.perm(E_3)\cdots$ followed by $e_n.perm(E_n)$. This recursive definition of $perm(E)$ defines $perm(E)$ in terms of n $perm(X)$s, each of which involves an X with $n-1$ elements. Both the base component and recursive component requirements of a complete recursive definition are satisfied.

When $n = 3$ and $E = (a, b, c)$, the preceding definition of $perm(E)$ yields $perm(E) = a.perm(\{b, c\}), b.perm(\{a, c\}), c.perm(\{b, a\})$. From the recursive definition $perm(\{b, c\})$ is $b.perm(\{c\}), c.perm(\{b\})$. So $a.perm(\{b, c\})$ is $ab.perm(\{c\})$, $ac.perm(\{b\}) = ab.c, ac.b = (abc, acb)$. Proceeding in a similar way, we obtain $b.perm(\{a, c\})$ is $ba.perm(\{c\}), bc.perm(\{a\}) = ba.c, bc.a = (bac, bca)$ and $c.perm(\{b, a\})$ is $cb.perm(\{a\}), ca.perm(\{b\}) = cb.a, ca.b = (cba, cab)$. So $perm(E) = (abc, acb, bac, bca, cba, cab)$.

Notice that $a.perm(\{b, c\})$ is actually the two permutations abc and acb. a is the prefix of these permutations, and $perm(\{b, c\})$ gives their suffixes. Similarly, $ac.perm(\{b\})$ denotes permutations whose prefix is ac and whose suffixes are the permutations $perm(\{b\})$.

Program 1.32 transforms the preceding recursive definition of $perm(E)$ into a C++ function. This code outputs all permutations whose prefix is `list[0:k-1]` and whose suffixes are the permutations of `list[k:m]`. The invocation `permutations(list,0,n-1)` outputs all `n!` permutations of `list[0:n-1]`. With this invocation, `k` is `0` and `m` is `n-1`. So the prefix of the generated permutations is null, and their suffixes are the permutations of `list[0:n-1]`. When `k` equals `m`, there is only one suffix `list[m]`, and now `list[0:m]` defines a permutation that is to be output. When `k < m`, the `else` clause is executed. Let E denote the elements in `list[k:m]` and let E_i be the set obtained by removing e_i = `list[i]` from E. The first `swap` in the `for` loop has the effect of setting `list[k]` = e_i and `list[k+1:m]` = E_i. Therefore, the following call to `permutations` computes $e_i.perm(E_i)$. The second `swap` restores `list[k:m]` to its state prior to the first `swap`.

Figure 1.3 shows the progress of Program 1.32 when invoked with `k = 0`, `m = 2`, and `list[0:2] = [a, b, c]`. The figure shows the contents of `list[0:2]` immediately after each call to `permutations`, as well as immediately after the second `swap` is done following a return from `permutations`. The unshaded entries denote `list[0:k-1]`, and the shaded entries denote `list[k:m]`. Configuration numbers are shown outside the array. Each edge of Figure 1.3 is traversed twice during the execution of `permutations(list, 0, 2)`: once when a call to `permutations` is made in the `for` loop and once when a return from `permutations` is made.

We begin with configuration 1. The first `swap` done in the `for` loop has no effect on the array; configuration 2 shows the state just after `permutations` is invoked from within the `for` loop. From configuration 2 we move to configuration 3. Configuration 3 is output because, in this configuration, `k = m`. Following this output, we make a return and execute the second `swap` statement in the `for` loop. As a result, we restore configuration 2. From configuration 2 we move forward to configuration 4, and the permutation `acb` is output. Then we back up through earlier configura-

Copyrighted material

```
template<class T>
void permutations(T list[], int k, int m)
{// Generate all permutations of list[k:m].
   int i;
   if (k == m) {// list[k:m] has one permutation, output it
                  copy(list, list+m+1,
                       ostream_iterator<T>(cout, ""));
                  cout << endl;
                }
   else  // list[k:m] has more than one permutation
         // generate these recursively
         for (i = k; i <= m; i++)
         {
            swap(list[k], list[i]);
            permutations(list, k+1, m);
            swap(list[k], list[i]);
         }
}
```

Program 1.32 Recursive method for permutations

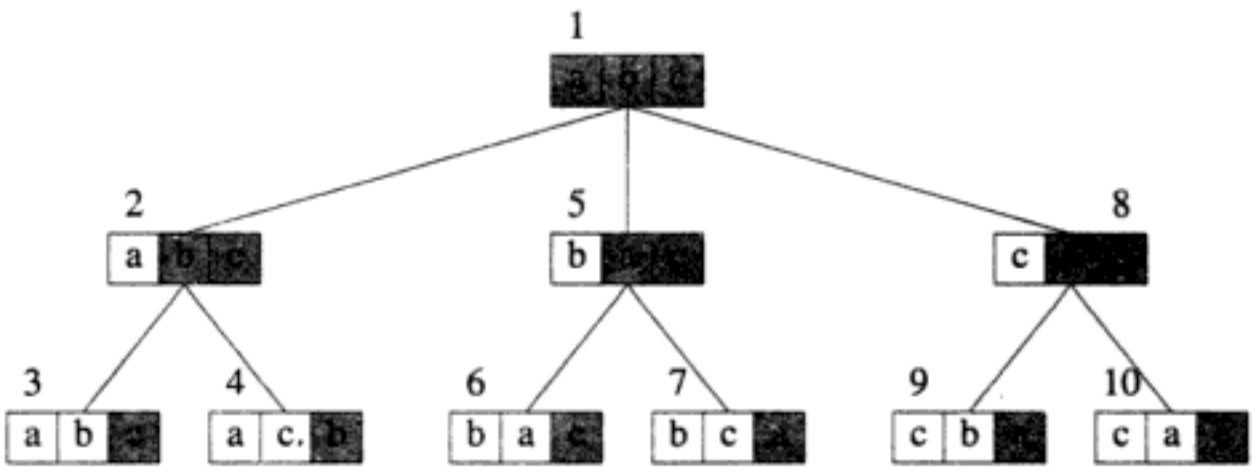


Figure 1.3 Generating the permutations of abc

tions until we can move forward again. We back up through configurations 2 and 1. From configuration 1 we move forward to configurations 5 and 6. The sequence of configurations encountered in the complete execution is 1, 2, 3, 2, 4, 2, 1, 5, 6, 5, 7, 5, 1, 8, 9, 8, 10, 8, 1. ■

Copyrighted material

EXERCISES

19. Write a nonrecursive function to compute $n!$. Test your code.

20. (a) Write a recursive function to compute the Fibonacci number F_n. Test your code.

    (b) Show that your code for part (a) computes the same F_i more than once when it is invoked to compute F_n for any $n > 2$.

    (c) Write a nonrecursive function to compute the Fibonacci number F_n. Your code should compute each Fibonacci number just once. Test your code.

21. Consider the function f, which is defined in Equation 1.4. n is a nonnegative integer.

$$f(n) = \begin{cases} n/2 & n \text{ is even} \\ f(3n+1) & n \text{ is odd} \end{cases} \tag{1.4}$$

    (a) Use Equation 1.4 to manually compute $f(5)$ and $f(7)$.

    (b) Identify the base and recursive components of the function definition. Show that repeated application of the recursive component transforms the occurrence of f on the right side to the occurrence of f in the base component.

    (c) Write a recursive C++ function to compute $f(n)$. Test your code.

    (d) Use your proof for part (b) to arrive at a nonrecursive C++ function to compute f. Your code should have no loops. Test your code.

22. [Ackermann's Function] Equation 1.5 defines Ackermann's function. In this definition, i and j are integers that are ≥ 1.

$$A(i,j) = \begin{cases} 2^j & i = 1 \text{ and } j \geq 1 \\ A(i-1,2) & i \geq 2 \text{ and } j = 1 \\ A(i-1, A(i, j-1)) & i, j \geq 2 \end{cases} \tag{1.5}$$

    (a) Use Equation 1.5 to manually compute $A(1,2)$, $A(2,1)$, and $A(2,2)$.

    (b) Identify the base and recursive components of the function definition.

    (c) Write a recursive C++ function to compute $A(i,j)$. Test your code.

Copyrighted material

23. [GCD] The **greatest common divisor (GCD)** of two nonnegative integers x and y is 0 when exactly one of them is 0. When at least one of x and y is nonzero, their GCD, $gcd(x, y)$, is the greatest integer that evenly divides both. So $gcd(0, 0) = 0$, $gcd(10, 0) = gcd(0, 10) = 10$, and $gcd(20, 30) = 10$. Euclid's GCD algorithm is a recursive algorithm that is believed to date back to 375 B.C.; it is perhaps the earliest example of a recursive algorithm. Euclid's algorithm implements the recursive definition given in Equation 1.6.

$$gcd(x, y) = \begin{cases} x & y = 0 \\ gcd(y, x \textbf{ mod } y) & y > 0 \end{cases} \tag{1.6}$$

In Equation 1.6 **mod** is the modulo operator that is implemented in C++ as the operator **%**. x **mod** y is the remainder of x/y.

(a) Use Equation 1.6 to manually compute $gcd(20, 30)$ and $gcd(112, 42)$.

(b) Identify the base and recursive components of the function definition. Show that repeated application of the recursive component transforms the occurrence of *gcd* on the right side to the occurrence of *gcd* in the base component.

(c) Write a recursive C++ function to compute $gcd(x, y)$. Test your code.

24. Write a recursive template function to determine whether element `x` is one of the elements in the array `a[0:n-1]`.

25. [Subset Generation] Write a recursive C++ function to output all subsets of n elements. For example, the subsets of the three-element set $\{a, b, c\}$ are $\{\}$ (empty set), $\{a\}$, $\{b\}$, $\{c\}$, $\{a, b\}$, $\{a, c\}$, $\{b, c\}$, and $\{a, b, c\}$. These subsets may be denoted by the 0/1 vector sequence 000, 100, 010, 001, 110, 101, 011, and 111, respectively (a 0 means that the corresponding element is not in the subset, and a 1 means that it is). So it is sufficient that your C++ code output all 0/1 sequences of length n.

26. [Gray Code] The **Hamming distance** between two vectors is the number of positions in which the vectors differ. For example, the Hamming distance between 100 and 010 is 2. A (binary) Gray code is a subset sequence in which the Hamming distance between every pair of consecutive vectors (also called codes) is 1. The three-element subset sequence given in Exercise 25 is not a Gray code. However, the three-element subset sequence 000, 100, 110, 010, 011, 111, 101, 001 is a Gray code. This sequence also has the property that the first and last vectors differ in exactly one place. In some applications of subset sequences, the cost of going from one subset to the next depends on the Hamming distance between these two subsets. In these applications, we desire a subset sequence that is a Gray code. A Gray code may be compactly

Copyrighted material

represented by giving the sequence of positions in which the vectors of the code change. For the three-element Gray code given above, the position change sequence is 1, 2, 1, 3, 1, 2, 1. Let $g(n)$ be the position change sequence for a Gray code for n elements. Equation 1.7 gives a recursive definition for $g(n)$.

$$g(n) = \begin{cases} 1 & n = 1 \\ g(n-1), n, g(n-1) & n > 1 \end{cases} \tag{1.7}$$

(a) Use Equation 1.6 to manually compute $g(4)$.

(b) Identify the base and recursive components of the function definition. Show that repeated application of the recursive component transforms both occurrences of g on the right side to the occurrence of g in the base component.

(c) Write a recursive C++ function to compute $g(n)$. Test your code.

1.8 THE STANDARD TEMPLATES LIBRARY

The C++ standard templates library (STL) is a collection of containers, adaptors, iterators, function objects (also known as functors) and algorithms. Through the judicious use of elements of the STL, the task of writing application codes is greatly simplified. In this book, we shall often solve a problem first using basic C++ language constructs to illustrate the mechanics of solving the problem. Later, we will show how the same problem can be solved more simply by using an element of the STL.

Example 1.4 [The STL algorithm `accumulate`] The STL has an algorithm `accumulate` that may be used to sum the elements in a sequence. The syntax is

```
accumulate(start, end, initialValue)
```

where `start` points to the first element to be accumulated and `end` points to one position after the last element to be accumulated. So, elements in the range `[start, end)` are accumulated. The invocation

```
accumulate(a, a+n, initialValue)
```

where `a` is a one-dimensional array, for example, returns the value

$$initialValue + \sum_{i=0}^{n-1} a[i]$$

Copyrighted material

CHAPTER 2

PERFORMANCE ANALYSIS

BIRD'S-EYE VIEW

The most important attribute of a program is correctness. A program that does not correctly perform the task it was designed to do is of little use. However, correct programs may also be of little use. This is the case, for example, when a correct program takes more memory than is available on the computer it is to run on as well as when a correct program takes more time than the user is willing to wait. We use the term *program performance* to refer to the memory and time requirements of a program. To appreciate the need for good data structures and algorithm design methods, you must be able to evaluate the performance of a program.

This chapter focuses on paper-and-pencil methods to determine the memory and time requirements of a program. The operation count and step-count approaches to estimate run time are developed, and the notions of best-case, worst-case, and average run time are introduced. A more advanced measure of run time—amortized complexity—is developed in the Web site for this book. You should not attempt to read the material on amortized complexity until you have completed Chapter 9.

Chapter 3 reviews asymptotic notations such as big oh, omega, theta, and little oh, which make up the lingua franca for performance analysis. The use of asymptotic notation often simplifies the analysis. Chapter 4 shows you how to measure the actual run time of a program by using a clocking method.

Copyrighted material

Many application codes are developed in this chapter. These applications, which will prove useful in later chapters, include

- Searching an array of elements for an element with a specified characteristic.
- Sorting an array of elements. Codes for the rank (or count) sort, selection sort, bubble sort, and insertion sort methods are developed.
- Evaluating a polynomial using Horner's rule.
- Performing matrix operations such as add, transpose, and multiply.

Copyrighted material

2.1 WHAT IS PERFORMANCE?

By the **performance of a program**, we mean the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program. One is analytical, and the other experimental. In **performance analysis** we use analytical methods, while in **performance measurement** we conduct experiments.

The **space complexity** of a program is the amount of memory it needs to run to completion. We are interested in the space complexity of a program for the following reasons:

- If the program is to be run on a multiuser computer system, then we may need to specify the amount of memory to be allocated to the program.
- For any computer system, we would like to know in advance whether or not sufficient memory is available to run the program.
- A problem might have several possible solutions with different space requirements. For instance, one C++ compiler for your computer might need only 1 MB of memory, while another might need 4 MB. The 1 MB compiler is the only choice if your computer has less than 4 MB of memory. Even users whose computers have the extra memory will prefer the smaller compiler if its capabilities are comparable to those of the bigger compiler. The smaller compiler leaves the user with more memory for other tasks.
- We can use the space complexity to estimate the size of the largest problem that a program can solve. For example, we may have a circuit simulation program that requires $10^6 + 100(c + w)$ bytes of memory to simulate circuits with c components and w wires. If the total amount of memory available is $5.01 * 10^8$ bytes, then we can simulate circuits with $c + w \leq 5,000,000$.

The **time complexity** of a program is the amount of computer time it needs to run to completion. We are interested in the time complexity of a program for the following reasons:

- Some computer systems require the user to provide an upper limit on the amount of time the program will run. Once this upper limit is reached, the program is aborted. An easy way out is to simply specify a time limit of a few thousand years. However, this solution could result in serious fiscal problems if the program runs into an infinite loop caused by some discrepancy in the data and you actually get billed for the computer time used. We would like to provide a time limit that is just slightly above the expected run time.
- The program we are developing might need to provide a satisfactory real-time response. For example, all interactive programs must provide such a response. A text editor that takes a minute to move the cursor one page down or one

Copyrighted material

page up will not be acceptable to many users. A spreadsheet program that takes several minutes to reevaluate the cells in a sheet will be satisfactory only to very patient users. A database management system that allows its users adequate time to drink two cups of coffee while it is sorting a relation will not find too much acceptance. Programs designed for interactive use must provide satisfactory real-time response. From the time complexity of the program or program module, we can decide whether or not the response time will be acceptable. If not, we need to either redesign the algorithm or give the user a faster computer.

If we have alternative ways to solve a problem, then the decision on which to use will be based primarily on the expected performance difference among these solutions. We will use some weighted measure of the space and time complexities of the alternative solutions.

EXERCISES

1. Give two more reasons why analysts are interested in the space complexity of a program.

2. Give two more reasons why analysts are interested in the time complexity of a program.

2.2 SPACE COMPLEXITY

2.2.1 Components of Space Complexity

The space needed by a program has the following components:

- *Instruction space*
  Instruction space is the space needed to store the compiled version of the program instructions.

- *Data space*
  Data space is the space needed to store all constant and variable values. Data space has two components:

  1. Space needed by constants (for example, the numbers 0 and 1 in Programs 1.29 and 1.30) and simple variables (such as `a`, `b`, and `c` in Program 1.1).
  2. Space needed by dynamically allocated objects such as arrays and class instances.

Copyrighted material

- *Environment stack space*.
  The environment stack is used to save information needed to resume execution of partially completed functions and methods. For example, if function `foo` invokes function `goo`, then we must at least save a pointer to the instruction of `foo` to be executed when `goo` terminates.

Instruction Space

The amount of instruction space that is needed depends on factors such as

- The compiler used to compile the program into machine code.
- The compiler options in effect at the time of compilation.
- The target computer.

The compiler is a very important factor in determining how much space the resulting code needs. Figure 2.1 shows three possible codes for the evaluation of `a+b+b*c+(a+b-c)/(a+b)+4`. These codes need a different amount of space, and the compiler in use determines exactly which code will be generated.

Even with the same compiler, the size of the generated program code can vary. For example, a compiler might provide the user with optimization options. These could include code-size optimization as well as execution-time optimization. In Figure 2.1, for instance, the compiler might generate the code of Figure 2.1(b) in nonoptimization mode. In optimization mode, the compiler might use the knowledge that `a+b+b*c` = `b*c+(a+b)` and generate the shorter and more time-efficient code of Figure 2.1(c). The use of the optimization mode will generally increase the time needed to compile the program.

The example of Figure 2.1 brings to light an additional contribution to the space requirements of a program. Space is needed for temporary variables such as `t1`, `t2`, $\cdots$, `t6`.

Another option that can have a significant effect on program space is the overlay option in which space is assigned only to the program module that is currently executing. When a new module is invoked, it is read in from a disk or other device, and the code for the new module overwrites the code of the old module. So program space corresponding to the size of the largest module (rather than the sum of the module sizes) is needed.

The configuration of the target computer also can affect the size of the compiled code. If the computer has floating-point hardware, then floating-point operations will translate into one machine instruction per operation. If this hardware is not installed, then code to simulate floating-point computations will be generated.

Data Space

The C++ language does not specify the space to be allocated for the various C++ data types. Figure 2.2 gives the space allocation used by most C++ compilers. The

Copyrighted material

```
LOAD  a
ADD   b
STORE t1
LOAD  b
MULT  c
STORE t2
LOAD  t1
ADD   t2
STORE t3
LOAD  a
ADD   b
SUB   c
STORE t4
LOAD  a
ADD   b
STORE t5
LOAD  t4
DIV   t5
STORE t6
LOAD  t3
ADD   t6
ADD   4
```

(a)

```
LOAD  a
ADD   b
STORE t1
SUB   c
DIV   t1
STORE t2
LOAD  b
MUL   c
STORE t3
LOAD  t1
ADD   t3
ADD   t2
ADD   4
```

(b)

```
LOAD  a
ADD   b
STORE t1
SUB   c
DIV   t1
STORE t2
LOAD  b
MUL   c
ADD   t2
ADD   t1
ADD   4
```

(c)

Figure 2.1 Three equivalent codes

data type `int` is typically assigned as many bytes (1 byte = 8 bits) as there are in a word. So, on a 4-byte per word computer, an `int` is 4 bytes long while on a 2-byte per word computer, an `int` is typically 2 bytes. We shall use the data of Figure 2.2 when computing the space required by variables and constants.

We can obtain the space requirement for a structured variable by adding up the space requirements of all its components. Similarly, we can obtain the space requirement of an array by multiplying the array size and the space needs of a single array element.

Consider the following array declarations:

```
double a[100];
int maze[rows][cols];
```

When computing the space allocated to an array, we shall be concerned only with the space allocated for the array elements. The array `a` has space for 100 elements of type `double`, each taking 8 bytes. The total space allocated to the array is therefore

Copyrighted material

Type	Space (bytes)	Range
bool	1	{true, false}
char	1	$[-128, -127]$
unsigned char	1	$[0, 255]$
short	2	$[-32768, 32767]$
unsigned short	2	$[0, 65535]$
long	4	$[-2^{31}, 2^{31} - 1]$
unsigned long	4	$[0, 2^{32} - 1]$
int	4	$[-2^{31}, 2^{31} - 1]$
unsigned int	4	$[0, 2^{32} - 1]$
float	4	$\pm 3.4E \pm 38$ (7 digits)
double	8	$\pm 1.7E \pm 308$ (15 digits)
long double	10	$\pm 1.2E \pm 4932$ (19 digits)
pointer	2	(near, _cs, _ds, _es, _ss pointers)
pointer	4	(far, huge pointers)

Figure 2.2 Space typically allocated to C++ data types on a 32-bit/word computer

800 bytes. The array `maze` has space for `rows*cols` elements of type `int`. The total space taken by this array is `4*rows*cols` bytes.

Environment Stack

Beginning performance analysts often ignore the space needed by the environment stack because they don't understand how functions (and in particular recursive ones) are invoked and what happens on termination. Each time a function is invoked the following data are saved on the environment stack:

- The return address.
- The values of all local variables and formal parameters in the function being invoked (necessary for recursive functions only).

Each time the recursive function `rSum` (Program 1.31) is invoked, whether from outside the function or from within, the current values of `a` and `n` and the program location to return to on completion are saved in the environment stack.

It is worth noting that some compilers may save the values of the local variables and formal parameters for both recursive and nonrecursive methods, while others may do so for recursive methods alone. So the compiler in use will affect the amount of space needed by the environment stack.

Copyrighted material

Summary

The space needed by a program depends on several factors. Some of these factors are not known at the time the program is conceived or written (e.g., the computer or the compiler that will be used). Until these factors have been determined, we cannot make an accurate analysis of the space requirements of a program.

We can, however, determine the contribution of those components that depend on characteristics of the problem instance to be solved. These characteristics typically include factors that determine the size of the problem instance (e.g., the number of inputs and outputs or magnitude of the numbers involved) being solved. For example, if we have a program that sorts n elements, we can determine space requirements as a function of n. For a program that adds two $n \times n$ matrices, we may use n as the instance characteristic, and for one that adds two $m \times n$ matrices, we may use m and n as the instance characteristics.

The size of the instruction space is relatively insensitive to the particular problem instance being solved. The contribution of the constants and simple variables to the data space is also independent of the characteristics of the problem instance to be solved except when the magnitude of the numbers involved becomes too large for the chosen data type. At this time we will need to either change the data type or rewrite the program using multiprecision arithmetic and then analyze the new program.

The space needed by some of the dynamically allocated memory may also be independent of the problem size. The environment stack space is generally independent of the instance characteristics unless recursive functions are in use. When recursive functions are in use, the instance characteristics will generally (but not always) affect the amount of space needed for the environment stack.

The amount of stack space needed by recursive functions is called the **recursion stack space**. For each recursive function this space depends on the space needed by the local variables and the formal parameters, the maximum depth of recursion (i.e., the maximum number of nested recursive calls), and the compiler being used. For Program 1.31 recursive calls get nested until `n` equals 0. At this time the nesting resembles Figure 2.3. The maximum depth of recursion for this program is therefore `n+1`. A smart compiler would replace a recursive call that is the last statement of a method (known as tail recursion) by equivalent iterative code. This technique could reduce, even eliminate, the recursion stack space.

We can divide the total space needed by a program into two parts:

- A fixed part that is independent of the instance characteristics. This part typically includes the instruction space (i.e., space for the code), space for simple variables, space for constants, and so on.

- A variable part that consists of the dynamically allocated space (to the extent that this space depends on the instance characteristics); and the recursion stack space (insofar as this space depends on the instance characteristics).

Copyrighted material

```
rSum(a,n)
   rSum(a,n-1)
      rSum(a,n-2)
         .
         .
         .
         rSum(a,1)
            rSum(a,0)
```

Figure 2.3 Nesting of recursive calls for Program 1.31

The space requirement of any program P may therefore be written as

$$c + S_P(\text{instance characteristics})$$

where c is a constant that denotes the fixed part of the space requirements and S_P denotes the variable component. An accurate analysis should also include the space needed by temporary variables generated during compilation (refer to Figure 2.1). This space is compiler dependent and, except in the case of recursive functions, independent of the instance characteristics. We will ignore the space needs of these compiler-generated variables.

When analyzing the space complexity of a program, we will concentrate solely on estimating S_P (instance characteristics). For any given problem we need to first determine which instance characteristics to use to measure the space requirements. The choice of instance characteristics is very problem specific, and we will resort to examples to illustrate the various possibilities. Generally speaking, our choices are limited to quantities related to the number and magnitude of the inputs to and outputs from the program. At times we also use more complex measures of the interrelationships among the data items.

2.2.2 Examples

Example 2.1 Consider Program 1.1. Before we can determine S_P, we must select the instance characteristics to be used for the analysis. Suppose we use the magnitudes of `a`, `b`, and `c` as the instance characteristic. Since `a`, `b`, and `c` are of type `int`, 4 bytes are allocated to each of the formal parameters. In addition, space is needed for the code. Neither the data space nor the instruction space is affected by the magnitudes of `a`, `b`, and `c`. Therefore, $S_{\texttt{abc}}(\text{instance characteristics}) = 0$. ■

Example 2.2 [Sequential Search] Program 2.1 examines the elements of the array `a` from left to right to see whether one of these elements equals `x`. If an element

Copyrighted material

```
template<class T>
int sequentialSearch(T a[], int n, const T& x)
{// Search the unordered list a[0:n-1] for x.
 // Return position if found; return -1 otherwise.
   int i;
   for (i = 0; i < n && a[i] != x; i++);
   if (i == n) return -1;
   else return i;
}
```

Program 2.1 Sequential search

equal to `x` is found, the function returns the position of the first occurrence of `x`. If the array has no element equal to `x`, the function of Program 2.1 returns -1.

We wish to obtain the space complexity of Program 2.1 in terms of the instance characteristic `n`. Although we need space for the formal parameters `a`, `x` and `n`, the constants 0 and -1, and the code, the space needed is independent of `n`. Therefore, $S_{\texttt{sequentialSearch}}(\texttt{n}) = 0$.

Note that the array `a` must be large enough to hold the `n` elements being searched. The space needed by this array ($n * s$ bytes, where s is the number of bytes needed by an object of type `T`). This space is, however, allocated in the function where the actual parameter corresponding to `a` is declared. As a result, we do not add the space requirements of this array into the space requirements of the function `sequentialSearch`. ■

Example 2.3 For method `sum` (Program 1.30), suppose we are interested in measuring space requirements as a function of the number of elements to be summed. Space is required for the formal parameters `a` and `n`, the local variables `i` and `theSum`, the constant 0, and the instructions. The amount of space needed does not depend on the value of `n`, so $S_{\texttt{sum}}(\texttt{n}) = 0$. ■

Example 2.4 Consider the function `rSum` (Program 1.31). As in the case of `sum`, assume that the instances are characterized by `n`. The recursion stack space includes space for the formal parameters `a` and `n` and the return address. In the case of `a`, a reference (4 bytes) is saved, while in the case of `n`, a value of type `int` (also 4 bytes) is saved on the recursion stack. If we assume that the return address also takes 4 bytes, we determine that each call to `rSum` requires 12 bytes of recursion stack space. Since the depth of recursion is `n+1`, the recursion stack space needed is `12(n+1)` bytes. So $S_{\texttt{rSum}}(\texttt{n}) = \texttt{12(n+1)}$.

Program 1.30 has a smaller space requirement than does Program 1.31. ■

Copyrighted material

Example 2.5 [Factorial] The space complexity of Program 1.29, which computes the factorial function, is analyzed as a function of `n` rather than as a function of the number of inputs (one) or outputs (one). The recursion depth is $\max\{n,1\}$. The recursion stack saves a return address (4 bytes) and the value of `n` (4 bytes) each time `factorial` is invoked. No additional space that is dependent on `n` is used, so $S_{\texttt{factorial}}(n) = 8 * \max\{n,1\}$. ■

Example 2.6 [Permutations] Program 1.32 outputs all permutations of a list of elements. With the initial invocation `permutations(list,0,n-1)`, the depth of recursion is `n`. Since each recursive call requires 20 bytes of recursion stack space (4 for each of return address, `list`, `k`, `m`, and `i`), the recursion stack space needed is `20n` bytes, so $S_{\texttt{permutations}}(n) = 20n$. ■

EXERCISES

3. Compile a sample C++ program using two C++ compilers. Is the code length the same or different?

4. List additional factors that may influence the space complexity of a program.

5. Using the data provided in Figure 2.2, determine the number of bytes needed by the following arrays:

   (a) `double y[3]`

   (b) `int matrix[10][100]`

   (c) `double x[100][5][20]`

   (d) `float z[10][10][10][5]`

   (e) `bool a[2][3][4]`

   (f) `long b[3][3][3][3]`

6. Program 2.2 gives a recursive function `rSequentialSearch` that searches the elements of the array `a[0:n-1]` for the element `x`. If `x` is found, the function returns the position of `x` in `a`. Otherwise, the function returns -1. Determine $S_P(n)$. Determine $S_{\texttt{rSequentialSearch}}(n)$.

7. Write a nonrecursive function to compute $n!$ (see Example 1.1). Compare the space requirements of your nonrecursive function and those of the recursive version (Program 1.29).

Copyrighted material

```
template<class T>
int rSequentialSearch(T a[], int n, const T& x)
{// Search the unordered list a[0:n-1] for x.
 // Return position if found; return -1 otherwise.
   if (n < 1) return -1;
   if (a[n-1] == x) return n - 1;
   return rSequentialSearch(a, n-1, x);
}
```

Program 2.2 Recursive sequential search

2.3 TIME COMPLEXITY

2.3.1 Components of Time Complexity

The time complexity of a program depends on all the factors that the space complexity depends on. A program will run faster on a computer capable of executing 10^9 instructions per second than on one that can execute only 10^7 instructions per second. The code of Figure 2.1(c) will require less execution time than the code of Figure 2.1(a). Some compilers will take less time than others to generate the corresponding computer code. Smaller problem instances will generally take less time than larger instances.

The time taken by a program P is the sum of the compile time and the run (or execution) time. The compile time does not depend on the instance characteristics. Also, we can assume that a compiled program will be run several times without recompilation. Consequently, we will concern ourselves with just the run time of a program. This run time is denoted by t_P(instance characteristics).

Because many of the factors t_P depends on are not known when a program is conceived, it is reasonable to only estimate t_P. If we knew the characteristics of the compiler to be used, we could determine the number of additions, subtractions, multiplications, divisions, compares, loads, stores, and so on that the code for P would make. Then we could obtain a formula for t_P. Letting n denote the instance characteristics, we might have an expression for $t_P(n)$ of the form

$$t_P(n) = c_a ADD(n) + c_s SUB(n) + c_m MUL(n) + c_d DIV(n) + \cdots \qquad (2.1)$$

where c_a, c_s, c_m, and c_d respectively denote the time needed for an addition, subtraction, multiplication, and division, and ADD, SUB, MUL, and DIV are functions whose value is the number of additions, subtractions, multiplications, and divisions that will be performed when the code for P is used on an instance with characteristic n.

Copyrighted material

Since the time needed for an arithmetic operation depends on the type (`int`, `float`, `double`, etc.) of the numbers in the operation, an exact formula for run time must separate the operation counts by data type. Fine-tuning Equation 2.1 in this way still does not give us an accurate formula to predict run time because today's computers do not necessarily perform arithmetic operations in sequence. For example, computers can perform an integer operation and a floating-point operation at the same time. Further, the capability to pipeline arithmetic operations and the fact that modern computers have a memory hierarchy (Section 4.5) means that the time to perform m additions isn't necessarily m times the time to perform one.

Since the analytical approach to determine the run time of a program is fraught with complications, we attempt only to estimate run time. Two more manageable approaches to estimating run time are (1) identify one or more key operations and determine the number of times these are performed and (2) determine the total number of steps executed by the program.

2.3.2 Operation Counts

One way to estimate the time complexity of a program or method is to select one or more operations, such as add, multiply, and compare, and to determine how many of each is done. The success of this method depends on our ability to identify the operations that contribute most to the time complexity. Several examples of this method follow.

Example 2.7 [Max Element] Program 1.37 returns the position of the largest element in the array `a[0:n-1]`. The time complexity of Program 1.37 can be estimated by determining the number of comparisons made between elements of the array `a`. When `n` ≤ 0, an exception is thrown and the number of comparisons is 0. When `n` $= 1$, the `for` loop is not entered. So no comparisons between elements of `a` are made. When `n` > 1, each iteration of the `for` loop makes one comparison between two elements of `a`, and the total number of element comparisons is `n-1`. Therefore, the number of element comparisons is max{`n-1`, 0}. The function `indexOfMax` performs other comparisons (for example, each iteration of the `for` loop is preceded by a comparison between `i` and `n`) that are not included in the estimate. Other operations such as initializing `indexOfMax` and incrementing the `for` loop index `i` are also not included in the estimate. If we included these other operations into our count, the count would increase by a constant factor. ■

Example 2.8 [Polynomial Evaluation] Consider the polynomial $P(x) = \sum_{i=0}^{n} c_i x^n$. If $c_n \neq 0$, $P(x)$ is a polynomial of degree n. Program 2.3 gives one way to compute $P(x)$ for a given value of x. It's time complexity can be estimated by determining the number of additions and multiplications performed inside the `for` loop. We will use the degree n as the instance characteristic. The `for` loop is entered a total of n times, and each time we enter the `for` loop one addition and two multiplications are done. (This operation count excludes the add performed each time the loop variable

Copyrighted material

i is incremented.) The number of additions is n, and the number of multiplications is $2n$.

```
template<class T>
T polyEval(T coeff[], int n, const T& x)
{// Evaluate the degree n polynomial with
 // coefficients coeff[0:n] at the point x.
   T y = 1, value = coeff[0];
   for (int i = 1; i <= n; i++)
   {// add in next term
      y *= x;
      value += y * coeff[i];
   }
   return value;
}
```

Program 2.3 Evaluating a polynomial

Horner's rule evaluates $P(x)$ as shown below.

$$P(x) = (\cdots(c_n * x + c_{n-1}) * x + c_{n-2}) * x + c_{n-3}) * x \cdots) * x + c_0$$

So $P(x) = 5 * x^3 - 4 * x^2 + x + 7$ is computed as $((5 * x - 4) * x + 1) * x + 7$. The corresponding C++ function is given in Program 2.4. Using the same measure as used for Program 2.3, we estimate the complexity of Program 2.4 as n additions and n multiplications. Since Program 2.3 performs the same number of additions but twice as many multiplications as does Program 2.4, we expect Program 2.4 to be faster. ■

Example 2.9 [Ranking] The **rank** of an element in a sequence is the number of smaller elements in the sequence plus the number of equal elements that appear to its left. For example if the sequence is given as the array a = [4, 3, 9, 3, 7], then the ranks are r = [2, 0, 4, 1, 3]. Function **rank** (Program 2.5) computes the ranks of the elements in the array a. We can estimate the complexity of **rank** by counting the number of comparisons between elements of a. These comparisons are done in the **if** statement. For each value of i, the number of element comparisons is i. So the total number of element comparisons is $1 + 2 + 3 + \cdots + n - 1 = (n-1)n/2$ (see Equation 1.3).

Note that our complexity estimate excludes the overhead associated with the **for** loops, the cost of initializing the array r, and the cost of incrementing r each time two elements of a are compared. ■

Copyrighted material

```
template<class T>
T horner(T coeff[], int n, const T& x)
{// Evaluate the degree n polynomial with
 // coefficients coeff[0:n] at the point x.
   T value = coeff[n];
   for (int i = 1; i <= n; i++)
      value = value * x + coeff[n - i];
   return value;
}
```

Program 2.4 Horner's rule for polynomial evaluation

```
template<class T>
void rank(T a[], int n, int r[])
{// Rank the n elements a[0:n-1].
 // Element ranks returned in r[0:n-1]
   for (int i = 0; i < n; i++)
      r[i] = 0;  // initialize

   // compare all element pairs
   for (int i = 1; i < n; i++)
      for (int j = 0; j < i; j++)
         if (a[j] <= a[i]) r[i]++;
         else r[j]++;
}
```

Program 2.5 Computing ranks

Example 2.10 [Rank Sort] Once the elements have been ranked using Program 2.5. they may be rearranged in increasing order so that `a[0]` $\leq$ `a[1]` $\leq \cdots \leq$ `a[n-1]` by moving elements to positions corresponding to their ranks. If we have space for an additional array `u`, we can use the function **rearrange** given in Program 2.6.

Assume that the invocation of **new** succeeds in allocating space to the array `u`. The number of element moves performed during the execution of function **rearrange** is $2n$. The complete sort requires $(n-1)n/2$ comparisons and $2n$ element moves. This method of sorting is known as **rank** or **count** sort. An alternative method to rearrange the elements is considered later (Program 2.11). This alternative method does not use an additional array such as `u`. ■

Copyrighted material

```
template<class T>
void rearrange(T a[], int n, int r[])
{// Rearrange the elements of a into sorted order
 // using an additional array u.
   T *u = new T [n]; // create additional array

   // move to correct place in u
   for (int i = 0; i < n; i++)
      u[r[i]] = a[i];

   // move back to a
   for (i = 0; i < n; i++)
      a[i] = u[i];

   delete [] u;
}
```

Program 2.6 Rearranging elements using an additional array

Example 2.11 [Selection Sort] Example 2.10 examined one way to rearrange the elements in an array `a` so that `a[0]` $\leq$ `a[1]` $\leq \cdots \leq$ `a[n-1]`. An alternative strategy is to determine the largest element and move it to `a`$[n-1]$, then determine the largest of the remaining $n-1$ elements and move it to `a`$[n-2]$, and so on. Figure 2.4(a) shows an example in which selection sort is used to sort the six-element array `a[0:5]` = [6, 5, 8, 4, 3, 1]. Shaded array positions designate the, as yet, unsorted part of the array. A heavy bar over an array position marks the maximum element, and a light bar marks the position into which the maximum element is to move.

Line 1 of the figure shows the initial configuration; the entire array is considered unsorted, the maximum element is in `a[2]`, and this maximum element is to be moved to `a[5]`. The move is accomplished by swapping the elements at the positions designated by the bars. Following the swap, we need concern ourselves only with sorting the elements `a[0:4]` because `a[5]` is known to contain the maximum element. Line 2 shows the configuration after the swap; the maximum element of `a[0:4]` is `a[0]`, and this element is to be swapped with `a[4]`. Line 3 shows the result. Line 6 shows the result following three more stages of find the max and swap. At this time the unsorted part of the array (`a[0:0]`) has a single element that is known to be less than or equal to the other elements in the array. So the entire array is sorted.

Program 2.7 gives the C++ function, `selectionSort`, which implements the above strategy. Program 1.37 gave the function `indexOfMax`. We can estimate the complexity of `selectionSort` by counting the number of element comparisons

Copyrighted material

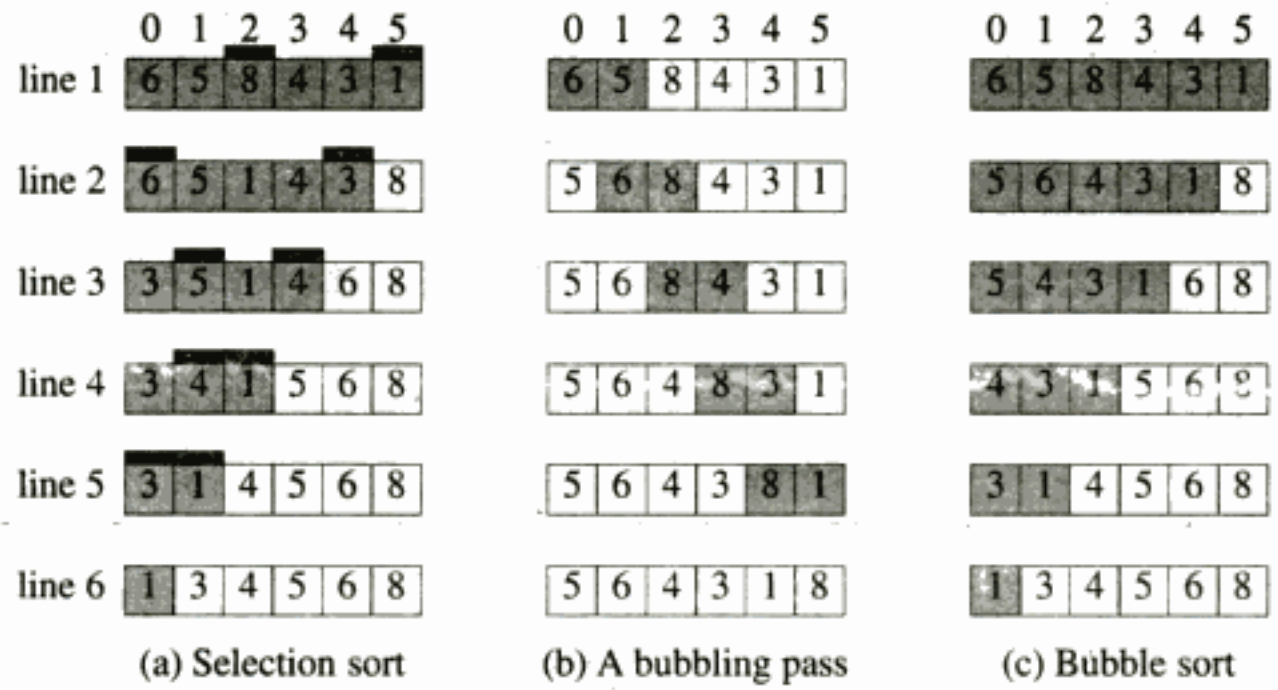


Figure 2.4 Selection and bubble sort

made. From Example 2.7 we know that each invocation `indexOfMax(a,size)`, `size` ≥ 1, results in `size-1` comparisons being made. So the total number of comparisons is $n-1+n-2+\cdots+1=(n-1)n/2$. The number of element moves is $3(n-1)$. Selection sort uses the same number of comparisons rank sort uses (Example 2.10) but requires 50 percent more element moves. We consider another version of selection sort in Example 2.16. ■

```
template<class T>
void selectionSort(T a[], int n)
{// Sort the n elements a[0:n-1].
   for (int size = n; size > 1; size--)
   {
      int j = indexOfMax(a, size);
      swap(a[j], a[size - 1]);
   }
}
```

Program 2.7 Selection sort

Example 2.12 [Bubble Sort] Bubble sort is another simple way to sort elements. This sort employs a "bubbling strategy" to get the largest element to the right. In a

Copyrighted material

bubbling pass, pairs of adjacent elements are compared. The elements are swapped in case the one on the left is greater than the one on the right. Suppose we have six integers in the order [6, 5, 8, 4, 3, 1] (see line 1 of Figure 2.4(b)). First the 6 and the 5 are compared and swapped to get the sequence shown in line 2. Next the 6 and 8 are compared, and no swap takes place. Then 8 and 4 are compared (line 3) and swapped; line 4 shows the result. The next comparison is between 8 and 3, and the two are swapped. The last comparison is between 8 and 1; these are swapped to get the configuration shown in line 6. The bubbling pass is now complete. At the end of the bubbling pass, we are assured that the largest element is in the right-most position.

The function `bubble` (Program 2.8) performs a bubbling pass over `a[0:n-1]`. The number of comparisons between pairs of elements of `a` is `n-1`.

```
template<class T>
void bubble(T a[], int n)
{// Bubble largest element in a[0:n-1] to right.
   for (int i = 0; i < n - 1; i++)
      if (a[i] > a[i+1]) swap(a[i], a[i + 1]);
}
```

Program 2.8 A bubbling pass

Since `bubble` causes the largest element to move to the right-most position, it can be used in place of `indexOfMax` in `selectionSort` (Program 2.7) to obtain a new sorting function (Program 2.9). The number of element comparisons is $(n-1)n/2$ as it is for `selectionSort`. Figure 2.4(c) shows an initial array configuration as well as the array configuration after each bubbling pass. ■

```
template<class T>
void bubbleSort(T a[], int n)
{// Sort a[0:n - 1] using bubble sort.
   for (int i = n; i > 1; i--)
      bubble(a, i);
}
```

Program 2.9 Bubble sort

Copyrighted material

2.3.3 Best, Worst, and Average Operation Counts

In the examples so far, the operation counts were nice functions of fairly simple instance characteristics like the number of inputs and/or outputs. Some of our examples would have been more complicated if we had chosen to count some other operations. For example, the number of swaps performed by `bubble` (Program 2.8) depends not only on the instance characteristic n but also on the particular values of the elements in the array `a`. The number of swaps varies from a low of 0 to a high of $n - 1$. Since the operation count isn't always uniquely determined by the chosen instance characteristics, we ask for the best, worst, and average counts. The average operation count is often quite difficult to determine. As a result, in several of the following examples we limit our analysis to determining the best and worst counts.

Example 2.13 [Sequential Search] We are interested in determining the number of comparisons between `x` and the elements of `a` during an execution of the sequential search code of Program 2.1. A natural instance characteristic to use is n. Unfortunately, the number of comparisons isn't uniquely determined by n. For example, if $n = 100$ and `x = a[0]`, then only 1 comparison is made. However, if `x` isn't equal to any of the `a[]`s, then 100 comparisons are made.

A search is **successful** when `x` is one of the `a[]`s. All other searches are **unsuccessful**. Whenever we have an unsuccessful search, the number of comparisons is n. For successful searches the best comparison count is 1, and the worst is n. For the average count assume that all array elements are distinct and that each is searched for with equal frequency. The average count for a successful search is

$$\frac{1}{n}\sum_{i=1}^{n} i = (n+1)/2$$

■

Example 2.14 [Insertion into a Sorted Array] You are to insert a new element into a sorted array so that the result is also a sorted array. For example, when you insert 3 into `a[0:4]` = [2,4,6,8,9], the result is `a[0:5]` = [2,3,4,6,8,9]. The insertion may be done by beginning at the right end and successively moving array elements one position right until we find the location for the new element. Figure 2.5(a) illustrates the process. In our example we moved 9, 8, 6, and 4 one position right and inserted 3 into the now-vacant spot `a[1]`.

Program 2.10 implements the above strategy to insert an element `x` into a sorted array `a[0:n-1]`.

We wish to determine the number of comparisons made between `x` and the elements of `a`. The natural instance characteristic to use is the number `n` of elements initially in `a`. The best or minimum number of comparisons is 1, which happens when the new element `x` is to be inserted at the right end. The maximum number of comparisons is `n`, which happens when `x` is to be inserted at the left end. For

Copyrighted material

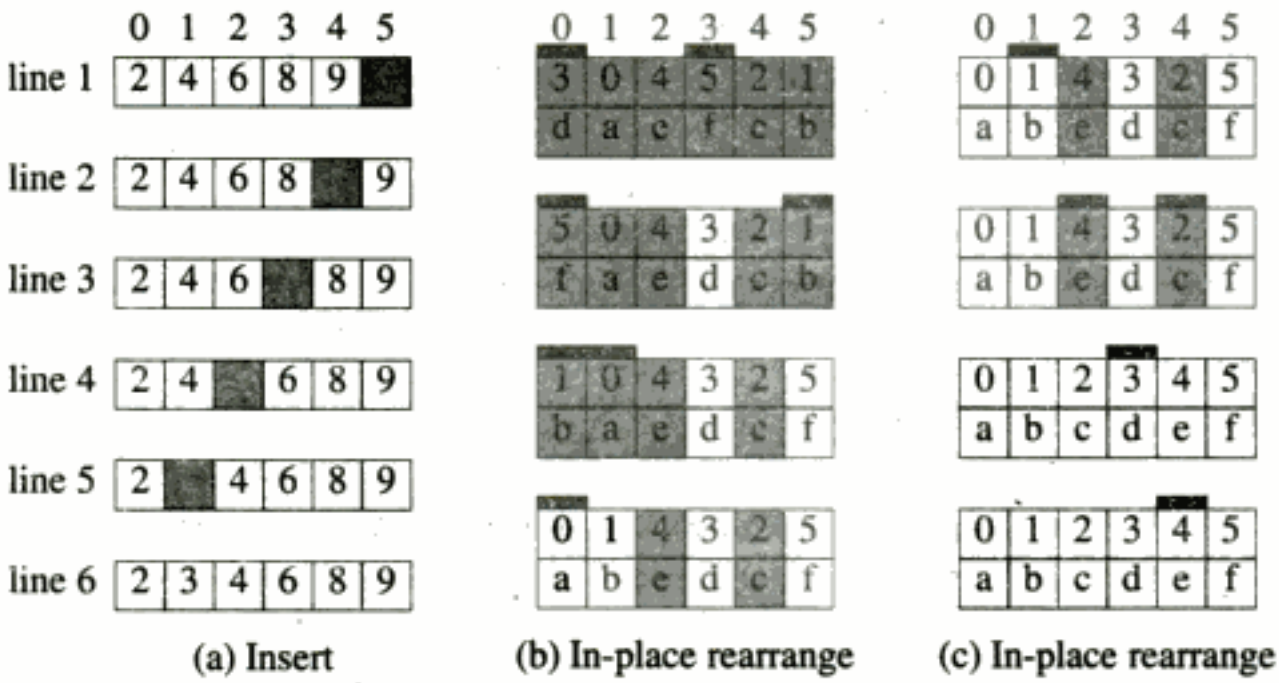


Figure 2.5 Insert and rearrange

```
template<class T>
void insert(T a[], int& n, const T& x)
{// Insert x into the sorted array a[0:n-1].
 // Assume a is of size > n
   int i;
   for (i = n-1; i >= 0 && x < a[i]; i--)
      a[i+1] = a[i];
   a[i+1] = x;
   n++; // one element added to a
}
```

Program 2.10 Inserting into a sorted array

the average assume that `x` has an equal chance of being inserted into any of the possible `n+1` positions. If `x` is eventually inserted into position `i+1` of `a`, $i \geq 0$, then the number of comparisons is `n-i`. If `x` is inserted into `a[0]`, the number of comparisons is `n`. So the average count is

$$\frac{1}{n+1}\left(\sum_{i=0}^{n-1}(n-i)+n\right) \quad = \quad \frac{1}{n+1}\left(\sum_{j=1}^{n} j+n\right)$$

Copyrighted material

$$= \frac{1}{n+1}\left(\frac{n(n+1)}{2} + n\right)$$
$$= \frac{n}{2} + \frac{n}{n+1}$$

So the average count is almost 1 more than half the worst-case count. ■

Example 2.15 [Rank Sort Revisited] Suppose the elements of an array have been ranked using method `rank` (Program 2.5, Example 2.9). We can perform an in-place rearrangement of elements into sorted order by examining the array positions one at a time beginning with position 0. If we are currently examining position `i` and `r[i]` = `i`, then we may advance to the next position. If `r[i]` ≠ `i`, then we swap the elements in positions `i` and `r[i]`. This swap moves the element previously in position `i` into its correct sorted position. The swap operation is repeated at position `i` until the element that belongs in position `i` in the sorted order is swapped into position `i`. Then we advance `i` to the next position.

Parts (b) and (c) of Figure 2.5 show how the above rearrangement strategy works. The initial array is `a[0:5]` = [d,a,e,f,c,b]. Element ranks are shown above the elements. So the initial rank array is `r[0:5]` = [3,0,4,5,2,1]. We begin at array position 0. Since `r[0]` ≠ `0`, `a[0]` and `a[r[0]]` = `a[3]` are to be swapped. The configurations of parts (b) and (c) of Figure 2.5 have a heavy bar above the position `a[i]` being examined (initially, `i` = `0`) and a light bar above the position `a[r[i]]` where `a[i]` is to move to. When `r[a[i]]` = `i`, the figure has only a heavy bar above `a[i]`. Shaded array positions denote elements that are not in their proper place (i.e., elements with `r[i]` ≠ `i`).

We begin with `i` = `0` and swap elements `a[i]` and `a[r[i]]` = `a[3]`; `r[0]` and `r[3]` are also swapped. This process results in the second configuration. Notice that `a[3]` now contains the proper element and `r[3]` = `3`. Next elements `a[0]` and `a[r[0]]` = `a[5])` together with their ranks are swapped to get the third configuration of Figure 2.5(b). When `a[0]` and `a[r[0]]` = `a[1]` (and their ranks) are swapped, we get the fourth configuration. Now `r[0]` = `0`, and we increment `i` to the next position 1. The new configuration is shown at the top of Figure 2.5(c). Since `r[i]` = `r[1]` = `1`, we advance `i` to the next position 2 (see the second configuration of Figure 2.5(c)). Now `a[i]` = `a[2]` and `a[r[2]]` = `a[4]` (as well as their ranks) are swapped. Following the swap, `r[2]` = `2`. Even though the rearrangement is complete at this time, our code will not be able to detect this and we continue to advance `i` to the right, making sure that each element is in its proper position. So `i` is advanced to the next position 3 (see the third configuration of Figure 2.5(c)). Then `i` is advanced to positions 4 and 5.

Program 2.11 gives the in-place rearrangement function `rearrange`.

The number of swaps performed varies from a low of 0 (when the elements are initially in sorted order) to a high of $2(n-1)$. Notice that each swap involving the `a[]`s moves at least one element into its sorted position (i.e., element `a[i]`).

Copyrighted material

```
template<class T>
void rearrange(T a[], int n, int r[])
{// In-place rearrangement into sorted order.
   for (int i = 0; i < n; i++)
      // get proper element to a[i]
      while (r[i] != i)
      {
         int t = r[i];
         swap(a[i], a[t]);
         swap(r[i], r[t]);
      }
}
```

Program 2.11 In-place rearrangement of elements

So after at most $n-1$ swaps, all n elements must be in sorted order. Exercise 20 establishes that this many element swaps may be needed on certain inputs. Hence the number of swaps is 0 in the best case and $2(n-1)$ in the worst case (includes rank swaps). When this in-place rearrangement function is used in place of the one in Program 2.6, the worst-case execution time increases because we need more element moves (each swap requires three moves). However, the space requirements are reduced. ■

Example 2.16 [Selection Sort Revisited] A shortcoming of the selection sort code of Program 2.7 is that it continues to work even after the elements have been sorted. For example, the `for` loop iterates $n-1$ times, even though the array may be sorted after the second iteration. To eliminate the unnecessary iterations, during the scan for the largest element we can check to see whether the array is already sorted. Program 2.12 gives the resulting selection sort function. Here we have incorporated the loop to find the largest element directly into `selectionSort`, rather than write it as a separate method.

Figure 2.6(a) shows the progress of Program 2.12 when started with `a[0:5] = [6,5,4,3,2,1]`. In the first iteration of the outer `for` loop, `size = 6` and the line `sorted = false` is executed when `i` = 1, 2, 3, 4, and 5. So following the swap of `a[0]` and `a[5]`, which results in the configuration of line 2, the outer `for` loop is reentered with `size` = 5. This time the line `sorted = false` is executed when `i` = 2, 3, and 4. Therefore, following the swap of `a[1]` and `a[4]`, the outer `for` loop is reentered; this time `size = 4`. Now the line `sorted = false` is executed when `i = 4`, and following the swap of `a[2]` and `a[3]`, the outer `for` loop is reentered. This time we are working with the array configuration of line 4; the line `sorted = false` is not executed, and execution of the outer `for` loop terminates. On the initial data of line 1 of Figure 2.6(a), the early terminating version makes one less

Copyrighted material

```
template<class T>
void selectionSort(T a[], int n)
{// Early-terminating version of selection sort.
  bool sorted = false;
  for (int size = n; !sorted && (size > 1); size--)
    {
      int indexOfMax = 0;
      sorted = true;
      // find largest
      for (int i = 1; i < size; i++)
         if (a[indexOfMax] <= a[i]) indexOfMax = i;
         else sorted = false; // out of order
      swap(a[indexOfMax], a[size - 1]);
    }
}
```

Program 2.12 Early-terminating version of selection sort

pass than it does when started with the data shown in line 1 of Figure 2.4(a).

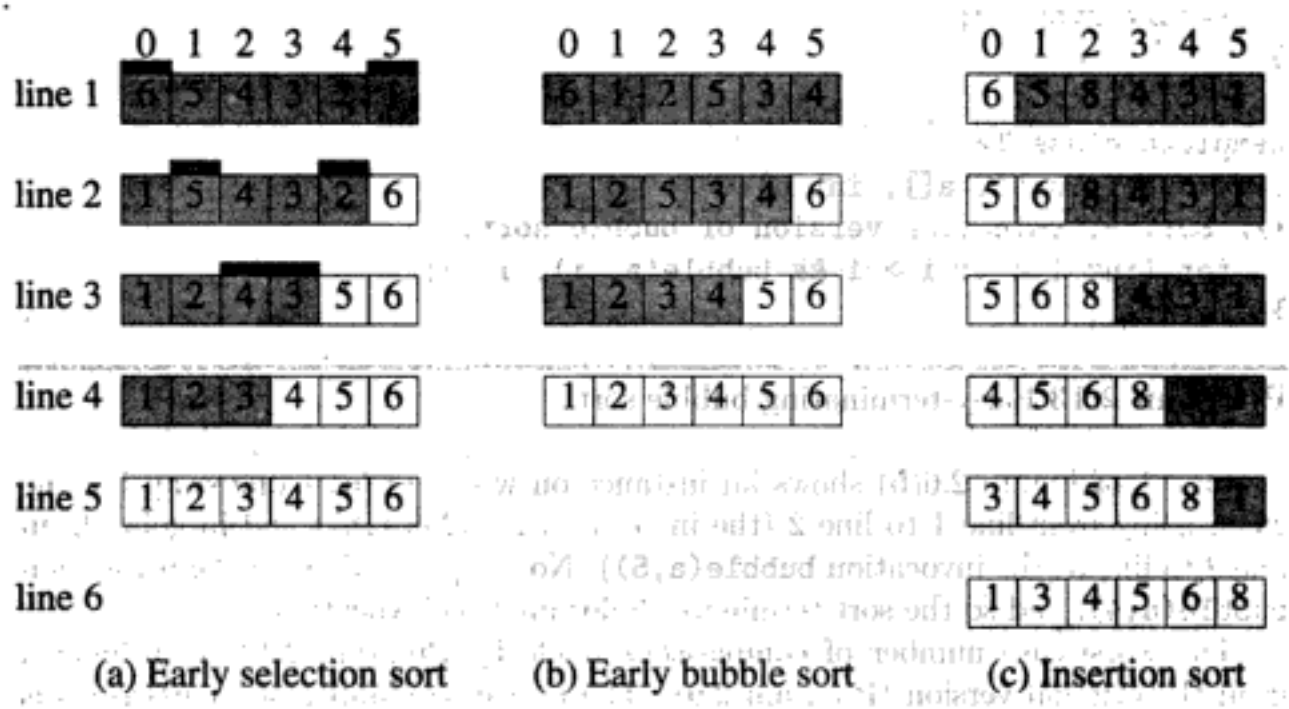


Figure 2.6 Sorting examples

The best case for the early-terminating version of selection sort arises when the

Copyrighted material

array `a` is sorted to begin with. Now the outer `for` loop iterates just once, and the number of comparisons between elements of `a` is $n-1$. In the worst case the outer `for` loop is iterated until `size` $= 1$ and the number of comparisons is $(n-1)n/2$. The best- and worst-case number of swaps remains the same as for Program 2.7. Notice that in the worst case we expect the early-terminating version to be slightly slower because of the additional work to maintain the variable `sorted`. ■

Example 2.17 [Bubble Sort Revisited] As in the case of selection sort, we can devise an early-terminating version of bubble sort. If a bubbling pass results in no swaps, then the array is in sorted order and no further bubbling passes are necessary. Program 2.13 gives the early-terminating version of bubble sort.

```
template<class T>
bool bubble(T a[], int n)
{// Bubble largest element in a[0:n-1] to right.
   bool swapped = false; // no swaps so far
   for (int i = 0; i < n - 1; i++)
      if (a[i] > a[i+1])
      {
         swap(a[i], a[i + 1]);
         swapped = true; // swap was done
      }
   return swapped;
}

template<class T>
void bubbleSort(T a[], int n)
{// Early-terminating version of bubble sort.
   for (int i = n; i > 1 && bubble(a, i); i--);
}
```

Program 2.13 Early-terminating bubble sort

Line 1 of Figure 2.6(b) shows an instance on which at least one swap is done when going from line 1 to line 2 (the invocation `bubble(a,6)`) and in going from line 2 to line 3 (the invocation `bubble(a,5)`). No swaps are done in the invocation `bubble(a,4)`, and so the sort terminates following this invocation.

The worst-case number of comparisons made by Program 2.13 is unchanged from the original version (Program 2.9). The best-case number of comparisons is $n-1$. ■

Example 2.18 [Insertion Sort] Program 2.10 can be used as the basis of a method to sort n elements. Since an array with one element is a sorted array, we start with

Copyrighted material

an array that contains just the first of the n elements to be sorted. By inserting the second element into this one-element array, we get a sorted array of size 2. The insertion of the third element yields a sorted array of size 3. Continuing in this way, we obtain a sorted array of size n.

Line 1 of Figure 2.6(c) shows the unsorted array `a[0:5]`. We start with a sorted segment `a[0:0]`; the remaining elements `a[1:5]` define the unsorted segment. The unsorted segment is the shaded segment in Figure 2.6(c). First `a[1]` is inserted into the sorted segment `a[0:0]` to get the configuration of line 2; `a[0:1]` is now the sorted segment, and `a[2:5]` is the unsorted segment. Next `a[2]` is inserted into the sorted segment, and we get line 3 of the figure. `a[0:2]` becomes the sorted segment, and `a[3:5]` is the unsorted segment. After three more inserts, the entire array is sorted.

Function `insertionSort` (Program 2.14) implements this strategy. We have rewritten function `insert` for this application, as the original version (Program 2.10) performs some unnecessary operations. Actually, we could have embedded the code of the new `insert` function directly into the sort function to get the insertion sort version of Program 2.15. Equivalently, we could make `insert` an `inline` function.

```
template<class T>
void insert(T a[], int n, const T& x)
{// Insert x into the sorted array a[0:n-1].
   int i;
   for (i = n-1; i >= 0 && x < a[i]; i--)
      a[i+1] = a[i];
   a[i+1] = x;
}

template<class T>
void insertionSort(T a[], int n)
{// Sort a[0:n-1] using the insertion sort method.
   for (int i = 1; i < n; i++)
   {
      T t = a[i];
      insert(a, i, t);
   }
}
```

Program 2.14 Insertion sort

Both versions of insertion sort perform the same number of comparisons. In the best case the number of comparisons is $n-1$, and in the worst case it is $(n-1)n/2$. ■

Copyrighted material

```
template<class T>
void insertionSort(T a[], int n)
{// Sort a[0:n-1] using the insertion sort method.
   for (int i = 1; i < n; i++)
   {// insert a[i] into a[0:i-1]
      T t = a[i];
      int j;
      for (j = i-1; j >= 0 && t < a[j]; j--)
         a[j+1] = a[j];
      a[j+1] = t;
   }
}
```

Program 2.15 Another version of insertion sort

2.3.4 Step Counts

As noted in some of the examples on operation counts, the operation-count method of estimating time complexity omits accounting for the time spent on all but the chosen operations. In the **step-count** method, we attempt to account for the time spent in all parts of the program/method. As was the case for operation counts, the step count is a function of the instance characteristics. Although any specific instance may have several characteristics (e.g., the number of inputs, the number of outputs, the magnitudes of the inputs and outputs), the number of steps is computed as a function of some subset of these. Usually we choose the characteristics that are of interest to us. For example, we might wish to know how the computing (or run) time (i.e., time complexity) increases as the number of inputs increases. In this case the number of steps will be computed as a function of the number of inputs alone. For a different program we might want to determine how the computing time increases as the magnitude of one of the inputs increases. In this case the number of steps will be computed as a function of the magnitude of this input alone. Thus before the step count of a program can be determined, we need to know exactly which characteristics of the problem instance are to be used. These characteristics define not only the variables in the expression for the step count but also how much computing can be counted as a single step.

After the relevant instance characteristics have been selected, we can define a step. A **step** is any computation unit that is independent of the selected characteristics. Thus 10 additions can be one step; 100 multiplications can also be one step; but n additions, where n is an instance characteristic, cannot be one step. Nor can $m/2$ additions or $p+q$ subtractions, where m, p, and q are instance characteristics, be counted as one step.

Copyrighted material

Definition 2.1 *A* **program step** *is loosely defined to be a syntactically or semantically meaningful segment of a program for which the execution time is independent of the instance characteristics.* ■

The amount of computing represented by one program step may be different from that represented by another. For example, the entire statement

```
return a+b+b*c+(a+b-c)/(a+b)+4;
```

can be regarded as a single step if its execution time is independent of the instance characteristics we are using. We may also count a statement such as

```
x = y;
```

as a single step.

We can determine the number of steps that a program or method takes to complete its task by creating a global variable `stepCount` with initial value 0. Next we introduce statements into the program to increment `stepCount` by the appropriate amount. Therefore, each time a statement in the original program or method is executed, `stepCount` is incremented by the step count of that statement. The value of `stepCount` when the program or method terminates is the number of steps taken.

Example 2.19 When statements to increment `stepCount` are introduced into Program 1.30, the result is Program 2.16. The change in the value of `stepCount` by the time this program terminates is the number of steps executed by Program 1.30.

```
template<class T>
T sum(T a[], int n)
{// Return sum of numbers a[0:n - 1].
   T theSum = 0;
   stepCount++;     // for theSum = 0
   for (int i = 0; i < n; i++)
   {
      stepCount++; // for the for statement
      theSum += a[i];
      stepCount++; // for assignment
   }
   stepCount++;    // for last execution of for statement
   stepCount++;    // for return
   return theSum;
}
```

Program 2.16 Counting steps in Program 1.30

Copyrighted material

Program 2.17, which is a simplified version of Program 2.16, determines only the change in the value of **stepCount**. We see that for every initial value of **stepCount**, both Programs 2.16 and 2.17 compute the same final value for **stepCount**. In the **for** loop of Program 2.17, the value of **stepCount** increases by a total of **2n**. If **stepCount** is 0 to start with, then **stepCount** will be **2n+3** on termination. Therefore, each invocation of **sum** (Program 1.30) executes a total of **2n+3** steps. ■

```
template<class T>
T sum(T a[], int n)
{// Return sum of numbers a[0:n - 1].
   for (int i = 0; i < n; i++)
      stepCount += 2;
   stepCount += 3;
   return 0;
}
```

Program 2.17 Simplified version of Program 2.16

Example 2.20 When we introduce statements to increment **stepCount** into function **rSum** (Program 1.31), we obtain Program 2.18. Note that since Program 1.31 is a recursive function, it requires space for the recursion stack. Thus the function may fail to complete its task for lack of sufficient memory for the recursion stack. For the step-count analysis, we will assume that sufficient memory is available for the function **rSum** to successfully complete its task.

```
template<class T>
T rSum(T a[], int n)
{// Return sum of numbers a[0:n - 1].
   stepCount++; // for if conditional
   if (n > 0) {stepCount++; // for return and rSum invocation
               return rSum(a, n-1) + a[n-1];}
   stepCount++; // for return
   return 0;
}
```

Program 2.18 Counting steps in Program 1.31

Let $t_{\texttt{rSum}}(\texttt{n})$ be the increase in the value of **stepCount** between the time **rSum** is initially invoked and the time it terminates. We see that $t_{\texttt{rSum}}(0) = 2$. When **n**

Copyrighted material

> 0, stepCount increases by 2 plus whatever increase results from the invocation of rSum from within the **then** clause. From the definition of $t_{\texttt{rSum}}$, it follows that this additional increase is $t_{\texttt{rSum}}(\texttt{n-1})$. So if the value of stepCount is 0 initially, its value at the time of termination is $2 + t_{\texttt{rSum}}(\texttt{n-1})$, $\texttt{n} > 0$.

When analyzing a recursive program for its step count, we often obtain a recursive formula for the step count (such as $t_{\texttt{rSum}}(\texttt{n}) = 2 + t_{\texttt{rSum}}(\texttt{n-1})$, $\texttt{n} > 0$ and $t_{\texttt{rSum}}(0) = 2$). This recursive formula is referred to as a **recurrence equation** (or simply as a **recurrence**). We can solve this recurrence by repeatedly substituting for $t_{\texttt{rSum}}$ as shown:

$$\begin{aligned} t_{\texttt{rSum}}(n) &= 2 + t_{\texttt{rSum}}(n-1) \\ &= 2 + 2 + t_{\texttt{rSum}}(n-2) \\ &= 4 + t_{\texttt{rSum}}(n-2) \\ &\vdots \\ &= 2n + t_{\texttt{rSum}}(0) \\ &= 2n + 2, \quad n \geq 0 \end{aligned}$$

So the step count for function rSum (Program 1.31) is 2n+2. ■

Comparing the step counts of Programs 1.30 and 1.31, we see that the count for Program 1.31 is less than that for Program 1.30. However, we cannot conclude that Program 1.30 is slower than Program 1.31, because a step doesn't correspond to a definite time unit. A step of rSum may take more time than a step of sum, so rSum might well be (and we expect it to be) slower than sum.

The step count is useful in that it tells us how the run time for a program changes with changes in the instance characteristics. From the step count for sum, we see that if n is doubled, the run time will also double (approximately); if n increases by a factor of 10, we expect the run time to increase by a factor of 10; and so on. So we expect the run time to grow *linearly* in n.

Rather than introduce statements to increment stepCount, we can build a table in which we list the total number of steps that each statement contributes to stepCount. We can arrive at this figure by first determining the number of steps per execution (s/e) of the statement and the total number of times (i.e., frequency) each statement is executed. Combining these two quantities gives us the total contribution of each statement. We can then add the contributions of all statements to obtain the step count for the entire program. This approach to obtaining the step count is called **profiling**.

The s/e of a statement is the amount by which stepCount *changes as a result of the execution of that statement.* An important difference between the step count of a statement and its s/e is illustrated by the following example. The statement

Copyrighted material

```
x = sum(a,m);
```

has a step count of 1, while the total change in `stepCount` resulting from the execution of this statement is actually 1 plus the change resulting from the invocation of `sum` (i.e., `2m+3`). Therefore, the s/e of the above statement is `1+2m+3 = 2m+4`.

Figure 2.7 lists the number of steps per execution and the frequency of each of the statements in function `sum` (Program 1.30). The total number of steps required by the program is `2n+3`. Note that the frequency of the `for` statement is `n+1` and not `n` because `i` has to be incremented to `n` before the `for` loop can terminate.

Statement	s/e	Frequency	Total steps
`T sum(T a[], int n)`	0	0	0
`{`	0	0	0
`  T theSum = 0;`	1	1	1
`  for (int i = 0; i < n; i++)`	1	$n+1$	$n+1$
`    theSum += a[i];`	1	n	n
`  return theSum;`	1	1	1
`}`	0	0	0
Total			$2n+3$

Figure 2.7 Step count for Program 1.30

Program 2.19 transposes a `rows` × `rows` matrix `a[0:rows-1][0:rows-1]`. Recall that `b` is the transpose of `a` iff (if and only if) `b[i][j] = a[j][i]` for all `i` and `j`.

```
template<class T>
void transpose(T **a, int rows)
{// In-place transpose of matrix a[0:rows-1][0:rows-1].
   for (int i = 0; i < rows; i++)
      for (int j = i+1;  j < rows; j++)
         swap(a[i][j], a[j][i]);
}
```

Program 2.19 Matrix transpose

Figure 2.8 gives the step-count table. Let us derive the frequency of the second `for` statement. For each value of `i`, this statement is executed `rows-i` times. So its frequency is

$$\sum_{i=0}^{rows-1} (rows - i) = \sum_{q=1}^{rows} q = rows(rows+1)/2$$

Copyrighted material

Statement	s/e	Frequency	Total steps
`void transpose(T **a, int rows)`	0	0	0
`{`	0	0	0
`  for (int i = 0; i < rows; i++)`	1	$rows + 1$	$rows + 1$
`    for (int j = i+1; j < rows; j++)`	1	$rows(rows + 1)/2$	$rows(rows + 1)/2$
`      swap(a[i][j], a[j][i]);`	1	$rows(rows - 1)/2$	$rows(rows - 1)/2$
`}`	0	0	0
Total			$rows^2 + rows + 1$

Figure 2.8 Step count for Program 2.19

The frequency for the **swap** statement is

$$\sum_{i=0}^{rows-1} (rows - i - 1) = \sum_{q=0}^{rows-1} q = rows(rows - 1)/2$$

In some cases the number of steps per execution of a statement varies from one execution to the next, for example, for the assignment statement of function **inef** (Program 2.20). Function **inef** is a very inefficient way to compute the prefix sums $b[j]$.

$$b[j] = \sum_{i=0}^{j} a[i] \text{ for } j = 0, 1, \cdots, n - 1$$

```
template <class T>
void inef(T a[], T b[], int n)
{// Compute prefix sums.
   for (int j = 0; j < n; j++)
      b[j] = sum(a, j + 1);
}
```

Program 2.20 Inefficient prefix sums

The step count for `sum(a,m)` has already been determined to be `2m+3` (see Example 2.19). Therefore, the number of steps per execution of the assignment statement `b[j] = sum(a, j + 1)` of `inef` is `2j+6`. To arrive at this step count, we have added 1 to the step count of `sum` to account for the cost of invoking the function `sum` and

Copyrighted material

of assigning the function value to `b[j]`. The frequency of this assignment statement is n. But the total number of steps resulting from this statement is not `(2j+6)`n. Instead, the number of steps is

$$\sum_{j=0}^{n-1}(2j+6) = n(n+5)$$

Figure 2.9 gives the complete analysis for this function.

Statement	s/e	Frequency	Total steps
`void inef(T a[], T b[], int n)`	0	0	0
`{`	0	0	0
`  for (int j = 0; j < n; j++)`	1	$n+1$	$n+1$
`    b[j] = sum(a, j + 1);`	$2j+6$	n	$n(n+5)$
`}`	0	0	0
Total			n^2+6n+1

Figure 2.9 Step count for Program 2.20

The notions of best, worst, and average operation counts are easily extended to the case of step counts. Examples 2.21 and 2.22 illustrate these notions.

Example 2.21 [Sequential Search] Figures 2.10 and 2.11 show the best- and worst-case step-count analyses for function `sequentialSearch` (Program 2.1).

Statement	s/e	Frequency	Total steps
`int sequentialSearch(T a[], int n, const T& x)`	0	0	0
`{`	0	0	0
`  int i;`	1	1	1
`  for (i = 0; i < n && a[i] != x; i++);`	1	1	1
`  if (i == n) return -1;`	1	1	1
`  else return i;`	1	1	1
`}`	0	0	0
Total			4

Figure 2.10 Best-case step count for Program 2.1

For the average step-count analysis for a successful search, we assume that the n values in `a` are distinct and that in a successful search, `x` has an equal probability of being any one of these values. Under these assumptions the average step count for a successful search is the sum of the step counts for the n possible successful searches divided by n. To obtain this average, we first obtain the step count for the case `x = a[j]` where `j` is in the range $[0, n-1]$ (see Figure 2.12).

Now we obtain the average step count for successful searches:

Copyrighted material

Statement	s/e	Frequency	Total steps
`int sequentialSearch(T a[], int n, const T& x)`	0	0	0
`{`	0	0	0
`  int i;`	1	1	1
`  for (i = 0; i < n && a[i] != x; i++);`	1	$n+1$	$n+1$
`  if (i == n) return -1;`	1	1	1
`  else return i;`	1	0	0
`}`	0	0	0
Total			$n+3$

Figure 2.11 Worst-case step count for Program 2.1

Statement	s/e	Frequency	Total steps
`int sequentialSearch(T a[], int n, const T& x)`	0	0	0
`{`	0	0	0
`  int i;`	1	1	1
`  for (i = 0; i < n && a[i] != x; i++);`	1	$j+1$	$j+1$
`  if (i == n) return -1;`	1	1	1
`  else return i;`	1	1	1
`}`	0	0	0
Total			$j+4$

Figure 2.12 Step count for Program 2.1 when `x = a[j]`

$$\frac{1}{n}\sum_{j=0}^{n-1}(j+4) = (n+7)/2$$

This value is a little more than half the step count for an unsuccessful search.

Now suppose that successful searches occur only 80 percent of the time and that each `a[i]` still has the same probability of being searched for. The average step count for `sequentialSearch` is

.8 * (average count for successful searches) + .2 * (count for an unsuccessful search)
$= .8(n+7)/2 + .2(n+3)$
$= .6n + 3.4$ ■

Example 2.22 [Insertion into a Sorted Array] The best- and worst-case step counts for function `insert` (Program 2.10) are obtained in Figures 2.13 and 2.14, respectively.

Copyrighted material

Statement	s/e	Frequency	Total steps
`void insert(T a[], int& n, const T& x)`	0	0	0
`{`	0	0	0
`  int i;`	1	1	1
`  for (i = n - 1; i >= 0 && x < a[i]; i--)`	1	1	1
`    a[i+1] = a[i];`	1	0	0
`  a[i+1] = x;`	1	1	1
`  n++; // one element added to a`	1	1	1
`}`	0	0	0
Total			4

Figure 2.13 Best-case step count for Program 2.10

Statement	s/e	Frequency	Total steps
`void insert(T a[], int& n, const T& x)`	0	0	0
`{`	0	0	0
`  int i;`	1	1	1
`  for (i = n - 1; i >= 0 && x < a[i]; i--)`	1	$n+1$	$n+1$
`    a[i+1] = a[i];`	1	n	n
`  a[i+1] = x;`	1	1	1
`  n++; // one element added to a`	1	1	1
`}`	0	0	0
Total			$2n+4$

Figure 2.14 Worst-case step count for Program 2.10

For the average step count, assume that `x` has an equal chance of being inserted into any of the possible `n+1` positions. If `x` is eventually inserted into position j, $j \geq 0$, then the step count is `2n-2j+4`. So the average count is

$$\begin{aligned}
\frac{1}{n+1}\sum_{j=0}^{n}(2n-2j+4) &= \frac{1}{n+1}[2\sum_{j=0}^{n}(n-j)+\sum_{j=0}^{n}4] \\
&= \frac{1}{n+1}[2\sum_{k=0}^{n}k+4(n+1)] \\
&= \frac{1}{n+1}[n(n+1)+4(n+1)] \\
&= \frac{(n+4)(n+1)}{n+1} \\
&= n+4
\end{aligned}$$

Copyrighted material

The average count is 2 more than half the worst-case count. ■

EXERCISES

8. According to the analysis in Example 2.8, Program 2.3 makes four additions and eight multiplications when evaluating the polynomial $3x^4+4x^3+5x^2+6x+7$, and Program 2.4 makes four additions and four multiplications. Identify these additions and multiplications for the case $x = 2$. Do this by showing the precise numbers that are being added or multiplied.

9. Give the rank array `r` for the case when `a[0:8]` = [3, 2, 6, 5, 9, 4, 7, 1, 8] (see Example 2.9).

10. Consider the selection sort function of Program 2.7. Draw a figure similar to Figure 2.4(a) for the case when `a[0:6]` = [3, 2, 6, 5, 9, 4, 8].

11. Consider the bubbling pass function of Program 2.8. Draw a figure similar to Figure 2.4(b) for the case when `a[0:6]` = [3, 2, 6, 5, 9, 4, 8].

12. For the bubble sort function of Program 2.9, draw a figure similar to Figure 2.4(c) for the case when `a[0:6]` = [3, 2, 6, 5, 9, 4, 8].

13. Suppose that we are to insert 3 into the sorted array `a[0:6]` = [1, 2, 4, 6, 7, 8, 9]. Draw a figure similar to Figure 2.5(a). Your figure should show the progress of Program 2.10.

14. The array `a[0:8]` = [g, h, i, f, c, a, d, b, e] is to be sorted using a rank sort. The ranks are determined to be `r[0:8]` = [6, 7, 8, 5, 2, 0, 3, 1, 4]. Draw a figure similar to Figures 2.5(b) and (c) to show the progress of the in-place rearrangement function of Program 2.11.

15. (a) Suppose that the array `a[0:9]` = [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] is sorted using the early-terminating version of selection sort (Program 2.12). Draw a figure similar to Figure 2.6(a) to show the progress of the sort.

    (b) Do part (a) for the case when `a[0:8]` = [8, 4, 5, 2, 1, 6, 7, 3, 0].

16. The array `a[0:9]` = [4, 2, 6, 7, 1, 0, 9, 8, 5, 3] is sorted using the early-terminating version of bubble sort (Program 2.13). Draw a figure similar to Figure 2.6(b) to show the progress of the sort.

17. The array `a[0:9]` = [4, 2, 6, 7, 1, 0, 9, 8, 5, 3] is to be sorted using insertion sort (Program 2.14). Draw a figure similar to Figure 2.6(c) to show the progress of the sort.

18. How many additions (i.e., invocations of `increment`) are done in the `for` loop of function `sum` (Program 1.30)?

Copyrighted material

19. How many multiplications are performed by the function `factorial` (Program 1.29)?

20. Create an input array `a` that causes function `rearrange` (Program 2.11) to do $n-1$ element reference swaps and $n-1$ rank swaps.

21. How many additions are performed between pairs of array elements by function `matrixAdd` (Program 2.21)?

```
template<class T>
void matrixAdd( T **a, T **b, T **c, int numberOfRows,
                                     int numberOfColumns)
{// Add matrices a and b to obtain matrix c.
   for (int i = 0; i < numberOfRows; i++)
      for (int j = 0;  j < numberOfColumns; j++)
         c[i][j] = a[i][j] + b[i][j];
}
```

Program 2.21 Matrix addition

22. How many swap operations are performed by function `transpose` (Program 2.19)?

23. Determine the number of multiplications done by function `squareMatrixMultiply` (Program 2.22), which multiplies two `n` × `n` matrices.

```
template<class T>
void squareMatrixMultiply(T **a, T **b, T **c, int n)
{// Multiply the n x n matrices a and b to get c.
   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
      {
         T sum = 0;
         for (int k = 0; k < n; k++)
            sum += a[i][k] * b[k][j];
         c[i][j] = sum;
      }
}
```

Program 2.22 Multiply two `n` × `n` matrices

24. Determine the number of multiplications done by function `matrixMultiply` (Program 2.23), which multiplies an `m` × `n` matrix and an `n` × `p` matrix.

Copyrighted material

```
template<class T>
void matrixMultiply(T **a, T **b, T **c, int m, int n, int p)
{// Multiply the m x n matrix a and the n x p matrix b
 // to get c.
   for (int i = 0; i < m; i++)
      for (int j = 0; j < p; j++)
      {
         T sum = 0;
         for (int k = 0; k < n; k++)
            sum += a[i][k] * b[k][j];
         c[i][j] = sum;
      }
}
```

Program 2.23 Multiply an `m` × `n` and an `n` × `p` matrix

25. Determine the number of **swap** operations performed by function **permutations** (Program 1.32).

26. Method **minmax** (Program 2.24) determines the locations of the minimum and maximum elements in an array `a`. Let n be the instance characteristic. What is the number of comparisons between elements of `a`? Program 2.25 gives an alternative function to determine the locations of the minimum and maximum elements. What are the best-case and worst-case numbers of comparisons between elements of `a`? What can you say about the expected relative performance of the two functions?

27. How many comparisons between the `a[]`s and `x` are made by the recursive function **rSequentialSearch** (Program 2.2)?

28. Program 2.26 gives an alternative iterative sequential search function. What is the worst-case number of comparisons between `x` and the elements of `a`? Compare this number with the corresponding number for Program 2.1. Which function should run faster? Why?

29. (a) Introduce statements to increment **stepCount** at all appropriate points in Program 2.27.

    (b) Simplify the resulting program by eliminating statements. Both the simplified program and the program of part (a) should compute the same value for **stepCount**.

    (c) What is the exact value of **stepCount** when the program terminates? You may assume that the initial value of **stepCount** is 0.

Copyrighted material

```
template<class T>
bool minmax(T a[], int n, int& indexOfMin, int& indexOfMax)
{// Locate min and max elements in a[0:n-1].
 // Return false if less than one element.
   if (n < 1) return false;
   indexOfMin = indexOfMax = 0; // initial guess
   for (int i = 1; i < n; i++)
   {
      if (a[indexOfMin] > a[i]) indexOfMin = i;
      if (a[indexOfMax] < a[i]) indexOfMax = i;
   }
   return true;
}
```

Program 2.24 Finding the minimum and maximum

```
template<class T>
bool minmax(T a[], int n, int& indexOfMin, int& indexOfMax)
{// Locate min and max elements in a[0:n-1].
 // Return false if less than one element.
   if (n < 1) return false;
   indexOfMin = indexOfMax = 0; // initial guess
   for (int i = 1; i < n; i++)
      if (a[indexOfMin] > a[i]) indexOfMin = i;
      else if (a[indexOfMax] < a[i]) indexOfMax = i;
   return true;
}
```

Program 2.25 Another function to find the minimum and maximum

(d) Use the frequency method to determine the step count for Program 2.27. Clearly show the step-count table.

30. Do Exercise 29 for each of the following functions:

(a) `indexOfMax` (Program 1.37).

(b) `minmax` (Program 2.24).

(c) `minmax` (Program 2.25). Determine the worst-case step count.

(d) `factorial` (Program 1.29).

(e) `polyEval` (Program 2.3).

Copyrighted material

```
template<class T>
int sequentialSearch(T a[], int n, const T& x)
{// Search the unordered list a[0:n-1] for x.
 // Return position if found; return -1 otherwise.
   a[n] = x; // assume extra position available
   int i;
   for (i = 0; a[i] != x; i++);
   if (i == n) return -1;
   return i;
}
```

Program 2.26 Another sequential search function

```
void d(int x[], int n)
{
   for (int i = 0; i < n; i += 2)
      x[i] += 2;

   i = 1;
   while (i <= n/2)
   {
      x[i] += x[i+1];
      i++;
   }
}
```

Program 2.27 Method for Exercise 29

(f) `horner` (Program 2.4).

(g) `rank` (Program 2.5).

(h) `permutations` (Program 1.32).

(i) `sequentialSearch` (Program 2.26). Determine the worst-case step count.

(j) `selectionSort` (Program 2.7). Determine the best- and worst-case step counts.

(k) `selectionSort` (Program 2.12). Determine the best- and worst-case step counts.

(l) `insertionSort` (Program 2.14). Determine the worst-case step count.

(m) `insertionSort` (Program 2.15). Determine the worst-case step count.

(n) `bubbleSort` (Program 2.9). Determine the worst-case step count.

Copyrighted material

(o) `bubbleSort` (Program 2.13). Determine the worst-case step count.

(p) `matrixAdd` (Program 2.21).

(q) `squareMatrixMultiply` (Program 2.22).

31. Do Exercise 29 parts (a), (b), and (c) for the following functions:

    (a) `transpose` (Program 2.19).

    (b) `inef` (Program 2.20).

32. Determine the average step counts for the following functions:

    (a) `rSequentialSearch` (Program 2.2).

    (b) `sequentialSearch` (Program 2.26).

    (c) `insert` (Program 2.10).

33. (a) Do Exercise 29 for Program 2.23.

    (b) Under what conditions will it be profitable to interchange the two outermost `for` loops?

34. Compare the worst-case number of element reference moves made by functions `selectionSort` (Program 2.12), `insertionSort` (Program 2.15), `bubbleSort` (Program 2.13), and rank sort using Program 2.11.

35. Must a program exhibit its worst-case time behavior and worst-case space behavior at the same time (i.e., for the same input)? Prove your answer.

36. Use repeated substitution to solve the following recurrences (see Example 2.20).

(a) $t(n) = \begin{cases} 2 & n = 0 \\ 2 + t(n-1) & n > 0 \end{cases}$

(b) $t(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ 1 + t(n-2) & n > 0 \end{cases}$

(c) $t(n) = \begin{cases} 0 & n = 0 \\ 2n + t(n-1) & n > 0 \end{cases}$

(d) $t(n) = \begin{cases} 1 & n = 0 \\ 2 * t(n-1) & n > 0 \end{cases}$

(e) $t(n) = \begin{cases} 1 & n = 0 \\ 3 * t(n-1) & n > 0 \end{cases}$

Copyrighted material

CHAPTER 3

ASYMPTOTIC NOTATION

BIRD'S-EYE VIEW

Chapter 2 showed you how to analyze the space and time complexities of a program. The methods of that chapter are somewhat cumbersome because they attempt to obtain exact counts, rather than estimates. In this chapter we review asymptotic notation, which is used to make statements about program performance when the instance characteristics are large. When we use this notation, we need only estimate the step count. Although the big oh notation is the most popular asymptotic notation used in the performance analysis of programs, the omega, theta, and little oh notations also are in common use.

Asymptotic notation is first introduced in an informal manner in Section 3.2. The informal treatment of this section is adequate to follow all the analyses done in this book. A more rigorous treatment is presented in Section 3.3. You may omit Section 3.3 and not face any dire consequences.

The use of asymptotic notation is illustrated through the applications developed in Chapters 1 and 2. Additionally, an important and efficient search method—binary search of a sorted array—is developed and analyzed in this chapter. This search method is also available as the STL algorithm `binary_Search`.

Copyrighted material

3.1 INTRODUCTION

Two important reasons to determine operation and step counts are (1) to predict the growth in run time as the instance characteristics increase and (2) to compare the time complexities of two programs that perform the same task. When using operation counts, we focus on certain "key" operations and ignore all others. Therefore, you must be very cautious when using an operation count for either of the above purposes. For example, a program may do $2n$ comparisons, but the total number of computation steps could be $6n^2 + 8n$. It would be incorrect to use the comparison count of $2n$ to conclude that the run time grows linearly in n. It would also be incorrect to conclude that a program that makes $2n$ comparisons is faster than a program, for the same task, that makes $3n$ comparisons; the $3n$ comparison program may actually do less total work than is done by the $2n$ comparison program.

The operation count method accounts for only some of the work that is done in a program. Step counts attempt to overcome this deficiency by accounting for all work. However, the notion of a step is inexact. Both the instructions `x = y` and `x = y+z+(x/y)` count as one step. Therefore, two analysts may arrive at $4n^2+6n+2$ and $7n^2+3n+4$ as the step count for the same program. We cannot conclude that the run time will grow as either $4n^2 + 6n + 2$ or $7n^2 + 3n + 4$, because any step count of the form $c_1n^2 + c_2n + c_3$, where $c_1 > 0$, c_2, and c_3 are constants, could be a correct step count for the program. Because of the inexactness of what a step stands for, the exact step count isn't very useful for comparative purposes either. However, when the difference in the step counts of two programs is very large as in $3n + 3$ versus $900n + 10$. We might feel quite safe in predicting that the program with step count $3n+3$ will run in less time than the one with step count $900n+10$.

We can use the step count to accurately predict the growth in run time for large instance sizes (i.e., in the asymptote as n approaches infinity) and to predict the relative performance of two programs when the instance size becomes large. Suppose that the step count of a program is determined to be $c_1n^2+c_2n+c_3$, $c_1 > 0$. When n becomes large, the c_1n^2 term will be much larger than the remaining terms c_2n+c_3. The ratio of these two expressions is $r(n) = (c_2n+c_3)/(c_1n^2) = c_2/(c_1n)+c_3/(c_1n^2)$. Figure 3.1 plots $r(n)$ for the case $c_1 = 1$, $c_2 = 2$, and $c_3 = 3$. Even though $r(n)$ never equals 0 for any finite n, $r(n)$ gets closer and closer to 0 as we make n bigger and bigger.

Regardless of the values of $c_1 > 0$, c_2, and c_3, the ratio $r(n)$ approaches 0 as n approaches infinity; that is,

$$\lim_{n\to\infty}\left(\frac{c_2}{c_1n}+\frac{c_3}{c_1n^2}\right)=0$$

So for large n, $c_2n + c_3$ is insignificant when compared to c_1n^2, and run time may be approximated by the c_1n^2 term. Let n_1 and n_2 be two large values of n. We conclude that

Copyrighted material

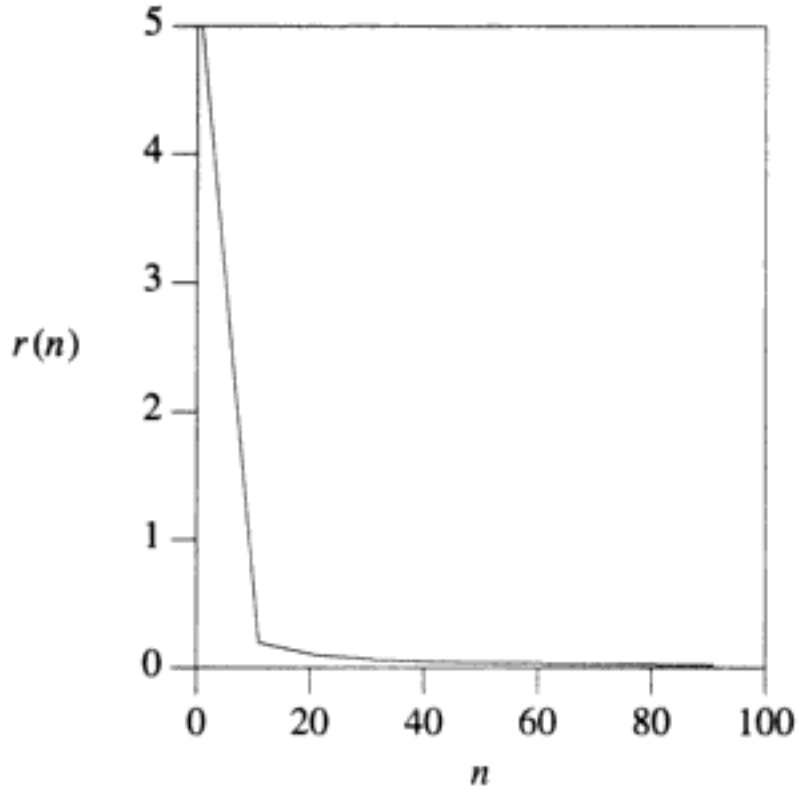


Figure 3.1 Plot of $r(n) = 2/n + 3/n^2$

$$\frac{t(n_1)}{t(n_2)} \approx \frac{c_1 n_1^2}{c_1 n_2^2} = \left(\frac{n_1}{n_2}\right)^2$$

Therefore, the run time is expected to increase by a factor of 4 (approximately) when the instance size is doubled; the run time increases by a factor of 9 when the instance size is tripled; and so on. To make this conclusion, all we need to know is that the biggest term in the step count is an n^2 term; the value of the coefficient c_1 is irrelevant to the conclusion.

Suppose that programs A and B perform the same task. Assume that John has determined the step counts of these programs to be $t_A(n) = n^2 + 3n$ and $t_B(n) = 43n$. It is entirely possible that Mary's analysis of the same programs yields $t_A(n) = 2n^2 + 3n$ and $t_B(n) = 83n$. In fact, assuming that John's analysis is correct, all other correct analyses would result in $t_A(n) = c_1 n^2 + c_2 n + c_3$ and $t_B(n) = c_4 n$, where c_1, c_2, c_3, and c_4 are constants, $c_1 > 0$, and $c_4 > 0$.

To see what conclusion we can draw about the relative performance of programs A and B knowing that the constant coefficients may vary from analyst to analyst, examine the plot of Figure 3.2. First, look at the curves for John's analysis, $t_A(n) = n^2 + 3n$ and $t_B(n) = 43n$. We conclude that for $n < 40$, program A is faster; for

Copyrighted material

$n > 40$, program B is faster; and $n = 40$ is the break-even point between the two programs. Now suppose that the analysis had instead concluded that $t_B(n) = 83n$. In this case we would conclude that for $n < 80$, program A is faster; for $n > 80$, program B is faster; and $n = 80$ is the break-even point between the two programs. Our conclusion that program B is faster than program A for large n does not change; only the break-even point changes.

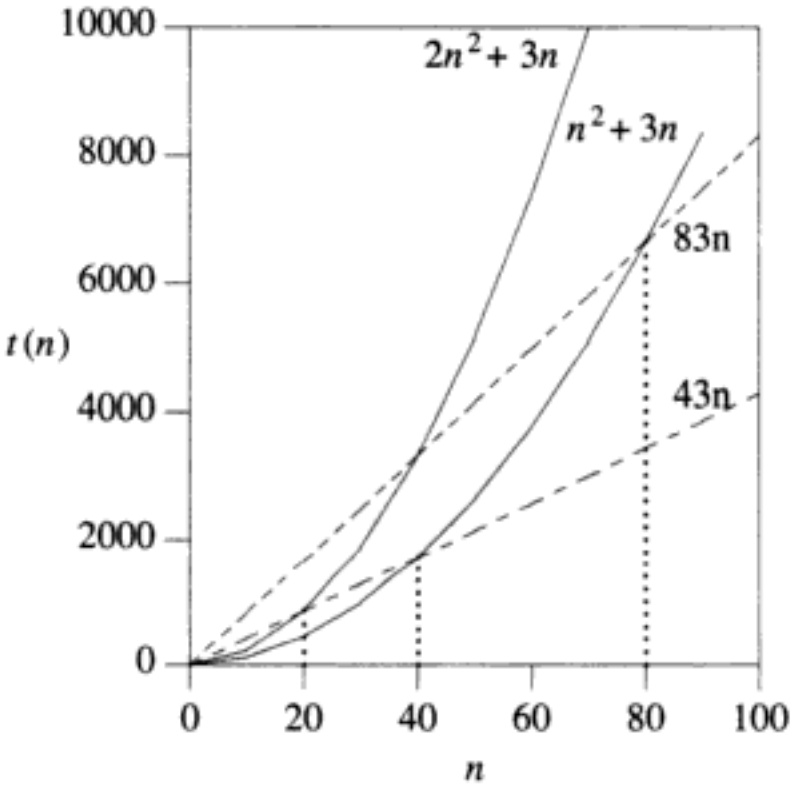


Figure 3.2 Comparing run time functions

What if John's analysis had concluded $t_A(n) = 2n^2 + 3n$? From Figure 3.2, we see that regardless of whether $t_B(n) = 43n$ or $83n$, program B remains faster than program A when n is suitably large ($n > 20$ when $t_B(n) = 43n$ and $n > 40$ when $t_B(n) = 83n$).

To arrive at the conclusion that program B is faster than program A when n is large, all we need to know is that the biggest term in the step count for program A is an n^2 term while that for program B is an n term; the values of the coefficients c_1 through c_4 are irrelevant to this conclusion. Asymptotic analysis focuses on determining the biggest terms (but not their coefficients) in the complexity function.

Copyrighted material

3.2 ASYMPTOTIC NOTATION

3.2.1 Big Oh Notation (O)

Definition 3.1 *Let $p(n)$ and $q(n)$ be two nonnegative functions. $p(n)$ is* **asymptotically bigger** *($p(n)$ asymptotically dominates $q(n)$) than the function $q(n)$ iff*

$$\lim_{n\to\infty} \frac{q(n)}{p(n)} = 0 \tag{3.1}$$

$q(n)$ is **asymptotically smaller** *than $p(n)$ iff $p(n)$ is asymptotically bigger than $q(n)$. $p(n)$ and $q(n)$ are* **asymptotically equal** *iff neither is asymptotically bigger than the other.* ■

Example 3.1 Since

$$\lim_{n\to\infty} \frac{10n+7}{3n^2+2n+6} = \frac{10/n+7/n^2}{3+2/n+6/n^2} = 0/3 = 0$$

$3n^2+2n+6$ is asymptotically bigger than $10n+7$ and $10n+7$ is asymptotically smaller than $3n^2+2n+6$. A similar derivation shows that $8n^4+9n^2$ is asymptotically bigger than $100n^3-3$, and that $2n^2+3n$ is asymptotically bigger than $83n$. $12n+6$ is asymptotically equal to $6n+2$. ■

In the following discussion the function $f(n)$ denotes the time or space complexity of a program as a function of the instance characteristic n. Since the time or space requirements of a program are nonnegative quantities, we assume that the function f has a nonnegative value for all values of n. Further, since n denotes an instance characteristic, we assume that $n \geq 0$. The function $f(n)$ will, in general, be a sum of terms. For example, the terms of $f(n) = 9n^2+3n+12$ are $9n^2$, $3n$, and 12. We may compare pairs of terms to determine which is bigger (see Definition 3.1). The biggest term in the example $f(n)$ is $9n^2$.

Figure 3.3 gives the terms that occur frequently in a step-count analysis. Although all the terms in Figure 3.3 have a coefficient of 1, in an actual analysis, the coefficients of these terms may have a different value.

We do not associate a logarithmic base with the functions in Figure 3.3 that include $\log n$ because for any constants a and b greater than 1, $\log_a n = \log_b n / \log_b a$. So $\log_a n$ and $\log_b n$ are asymptotically equal.

Using Definition 3.1, we obtain the following ordering for the terms of Figure 3.3 ($<$ is to be read as "is asymptotically smaller than"):

$$1 < \log n < n < n\log n < n^2 < n^3 < 2^n < n!$$

Copyrighted material

Term	Name
1	constant
$\log n$	logarithmic
n	linear
$n \log n$	$n \log n$
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

Figure 3.3 Commonly occurring terms

Asymptotic notation describes the behavior of the time or space complexity for large instance characteristics. Although we will develop asymptotic notation with reference to step counts alone, our development also applies to space complexity and operation counts. The terms *time complexity* and *step count* are used as synonyms. When the instance characteristic is described by a single variable, say n, asymptotic notation describes the complexity using a single term, the asymptotically biggest term in the step count.

The notation $f(n) = O(g(n))$ (read as "$f(n)$ is big oh of $g(n)$") means that $f(n)$ is asymptotically smaller than or equal to $g(n)$. Therefore, in an asymptotic sense $g(n)$ is an upper bound for $f(n)$. You may use this as a working definition of "big oh"; a formal definition is provided in Section 3.3.1.

Example 3.2 From Example 3.1 and the working definition of big oh, it follows that $10n + 7 = O(3n^2 + 2n + 6)$; $100n^3 - 3 = O(8n^4 + 9n^2)$; $12n + 6 = O(6n + 2)$; $3n^2 + 2n + 6 \neq O(10n + 7)$; and $8n^4 + 9n^2 \neq O(100n^3 - 3)$. ■

Although Example 3.2 uses the big oh notation in a correct way, it is customary to use $g(n)$ functions that are **unit terms** (i.e., $g(n)$ is a single term whose coefficient is 1) except when $f(n) = 0$. In addition, it is customary to use, for $g(n)$, the smallest unit term for which the statement $f(n) = O(g(n))$ is true. When $f(n) = 0$, it is customary to use $g(n) = 0$.

Example 3.3 The customary way to describe the asymptotic behavior of the functions used in Example 3.2 is $10n + 7 = O(n)$; $100n^3 - 3 = O(n^3)$; $12n + 6 = O(n)$; $3n^2 + 2n + 6 \neq O(n)$; and $8n^4 + 9n^2 \neq O(n^3)$. ■

In asymptotic complexity analysis, we determine the biggest term in the complexity; the coefficient of this biggest term is set to 1. The unit terms of a step-count

Copyrighted material

function are step-count terms with their coefficients changed to 1. For example, the unit terms of $3n^2 + 6n \log n + 7n + 5$ are n^2, $n \log n$, n, and 1; the biggest unit term is n^2. So when the step count of a program is $3n^2 + 6n \log n + 7n + 5$, we say that its asymptotic complexity is $O(n^2)$.

Example 3.4 In Example 2.19, we determined that $t_{\texttt{sum}}(n) = 2n + 3$. Since the biggest unit term in $t_{\texttt{sum}}(n)$ is n, $t_{\texttt{sum}}(n) = O(n)$.

Since $t_{\texttt{rSum}}(n) = 2n + 2$ (see Example 2.20), $t_{\texttt{rSum}}(n) = O(n)$.

The step count for Program 2.19 is $rows^2 + rows + 1$ (see Figure 2.8). The biggest unit term is $rows^2$. Therefore, $t_{\texttt{transpose}}(rows) = O(rows^2)$. ■

Notice that $f(n) = O(g(n))$ is not the same as $O(g(n)) = f(n)$. In fact, saying that $O(g(n)) = f(n)$ is meaningless. The use of the symbol $=$ is unfortunate, as this symbol commonly denotes the equals relation. We can avoid some of the confusion that results from the use of this symbol (which is standard terminology) by reading the symbol $=$ as "is" and not as "equals."

Definition 3.2 *Let $t(m,n)$ and $u(m,n)$ be two terms. $t(m,n)$ is asymptotically bigger than $u(m,n)$ (equivalently, $u(m,n)$ is asymptotically smaller than $t(m,n)$) iff either*

$$\lim_{n\to\infty} \frac{u(m,n)}{t(m,n)} = 0 \text{ and } \lim_{m\to\infty} \frac{u(m,n)}{t(m,n)} \neq \infty$$

or

$$\lim_{n\to\infty} \frac{u(m,n)}{t(m,n)} \neq \infty \text{ and } \lim_{m\to\infty} \frac{u(m,n)}{t(m,n)} = 0$$ ■

We may obtain a working definition of big oh for the case of functions in more than one variable as follows.

- Let $f(m,n)$ be the step count of a program. From $f(m,n)$ remove all terms that are asymptotically smaller than at least one other term in $f(m,n)$.
- Change the coefficients of all remaining terms to 1.

Example 3.5 Consider $f(m,n) = 3m^2n + m^3 + 10mn + 2n^2$. $10mn$ is smaller than $3m^2n$ because

$$\lim_{n\to\infty} \frac{10mn}{3m^2n} = \frac{10}{3m} \neq \infty \text{ and } \lim_{m\to\infty} \frac{10mn}{3m^2n} = \lim_{m\to\infty} \frac{10}{3m} = 0$$

None of the remaining terms is smaller than another. Dropping the $10mn$ term and changing the coefficients of the remaining terms to 1, we get $f(m,n) = O(m^2n + m^3 + n^2)$. ■

Copyrighted material

3.2.2 Omega (Ω) and Theta (Θ) Notations

Although the big oh notation is the most frequently used asymptotic notation, the omega and theta notations are sometimes used to describe the asymptotic complexity of a program. We provide a working definition of these notations in this section. See Sections 3.3.2 and 3.3.3 for a more formal definition.

The notation $f(n) = \Omega(g(n))$ (read as "$f(n)$ is omega of $g(n)$") means that $f(n)$ is asymptotically bigger than or equal to $g(n)$. Therefore, in an asymptotic sense, $g(n)$ is a lower bound for $f(n)$. The notation $f(n) = \Theta(g(n))$ (read as "$f(n)$ is theta of $g(n)$") means that $f(n)$ is asymptotically equal to $g(n)$.

Example 3.6 $10n+7 = \Omega(n)$ because $10n+7$ is asymptotically equal to n; $100n^3 - 3 = \Omega(n^3)$; $12n+6 = \Omega(n)$; $3n^3+2n+6 = \Omega(n)$; $8n^4+9n^2 = \Omega(n^3)$; $3n^3+2n+6 \neq \Omega(n^5)$; and $8n^4+9n^2 \neq \Omega(n^5)$.

$10n+7 = \Theta(n)$ because $10n+7$ is asymptotically equal to n; $100n^3-3 = \Theta(n^3)$; $12n+6 = \Theta(n)$; $3n^3+2n+6 \neq \Theta(n)$; $8n^4+9n^2 \neq \Theta(n^3)$; $3n^3+2n+6 \neq \Theta(n^5)$; and $8n^4+9n^2 \neq \Theta(n^5)$.

Since $t_{\texttt{sum}}(n) = 2n+3$ (see Example 2.19) and $2n+3$ is asymptotically equal to n, $t_{\texttt{sum}}(n) = \Theta(n)$.

Since $t_{\texttt{rSum}}(n) = 2n+2$ (see Example 2.20) and $2n+2$ asymptotically equals n, $t_{\texttt{rSum}}(n) = \Theta(n)$.

The step count for Program 2.19 is $rows^2+rows+1$ (see Figure 2.8), and $rows^2+rows+1$ asymptotically equals $rows^2$. Therefore, $t_{\texttt{transpose}}(rows) = \Theta(rows^2)$.

The best-case step count for `sequentialSearch` (Program 2.1) is 4 (Figure 2.10), the worst-case step count is $n+3$, and the average step count is $0.6n+3.4$. So the best-case asymptotic complexity of `sequentialSearch` is $\Theta(1)$, and the worst-case and average complexities are $\Theta(n)$. It is also correct to say that the complexity of `sequentialSearch` is $\Omega(1)$ and $O(n)$ because 1 is a lower bound (in an asymptotic sense) and n is an upper bound (in an asymptotic sense) on the step count.

From Figures 2.13 and 2.14, it follows that $4 \leq t_{\texttt{insert}}(n) \leq 2n+4$. Therefore, $t_{\texttt{insert}}(n)$ is both $\Omega(1)$ and $O(n)$. ■

At times it is useful to interpret $O(g(n))$, $\Omega(g(n))$, and $\Theta(g(n))$ as being the following sets:

$$O(g(n)) = \{f(n) | f(n) = O(g(n))\}$$

$$\Omega(g(n)) = \{f(n) | f(n) = \Omega(g(n))\}$$

$$\Theta(g(n)) = \{f(n) | f(n) = \Theta(g(n))\}$$

Under this interpretation, statements such as $O(g_1(n)) = O(g_2(n))$ and $\Theta(g_1(n)) = \Theta(g_2(n))$ are meaningful. When using this interpretation, it is also convenient to read $f(n) = O(g(n))$ as "f of n is in (or is a member of) big oh of g of n" and so on.

Copyrighted material

The working definitions of big oh, omega, and theta are all you need to understand the analyses done in this book. The next section contains a more formal treatment of asymptotic notation that will help you with more complex analyses.

EXERCISES

1. Use Equation 3.1 to show that $p(n)$ is asymptotically bigger than $q(n)$ for the following functions:
   (a) $p(n) = 3n^4 + 2n^2$, $q(n) = 100n^2 + 6$
   (b) $p(n) = 6n^{1.5} + 12$, $q(n) = 100n$
   (c) $p(n) = 7n^2 \log n$, $q(n) = 10n^2$
   (d) $p(n) = 17n^2 2^n$, $q(n) = 100n2^n + 33n$
2. Express the following step counts using big oh notation. Your $g(n)$ function should be the smallest possible unit term.
   (a) $2n^3 - 6n + 30$
   (b) $44n^{1.5} + 33n - 200$
   (c) $16n^2 \log n + 5n^2$
   (d) $31n^3 + 17n^2 \log n$
   (e) $23n2^n - 3n^3$
3. Use the working definition of big oh and Equation 3.1 to show the following:
   (a) $2n + 7 \neq O(1)$
   (b) $12n^2 + 8n + 7 \neq O(n)$
   (c) $5n^3 + 6n^2 \neq O(n^2)$
   (d) $15n^3 \log n + 16n^2 \neq O(n^3)$
4. Express the step counts of Exercise 2 using omega notation.
5. Use the working definition of omega and Equation 3.1 to show the following:
   (a) $2n + 7 \neq \Omega(n^2)$
   (b) $12n^2 + 8n + 7 \neq \Omega(n^3)$
   (c) $5n^3 + 6n^2 \neq \Omega(n^3 \log n)$
   (d) $15n^3 \log n + 16n^2 \neq \Omega(n^4)$
6. Express the step counts of Exercise 2 using theta notation.
7. Let $t(n)$ be the step count of a program. Express the following step-count information using asymptotic notation. Use the most appropriate $g(n)$ functions.

Copyrighted material

(a) $6 \leq t(n) \leq 20$

(b) $6 \leq t(n) \leq 2n$

(c) $3n^2 + 1 \leq t(n) \leq 4n^2 + 3n + 9$

(d) $3n^2 + 1 \leq t(n) \leq 4n^2 \log n + 3n^2 + 9$

(e) $t(n) \geq 5n^3 + 7$

(f) $t(n) \geq 32n \log n + 77n - 6$

(g) $t(n) = 17n^2 + 3n$

8. Express the following step counts using big oh notation. m and n are instance characteristics.

(a) $7m^2n^2 + 2m^3n + mn + 5mn^2$

(b) $2m^2 \log n + 3mn + 5m \log n + m^2n^2$

(c) $m^4 + n^3 + m^3n^2$

(d) $3mn^2 + 7m^2n + 4mn + 8m + 2n + 16$

3.3 ASYMPTOTIC MATHEMATICS (OPTIONAL)

3.3.1 Big Oh Notation (O)

The big oh notation describes an upper bound on the asymptotic growth rate of the function f.

Definition 3.3 [Big oh] *$f(n) = O(g(n))$ iff positive constants c and n_0 exist such that $f(n) \leq cg(n)$ for all n, $n \geq n_0$.* ■

The definition states that the function f is at most c times the function g except possibly when n is smaller than n_0. Here c is some positive constant. Thus g is an upper bound (up to a constant factor c) on the value of f for all suitably large n (i.e., $n \geq n_0$). Figure 3.4 illustrates what it means for a function $g(n)$ to upper bound (up to a constant factor c) another function $f(n)$. Although $f(n)$ may be less than, equal to, or greater than $cg(n)$ for several values of n, there must exist a value m of n beyond which $f(n)$ is never greater than $cg(n)$. The n_0 in the definition of big oh could be any integer $\geq m$.

When providing an upper-bound function g for f, we will normally use only simple functional forms. These typically contain a single term in n with a multiplicative constant of 1.

Example 3.7 [Linear Function] Consider $f(n) = 3n + 2$. When n is at least 2, $3n + 2 \leq 3n + n \leq 4n$. So $f(n) = O(n)$. Thus $f(n)$ is bounded from above by a linear function. We can arrive at the same conclusion in other ways. For example,

Copyrighted material

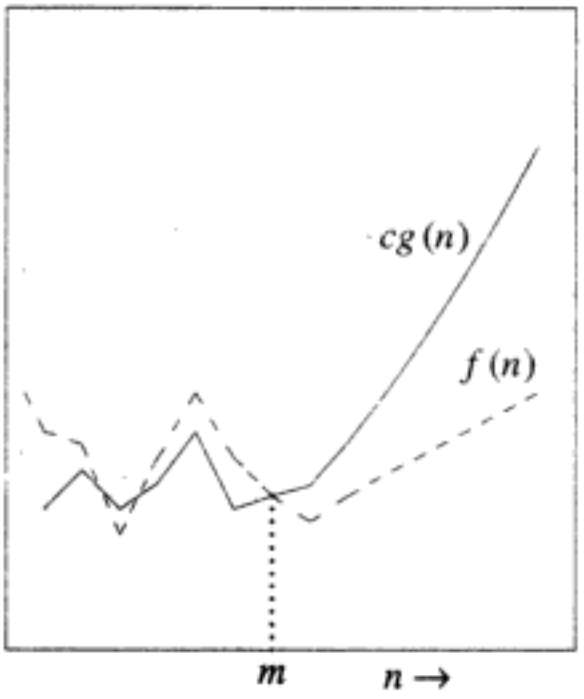


Figure 3.4 $g(n)$ is an upper bound (up to a constant factor c) on $f(n)$

$3n + 2 \leq 10n$ for $n > 0$. Therefore, we can also satisfy the definition of big oh by selecting $c = 10$ and n_0 equal to any integer greater than 0. Alternatively, $3n + 2 \leq 3n + 2n = 5n$ for $n \geq 1$, so we can satisfy the definition of big oh by setting $c = 5$ and $n_0 = 1$. The values of c and n_0 used to satisfy the definition of big oh are not important because we will be saying only that $f(n)$ is big oh of $g(n)$ and in this statement neither c nor n_0 play a role.

For $f(n) = 3n + 3$, we note that for $n \geq 3$, $3n + 3 \leq 3n + n \leq 4n$. So $f(n) = O(n)$. Similarly, $f(n) = 100n + 6 \leq 100n + n = 101n$ for $n \geq n_0 = 6$. Therefore, $100n + 6 = O(n)$. As expected, $3n + 2$, $3n + 3$, and $100n + 6$ are all big oh of n; that is, they are bounded from above by a linear function (for suitably large n). ■

Example 3.8 [Quadratic Function] Suppose that $f(n) = 10n^2 + 4n + 2$. We see that for $n \geq 2$, $f(n) \leq 10n^2 + 5n$. Now we note that for $n \geq 5$, $5n \leq n^2$. Hence for $n \geq n_0 = 5$, $f(n) \leq 10n^2 + n^2 = 11n^2$. Therefore, $f(n) = O(n^2)$.

As another example of a quadratic complexity, consider $f(n) = 1000n^2 + 100n - 6$. We easily see that $f(n) \leq 1000n^2 + 100n$ for all n. Furthermore, $100n \leq n^2$ for $n \geq 100$. Hence $f(n) < 1001n^2$ for $n \geq n_0 = 100$. So $f(n) = O(n^2)$. ■

Example 3.9 [Exponential Function] As an example of exponential complexity, consider $f(n) = 6 * 2^n + n^2$. Observe that for $n \geq 4$, $n^2 \leq 2^n$. So $f(n) \leq 6 * 2^n + 2^n = 7 * 2^n$ for $n \geq 4$. Therefore, $6 * 2^n + n^2 = O(2^n)$. ■

Copyrighted material

Example 3.10 [Constant Function] When $f(n)$ is a constant, as in $f(n) = 9$ or $f(n) = 2033$, we write $f(n) = O(1)$. The correctness of this is easily established. For example, $f(n) = 9 \leq 9 * 1$; setting $c = 9$ and $n_0 = 0$ satisfies the definition of big oh. Similarly, $f(n) = 2033 \leq 2033 * 1$, and the definition of big oh is satisfied by setting $c = 2033$ and $n_0 = 0$. ■

Example 3.11 [Loose Bounds] $3n + 3 = O(n^2)$ as $3n + 3 \leq 3n^2$ for $n \geq 2$. Although n^2 is an upper bound for $3n + 3$, it is not a tight upper bound; we can find a smaller function (in this case linear) that also satisfies the big oh relation.

$10n^2 + 4n + 2 = O(n^4)$ as $10n^2 + 4n + 2 \leq 10n^4$ for $n \geq 2$. Once again, n^4 does not provide a tight upper bound for $100n^2 + 4n + 2$.

Similarly, $6n2^n + 20 = O(n^2 2^n)$, but it is not a tight upper bound because we can find a smaller function, namely, $n2^n$, for which the definition of big oh is satisfied. That is, $6n2^n + 20 = O(n2^n)$. ■

Note that the strategy in each of the preceding derivations is to replace the low-order terms by higher-order terms until only a single term remains.

Example 3.12 [Incorrect Bounds] $3n + 2 \neq O(1)$, as there is no $c > 0$ and n_0 such that $3n + 2 \leq c$ for all n, $n \geq n_0$. We can use contradiction to prove this condition formally. Suppose that such a c and n_0 exist. Then $n \leq (c-2)/3$ for all n, $n \geq n_0$. This is not true for $n > \max\{n_0, (c-2)/3\}$.

To prove $10n^2 + 4n + 2 \neq O(n)$, suppose the equality holds. That is, $10n^2 + 4n + 2 = O(n)$. There exists a positive c and an n_0 such that $10n^2 + 4n + 2 \leq cn$ for all $n \geq n_0$. Dividing both sides of the relation by n, we get $10n + 4 + 2/n \leq c$ for $n \geq n_0$. This relation cannot be true because the left side increases as n increases, whereas the right side does not change. In particular, we get a contradiction for $n \geq \max\{n_0, (c-4)/10\}$.

$f(n) = 3n^2 2^n + 4n2^n + 8n^2 \neq O(2^n)$. To prove this inequality, suppose that $f(n) = O(2^n)$. Then a $c > 0$ and an n_0 exist such that $f(n) \leq c * 2^n$ for $n \geq n_0$. Dividing both sides by 2^n, we get $3n^2 + 4n + 8n^2/2^n \leq c$ for $n \geq n_0$. Once again, the left side of the relation is an increasing function of n while the right side is constant. So the relation cannot hold for "large" n. ■

As illustrated in Example 3.11, the statement $f(n) = O(g(n))$ states only that $cg(n)$ is an upper bound on the value of $f(n)$ for all n, $n \geq n_0$. It doesn't say anything about how good or tight this bound is. Notice that $n = O(n^2)$, $n = O(n^{2.5})$, $n = O(n^3)$, and $n = O(2^n)$. For the statement $f(n) = O(g(n))$ to be informative, $g(n)$ should be as small a function of n as possible for which $f(n) = O(g(n))$. So although we often say $3n + 3 = O(n)$, we almost never say $3n + 3 = O(n^2)$, even though the latter statement is correct.

Theorem 3.1 obtains a very useful result concerning the order of $f(n)$ (i.e., the $g(n)$ in $f(n) = O(g(n))$) when $f(n)$ is a polynomial in n.

Copyrighted material

Theorem 3.1 *If* $f(n) = a_m n^m + \cdots + a_1 n + a_0$ *and* $a_m > 0$, *then* $f(n) = O(n^m)$.

Proof $f(n) \leq \sum_{i=0}^{m} |a_i| n^i \leq n^m \sum_{0}^{m} |a_i| n^{i-m} \leq n^m \sum_{0}^{m} |a_i|$ for $n \geq 1$. So $f(n) = O(n^m)$. ■

Example 3.13 Let us apply Theorem 3.1 to the functions of Examples 3.7, 3.8, and 3.10. For the three linear functions of Example 3.7, $m = 1$, and so these functions are $O(n)$. For the functions of Example 3.8, $m = 2$, and so all are $O(n^2)$. For the constants of Example 3.10, $m = 0$, so both constants are $O(1)$. ■

We can extend the strategy used in Example 3.12 to show that an upper bound is incorrect to the case when an upper bound is correct, as shown in the following theorem. *It is usually easier to show* $f(n) = O(g(n))$ *by using this theorem than by using the definition of big oh.*

Theorem 3.2 [Big oh ratio theorem] *Let* $f(n)$ *and* $g(n)$ *be such that* $\lim_{n\to\infty} f(n)/g(n)$ *exists.* $f(n) = O(g(n))$ *iff* $\lim_{n\to\infty} f(n)/g(n) \leq c$ *for some finite constant* c.

Proof If $f(n) = O(g(n))$, then positive c and an n_0 exist such that $f(n)/g(n) \leq c$ for all $n \geq n_0$. Hence $\lim_{n\to\infty} f(n)/g(n) \leq c$. Suppose that $\lim_{n\to\infty} f(n)/g(n) \leq c$. It follows that an n_0 exists for which $f(n) \leq \max\{1, c\} * g(n)$ for all $n \geq n_0$. ■

Example 3.14 $3n+2 = O(n)$ as $\lim_{n\to\infty}(3n+2)/n = 3$. $10n^2 + 4n + 2 = O(n^2)$ as $\lim_{n\to\infty}(10n^2+4n+2)/n^2 = 10$. $6*2^n + n^2 = O(2^n)$ as $\lim_{n\to\infty}(6*2^n+n^2)/2^n = 6$. $2n^2 - 3 = O(n^4)$ as $\lim_{n\to\infty}(2n^2-3)/n^4 = 0$. $3n^2+5 \neq O(n)$ as $\lim_{n\to\infty}(3n^2+5)/n = \infty$. ■

3.3.2 Omega Notation (Ω)

The omega notation, which is the lower-bound analog of the big oh notation, permits us to bound the asymptotic growth rate of f from below.

Definition 3.4 [Omega] $f(n) = \Omega(g(n))$ *iff positive constants* c *and* n_0 *exist such that* $f(n) \geq cg(n)$ *for all* n, $n \geq n_0$. ■

When we write $f(n) = \Omega(g(n))$, we are saying that f is at least c times the function g except possibly when n is smaller than n_0. Here c is some positive constant. Thus g is a lower bound (up to a constant factor c) on the value of f for all suitably large n (i.e., $n \geq n_0$). Figure 3.5 illustrates what it means for a function $g(n)$ to lower bound (up to a constant factor c) another function $f(n)$. Although $f(n)$ may be less than, equal to, or greater than $cg(n)$ for several values of n, there must exist a value m of n beyond which $f(n)$ is never less than $cg(n)$. The n_0 in the definition of omega could be any integer $\geq m$.

As in the case of the big oh notation, we normally use only simple functional forms for g.

Copyrighted material

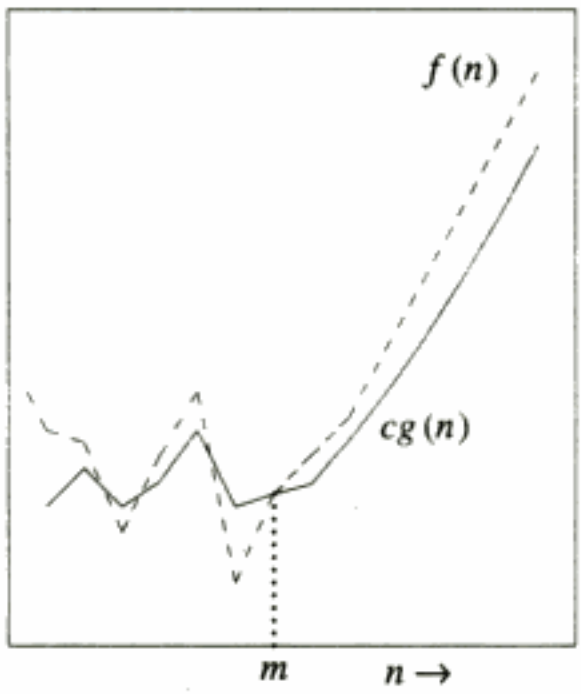


Figure 3.5 $g(n)$ is a lower bound (up to a constant factor c) on $f(n)$

Example 3.15 $f(n) = 3n + 2 > 3n$ for all n. So $f(n) = \Omega(n)$. Also, $f(n) = 3n+3 > 3n$, and so $f(n) = \Omega(n)$. Since $f(n) = 100n+6 > 100n$, $100n+6 = \Omega(n)$. So $3n + 2$, $3n + 3$, and $100n + 6$ are all bounded from below by a linear function.

$f(n) = 10n^2 + 4n + 2 > 10n^2$ for $n \geq 0$. So $f(n) = \Omega(n^2)$. Similarly, $1000n^2 + 100n - 6 = \Omega(n^2)$. Furthermore, since $6 * 2^n + n^2 > 6 * 2^n$, $6 * 2^n + n^2 = \Omega(2^n)$.

Observe also that $3n + 3 = \Omega(1)$; $10n^2 + 4n + 2 = \Omega(n)$; $10n^2 + 4n + 2 = \Omega(1)$; $6 * 2^n + n^2 = \Omega(n^{100})$; $6 * 2^n + n^2 = \Omega(n^{50.2})$; $6 * 2^n + n^2 = \Omega(n^2)$; $6 * 2^n + n^2 = \Omega(n)$; and $6 * 2^n + n^2 = \Omega(1)$.

To see that $3n + 2 \neq \Omega(n^2)$, suppose that $3n + 2 = \Omega(n^2)$. Then positive c and n_0 exist such that $3n + 2 \geq cn^2$ for all $n \geq n_0$. So $cn^2/(3n + 2) \leq 1$ for all $n \geq n_0$. This relation cannot be true because its left side increases to infinity as n becomes large. ■

As in the case of the big oh notation, there are several functions $g(n)$ for which $f(n) = \Omega(g(n))$. $g(n)$ is only a lower bound (up to a constant factor) on $f(n)$. For the statement $f(n) = \Omega(g(n))$ to be informative, $g(n)$ should be as large a function of n as possible for which the statement $f(n) = \Omega(g(n))$ is true. So although we say that $3n + 3 = \Omega(n)$ and that $6 * 2^n + n^2 = \Omega(2^n)$, we almost never say that $3n + 3 = \Omega(1)$ or that $6 * 2^n + n^2 = \Omega(1)$, even though both these statements are correct.

Theorem 3.3 is the analog of Theorem 3.1 for the omega notation.

Theorem 3.3 *If* $f(n) = a_m n^m + \cdots + a_1 n + a_0$ *and* $a_m > 0$, *then* $f(n) = \Omega(n^m)$.

Copyrighted material

Proof See Exercise 12. ■

Example 3.16 From Theorem 3.3, it follows that $3n + 2 = \Omega(n)$, $10n^2 + 4n + 2 = \Omega(n^2)$, and $100n^4 + 3500n^2 + 82n + 8 = \Omega(n^4)$. ■

Theorem 3.4 is the analog of Theorem 3.2, and it is usually easier to show $f(n) = \Omega(g(n))$ by using Theorem 3.4 than by using the definition of omega.

Theorem 3.4 [Omega ratio theorem] *Let $f(n)$ and $g(n)$ be such that $\lim_{n\to\infty} g(n)/f(n)$ exists. $f(n) = \Omega(g(n))$ iff $\lim_{n\to\infty} g(n)/f(n) \leq c$ for some finite constant c.*

Proof See Exercise 13. ■

Example 3.17 $3n + 2 = \Omega(n)$ as $\lim_{n\to\infty} n/(3n + 2) = 1/3$. $10n^2 + 4n + 2 = \Omega(n^2)$ as $\lim_{n\to\infty} n^2/(10n^2 + 4n + 2) = 0.1$. $6 * 2^n + n^2 = \Omega(2^n)$ as $\lim_{n\to\infty} 2^n/(6 * 2^n + n^2) = 1/6$. $6n^2 + 2 = \Omega(n)$ as $\lim_{n\to\infty} n/(6n^2 + 2) = 0$. $3n^2 + 5 \neq \Omega(n^3)$ as $\lim_{n\to\infty} n^3/(3n^2 + 5) = \infty$. ■

3.3.3 Theta Notation (Θ)

The theta notation is used when the function f can be bounded both from above and below by the same function g.

Definition 3.5 [Theta] *$f(n) = \Theta(g(n))$ iff positive constants c_1 and c_2 and an n_0 exist such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all n, $n \geq n_0$.* ■

When we write $f(n) = \Theta(g(n))$, we are saying that f lies between c_1 times the function g and c_2 times the function g except possibly when n is smaller than n_0. Here c_1 and c_2 are positive constants. Thus g is both a lower and upper bound (up to a constant factor c) on the value of f for all suitably large n (i.e., $n \geq n_0$). Another way to view the theta notation is that it says $f(n)$ is both $\Omega(g(n))$ and $O(g(n))$.

Figure 3.6 illustrates what it means for a function $g(n)$ to both upper and lower bound (up to a constant factor) another function $f(n)$. There must exist a value m of n beyond which $f(n)$ lies between $c_1 g(n)$ and $c_2 g(n)$. The n_0 in the definition of theta could be any integer $\geq m$.

As in the case of the big oh and omega notations, we normally use only simple functional forms for g.

Example 3.18 From Examples 3.7, 3.8, 3.9, and 3.15, it follows that $3n + 2 = \Theta(n)$; $3n + 3 = \Theta(n)$; $100n + 6 = \Theta(n)$; $10n^2 + 4n + 2 = \Theta(n^2)$; $1000n^2 + 100n - 6 = \Theta(n^2)$; and $6 * 2^n + n^2 = \Theta(2^n)$.

$10 * \log_2 n + 4 = \Theta(\log_2 n)$ as $\log_2 n < 10 \log_2 n + 4 \leq 11 \log_2 n$ for $n \geq 16$. As remarked earlier, $\log_a n$ is $\log_b n$ times a constant, and we write $\Theta(\log_a n)$ simply as $\Theta(\log n)$.

Copyrighted material

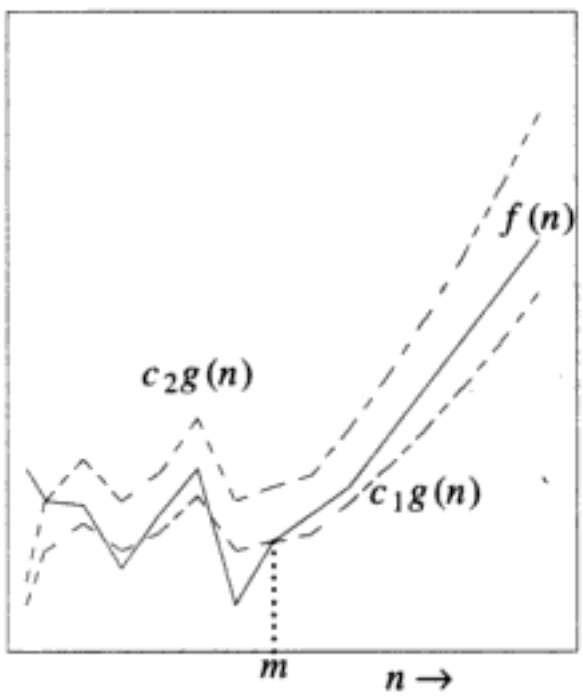


Figure 3.6 $g(n)$ is a lower and upper bound (up to a constant factor) on $f(n)$

In Example 3.12 we showed that $3n + 2 \neq O(1)$. So $3n + 2 \neq \Theta(1)$. Similarly, we may show that $3n+3 \neq \Theta(1)$ and $100n+6 \neq \Theta(1)$. Since $3n+3 \neq \Omega(n^2)$, $3n + 3 \neq \Theta(n^2)$. Since $10n^2 + 4n + 2 \neq O(n)$, $10n^2 + 4n + 2 \neq \Theta(n)$. Also, since $10n^2 + 4n + 2 \neq O(1)$, it is not $\Theta(1)$.

Since $6 * 2^n + n^2$ is not $O(n^2)$, it is not $\Theta(n^2)$. Similarly, $6 * 2^n + n^2 \neq \Theta(n^{100})$; and $6 * 2^n + n^2 \neq \Theta(1)$. ■

As mentioned earlier it is common practice to use only g functions with a multiplicative factor of 1. We almost never say that $3n + 3 = O(3n)$ or $10 = O(100)$ or $10n^2 + 4n + 2 = \Omega(4 * n^2)$ or $6 * 2^n + n^2 = \Omega(6 * 2^n)$ or $6 * 2^n + n^2 = \Theta(4 * 2^n)$, even though each of these statements is true.

Theorem 3.5 *If* $f(n) = a_m n^m + \cdots + a_1 n + a_0$ *and* $a_m > 0$, *then* $f(n) = \Theta(n^m)$.

Proof See Exercise 12. ■

Example 3.19 From Theorem 3.5 it follows that $3n + 2 = \Theta(n)$, $10n^2 + 4n + 2 = \Theta(n^2)$, and $100n^4 + 3500n^2 + 82n + 8 = \Theta(n^4)$. ■

Theorem 3.6 is the analog of Theorems 3.2 and 3.4.

Theorem 3.6 [Theta ratio theorem] *Let* $f(n)$ *and* $g(n)$ *be such that* $\lim_{n\to\infty} f(n)/g(n)$ *and* $\lim_{n\to\infty} g(n)/f(n)$ *exist.* $f(n) = \Theta(g(n))$ *iff* $\lim_{n\to\infty} f(n)/g(n) \leq c$ *and* $\lim_{n\to\infty} g(n)/f(n) \leq c$ *for some finite constant* c.

Copyrighted material

Proof See Exercise 13. ■

Example 3.20 $3n+2 = \Theta(n)$ as $\lim_{n\to\infty}(3n+2)/n = 3$ and $\lim_{n\to\infty} n/(3n+2) = 1/3 < 3$; $10n^2 + 4n + 2 = \Theta(n^2)$ as $\lim_{n\to\infty}(10n^2+4n+2)/n^2 = 10$; and $\lim_{n\to\infty} n^2/(10n^2+4n+2) = 0.1 < 10$. $6*2^n + n^2 = \Theta(2^n)$ as $\lim_{n\to\infty}(6*2^n+n^2)/2^n = 6$ and $\lim_{n\to\infty} 2^n/(6*2^n+n^2) = 1/6 < 6$. $6n^2+2 \neq \Theta(n)$ as $\lim_{n\to\infty}(6n^2+2)/n = \infty$. ■

3.3.4 Little Oh Notation (o)

The little oh notation describes a strict upper bound on the asymptotic growth rate of the function f. Informally, $f(n)$ is little oh of $g(n)$ iff $f(n)$ is asymptotically smaller than $g(n)$ (recall that $f(n)$ is big oh of $g(n)$ iff $f(n)$ is asymptotically smaller than or equal to $g(n)$).

Definition 3.6 [Little oh] *$f(n) = o(g(n))$ (read as "f of n is little oh of g of n") iff $f(n) = O(g(n))$ and $f(n) \neq \Omega(g(n))$.* ■

Example 3.21 [Little oh] $3n+2 = o(n^2)$ as $3n+2 = O(n^2)$ and $3n+2 \neq \Omega(n^2)$. However, $3n+2 \neq o(n)$. Similarly, $10n^2 + 4n + 2 = o(n^3)$, but is not $o(n^2)$. ■

The little oh notation is often used in step-count analyses. A step count of $3n + o(n)$ would mean that the step count is $3n$ plus terms that are asymptotically smaller than n. When performing such an analysis, one can ignore portions of the program that are known to contribute less than $\Theta(n)$ steps.

3.3.5 Properties

The following theorem is useful in computations involving asymptotic notation.

Theorem 3.7 *These statements are true for every real number x, $x > 0$ and for every real ϵ, $\epsilon > 0$:*

1. *An n_0 exists such that $(\log n)^x < (\log n)^{x+\epsilon}$ for every n, $n \geq n_0$.*
2. *An n_0 exists such that $(\log n)^x < n^\epsilon$ for every n, $n \geq n_0$.*
3. *An n_0 exists such that $n^x < n^{x+\epsilon}$ for every n, $n \geq n_0$.*
4. *For every real y, an n_0 exists such that $n^x(\log n)^y < n^{x+\epsilon}$ for every n, $n \geq n_0$.*
5. *An n_0 exists such that $n^x < 2^n$ for every n, $n \geq n_0$.*

Proof Follows from the definition of the individual functions. ■

Copyrighted material

Example 3.22 From Theorem 3.7 we obtain the following: $n^3 + n^2 \log n = \Theta(n^3)$; $2^n/n^2 = \Omega(n^k)$ for every natural number k; $n^4 + n^{2.5} \log^{20} n = \Theta(n^4)$; $2^n n^4 \log^3 n + 2^n n^4 / \log n = \Theta(2^n n^4 \log^3 n)$. ■

Figure 3.7 lists some of the more useful identities involving the big oh, omega, and theta notations. In this table all symbols other than n are positive constants. Figure 3.8 lists some useful inference rules for sums and products.

	$f(n)$	Asymptotic
E1	c	$\oplus(1)$
E2	$\sum_{i=0}^{k} c_i n^i$	$\oplus(n^k)$
E3	$\sum_{i=1}^{n} i$	$\oplus(n^2)$
E4	$\sum_{i=1}^{n} i^2$	$\oplus(n^3)$
E5	$\sum_{i=1}^{n} i^k$, $k > 0$	$\oplus(n^{k+1})$
E6	$\sum_{i=0}^{n} r^i$, $r > 1$	$\oplus(r^n)$
E7	$n!$	$\oplus(\sqrt{n}(n/e)^n)$
E8	$\sum_{i=1}^{n} 1/i$	$\oplus(\log n)$

$\oplus$ can be any one of O, Ω, and Θ

Figure 3.7 Asymptotic identities

Figures 3.7 and 3.8 prepare you to use asymptotic notation to describe the time complexity (or step count) of a program.

The definitions of O, Ω, Θ, and o can be extended to include functions of more than one variable. For example, $f(n, m) = O(g(n, m))$ iff positive constants c, n_0, and m_0 exist such that $f(n, m) \leq cg(n, m)$ for all $n \geq n_0$ and all $m \geq m_0$.

EXERCISES

9. Show that the following equalities are correct, using the definitions of O, Ω, Θ, and o only. Do not use Theorems 3.1 through 3.6, or Figures 3.7 and 3.8.

   (a) $5n^2 - 6n = \Theta(n^2)$.

Copyrighted material

I1 $\{f(n) = \oplus(g(n))\} \rightarrow \sum_{n=a}^{b} f(n) = \oplus(\sum_{n=a}^{b} g(n))$.

I2 $\{f_i(n) = \oplus(g_i(n)), 1 \leq i \leq k\} \rightarrow \sum_{i=1}^{k} f_i(n) = \oplus(\max_{1 \leq i \leq k}\{g_i(n)\})$.

I3 $\{f_i(n) = \oplus(g_i(n)), 1 \leq i \leq k\} \rightarrow \prod_{i=1}^{k} f_i(n) = \oplus(\prod_{i=1}^{k} g_i(n))$.

I4 $\{f_1(n) = O(g_1(n)), f_2(n) = \Theta(g_2(n))\} \rightarrow f_1(n) + f_2(n) = O(g_1(n) + g_2(n))$.

I5 $\{f_1(n) = \Theta(g_1(n)), f_2(n) = \Omega(g_2(n))\} \rightarrow f_1(n) + f_2(n) = \Omega(g_1(n) + g_2(n))$.

I6 $\{f_1(n) = O(g(n)), f_2(n) = \Theta(g(n))\} \rightarrow f_1(n) + f_2(n) = \Theta(g(n))$.

Figure 3.8 Inference rules for $\oplus \in \{O, \Omega, \Theta\}$

(b) $n! = O(n^n)$.
(c) $2n^2 2^n + n \log n = \Theta(n^2 2^n)$.
(d) $\sum_{i=0}^{n} i^2 = \Theta(n^3)$.
(e) $\sum_{i=0}^{n} i^3 = \Theta(n^4)$.
(f) $n^{2^n} + 6 * 2^n = \Theta(n^{2^n})$.
(g) $n^3 + 10^6 n^2 = \Theta(n^3)$.
(h) $6n^3/(\log n + 1) = O(n^3)$.
(i) $n^{1.001} + n \log n = \Theta(n^{1.001})$.
(j) $n^{k+\epsilon} + n^k \log n = \Theta(n^{k+\epsilon})$ for all k and ϵ, $k \geq 0$, and $\epsilon > 0$.

10. Do Exercise 9 using Theorems 3.2, 3.4, and 3.6.

11. Show that the following equalities are incorrect:

(a) $10n^2 + 9 = O(n)$.
(b) $n^2 \log n = \Theta(n^2)$.
(c) $n^2/\log n = \Theta(n^2)$.
(d) $n^3 2^n + 6n^2 3^n = O(n^3 2^n)$.

12. Prove Theorems 3.3 and 3.5.

13. Prove Theorems 3.4 and 3.6.

14. Prove that $f(n) = o(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.

15. Prove that equivalences E5 to E8 (Figure 3.7) are correct.

Copyrighted material

16. Prove the correctness of inference rules I1 to I6 (Figure 3.8).

17. Which of the following inferences are true? Why?

(a) $\{f(n) = O(F(n)), g(n) = O(G(n))\} \rightarrow f(n)/g(n) = O(F(n)/G(n))$.
(b) $\{f(n) = O(F(n)), g(n) = O(G(n))\} \rightarrow f(n)/g(n) = \Omega(F(n)/G(n))$.
(c) $\{f(n) = O(F(n)), g(n) = O(G(n))\} \rightarrow f(n)/g(n) = \Theta(F(n)/G(n))$.
(d) $\{f(n) = \Omega(F(n)), g(n) = \Omega(G(n))\} \rightarrow f(n)/g(n) = \Omega(F(n)/G(n))$.
(e) $\{f(n) = \Omega(F(n)), g(n) = \Omega(G(n))\} \rightarrow f(n)/g(n) = O(F(n)/G(n))$.
(f) $\{f(n) = \Omega(F(n)), g(n) = \Omega(G(n))\} \rightarrow f(n)/g(n) = \Theta(F(n)/G(n))$.
(g) $\{f(n) = \Theta(F(n)), g(n) = \Theta(G(n))\} \rightarrow f(n)/g(n) = \Theta(F(n)/G(n))$.
(h) $\{f(n) = \Theta(F(n)), g(n) = \Theta(G(n))\} \rightarrow f(n)/g(n) = \Omega(F(n)/G(n))$.
(i) $\{f(n) = \Theta(F(n)), g(n) = \Theta(G(n))\} \rightarrow f(n)/g(n) = O(F(n)/G(n))$.

3.4 COMPLEXITY ANALYSIS EXAMPLES

In Section 3.2 we saw several examples in which we started with the step count of a program and then arrived at its asymptotic complexity. Actually, we can determine the asymptotic complexity quite easily without determining the exact step count. The procedure is to first determine the asymptotic complexity of each statement (or group of statements) in the program and then add up these complexities. Figures 3.9 to 3.12 determine the asymptotic complexity of several methods without performing an exact step-count analysis. These figures use the following fact that when $f_1(n) = \Theta(g_1(n))$ and $f_2(n) = \Theta(g_2(n))$, then $f_1(n) + f_2(n) = \Theta(\max\{g_1(n), g_2(n)\})$.

Statement	s/e	Frequency	Total Steps
`T sum(T a[], int n)`	0	0	$\Theta(0)$
`{`	0	0	$\Theta(0)$
`  T theSum = 0;`	1	1	$\Theta(1)$
`  for (int i = 0; i < n; i++)`	1	$n+1$	$\Theta(n)$
`    theSum += a[i];`	1	n	$\Theta(n)$
`  return theSum;`	1	1	$\Theta(1)$
`}`	0	0	$\Theta(0)$

$$t_{\texttt{sum}}(n) = \Theta(\max\{g_i(n)\}) = \Theta(n)$$

Figure 3.9 Asymptotic complexity of `sum` (Program 1.30)

While the analyses of Figures 3.9 through 3.12 are actually carried out in terms of step counts, it is correct to interpret $t_P(n) = \Theta(g(n))$, $t_P(n) = O(g(n))$, or $t_P(n) = \Omega(g(n))$ as a statement about the computing time of program P because each step takes only $\Theta(1)$ time to execute.

Copyrighted material

Statement	s/e	Frequency	Total Steps
`void transpose(T **a, int rows)`	0	0	$\Theta(0)$
`{`	0	0	$\Theta(0)$
`  for (int i = 0; i < rows; i++)`	1	$rows + 1$	$\Theta(rows)$
`    for (int j = i+1; j < rows; j++)`	1	$rows(rows+1)/2$	$\Theta(rows^2)$
`      swap(a[i][j], a[j][i]);`	1	$rows(rows-1)/2$	$\Theta(rows^2)$
`}`	0	0	$\Theta(0)$

$$t_{\texttt{transpose}}(rows) = \Theta(rows^2)$$

Figure 3.10 Asymptotic complexity of `transpose` (Program 2.19)

Statement	s/e	Frequency	Total Steps
`void inef(T a[], T b[], int n)`	0	0	$\Theta(0)$
`{`	0	0	$\Theta(0)$
`  for (int j = 0; j < n; j++)`	1	$n + 1$	$\Theta(n)$
`    b[j] = sum(a, j + 1);`	$2j + 6$	n	$\Theta(n^2)$
`}`	0	0	$\Theta(0)$

$$t_{\texttt{inef}}(n) = \Theta(n^2)$$

Figure 3.11 Asymptotic complexity of `inef` (Program 2.20)

Statement	s/e	Frequency	Total Steps
`int sequentialSearch(T a[], int n, const T& x)`	0	0	$\Theta(0)$
`{`	0	0	$\Theta(0)$
`  int i;`	1	1	$\Theta(1)$
`  for (i = 0; i < n && a[i] != x; i++);`	1	$\Omega(1), O(n)$	$\Omega(1), O(n)$
`  if (i == n) return -1;`	1	1	$\Theta(1)$
`  else return i;`	1	$\Omega(0), O(1)$	$\Omega(0), O(1)$
`}`	0	0	$\Theta(0)$

$$t_{\texttt{sequentialSearch}}(n) = \Omega(1) \qquad t_{\texttt{sequentialSearch}}(n) = O(n)$$

Figure 3.12 Asymptotic complexity of `sequentialSearch` (Program 2.1)

After you have had some experience using the table method, you will be in a position to arrive at the asymptotic complexity of a program by taking a more global approach. We elaborate on this method in the following examples.

Example 3.23 [Permutations] Consider the permutation generation code of Program 1.32. Assume that `m = n-1`. When `k = m`, the time taken is cn, where c is a constant. When `k < m`, the `else` clause is entered. At this time the `for` loop is entered

Copyrighted material

`m-k+1` times. Each iteration of this loop takes $dt_{\texttt{permutations}}(\texttt{k+1,m})$ time, where d is a constant. So $t_{\texttt{permutations}}(\texttt{k,m}) = d(\texttt{m-k+1})t_{\texttt{permutations}}(\texttt{k+1,m})$ when `k<m`. Using the substitution method, we obtain $t_{\texttt{permutations}}(\texttt{0,m}) = \Theta(\texttt{(m+1)*(m+1)!})$ $= \Theta(\texttt{n*n!})$. ■

Example 3.24 [Binary Search] Program 3.1 is a method to search a sorted array `a` for the element `x`. The STL algorithm `binary_Search` is quite similar. The variables `left` and `right` keep track of the two ends of the array segment to be searched. Initially we are to search between positions 0 and `n-1`. So `left` and `right` are, respectively, initialized to these values. We maintain the following invariant throughout:

`x` is one of `a[0:n-1]` iff `x` is one of `a[left:right]`

```
template<class T>
int binarySearch(T a[], int n, const T& x)
{// Search a[0] <= a[1] <= ... <= a[n-1] for x.
 // Return position if found; return -1 otherwise.
   int left = 0;                          // left end of segment
   int right = n - 1;                     // right end of segment
   while (left <= right) {
      int middle = (left + right)/2;      // middle of segment
      if (x == a[middle]) return middle;
      if (x > a[middle]) left = middle + 1;
      else right = middle - 1;
      }
   return -1; // x not found
}
```

Program 3.1 Binary search

The search begins by comparing `x` with the element in the middle of the segment to be searched. If `x` equals this element, the search terminates. If `x` is smaller than this element, then we need only search the left half and so `right` is updated to `middle-1`. If `x` is bigger than the middle element, only the right half needs to be searched and `left` is updated to `middle+1`.

Each iteration of the `while` loop—except the last one—results in a decrease in the size of the segment of `a` that has to be searched by a factor of about 2. So this loop iterates $\Theta(\log \texttt{n})$ times in the worst case. As each iteration takes $\Theta(1)$ time, the overall worst-case complexity is $\Theta(\log \texttt{n})$. ■

Copyrighted material

Example 3.25 [Insertion Sort] Program 2.15 uses the insertion sort method to sort n elements. For each value of `i`, the innermost `for` loop has a worst-case complexity $\Theta(\mathtt{i})$. As a result, the worst-case time complexity of Program 2.15 is $\Theta(1 + 2 + 3 + \cdots + n - 1) = \Theta(n^2)$. The best-case time complexity of Program 2.15 is $\Theta(n)$. ■

EXERCISE

18. Determine the asymptotic time complexity of the following methods. Set up a frequency table similar to Figures 3.9 through 3.12.

    (a) `factorial` (Program 1.29).

    (b) `minmax` (Program 2.24).

    (c) `minmax` (Program 2.25).

    (d) `matrixAdd` (Program 2.21).

    (e) `squareMatrixMultiply` (Program 2.22).

    (f) `matrixMultiply` (Program 2.23).

    (g) `indexOfMax` (Program 1.37).

    (h) `polyEval` (Program 2.3).

    (i) `horner` (Program 2.4).

    (j) `rank` (Program 2.5).

    (k) `permutations` (Program 1.32).

    (l) `selectionSort` (Program 2.7).

    (m) `selectionSort` (Program 2.12).

    (n) `insertionSort` (Program 2.14).

    (o) `insertionSort` (Program 2.15).

    (p) `bubbleSort` (Program 2.9).

    (q) `bubbleSort` (Program 2.13).

3.5 PRACTICAL COMPLEXITIES

We have seen that the time complexity of a program is generally some function of the instance characteristics. This function is very useful in determining how the time requirements vary as the instance characteristics change. We can also use the complexity function to compare two programs P and Q that perform the same task. Assume that program P has complexity $\Theta(n)$ and that program Q has complexity $\Theta(n^2)$. We can assert that program P is faster than program Q is for "sufficiently

Copyrighted material

large" n. To see the validity of this assertion, observe that the actual computing time of P is bounded from above by cn for some constant c and for all n, $n \geq n_1$, while that of Q is bounded from below by dn^2 for some constant d and all n, $n \geq n_2$. Since $cn \leq dn^2$ for $n \geq c/d$, program P is faster than program Q whenever $n \geq \max\{n_1, n_2, c/d\}$.

One should always be cautiously aware of the presence of the phrase *sufficiently large* in the assertion of the preceding discussion. When deciding which of the two programs to use, we must know whether the n we are dealing with is, in fact, sufficiently large. If program P actually runs in $10^6 n$ milliseconds while program Q runs in n^2 milliseconds and if we always have $n \leq 10^6$, then program Q is the one to use.

To get a feel for how the various functions grow with n, you should study Figures 3.13 and 3.14 very closely. These figures show that 2^n grows very rapidly with n. In fact, if a program needs 2^n steps for execution, then when $n \doteq 40$, the number of steps needed is approximately $1.1 * 10^{12}$. On a computer performing 1,000,000,000 steps per second, this program would require about 18.3 minutes. If $n = 50$, the same program would run for about 13 days on this computer. When $n = 60$, about 310.56 years will be required to execute the program, and when $n = 100$, about $4 * 10^{13}$ years will be needed. We can conclude that the utility of programs with exponential complexity is limited to small n (typically $n \leq 40$).

$\log n$	n	$n \log n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65,536
5	32	160	1024	32,768	4,294,967,296

Figure 3.13 Value of various functions

Programs that have a complexity that is a high-degree polynomial are also of limited utility. For example, if a program needs n^{10} steps, then our 1,000,000,000 steps per second computer needs 10 seconds when $n = 10$; 3171 years when $n = 100$; and $3.17 * 10^{13}$ years when $n = 1000$. If the program's complexity had been n^3 steps instead, then the computer would need 1 second when $n = 1000$, 110.67 minutes when $n = 10{,}000$, and 11.57 days when $n = 100{,}000$.

Figure 3.15 gives the time that a 1,000,000,000 instructions per second computer needs to execute a program of complexity $f(n)$ instructions. One should note that currently only the fastest computers can execute about 1,000,000,000 instructions per second. From a practical standpoint, it is evident that for reasonably large n (say $n > 100$) only programs of small complexity (such as n, $n \log n$, n^2, and n^3)

Copyrighted material

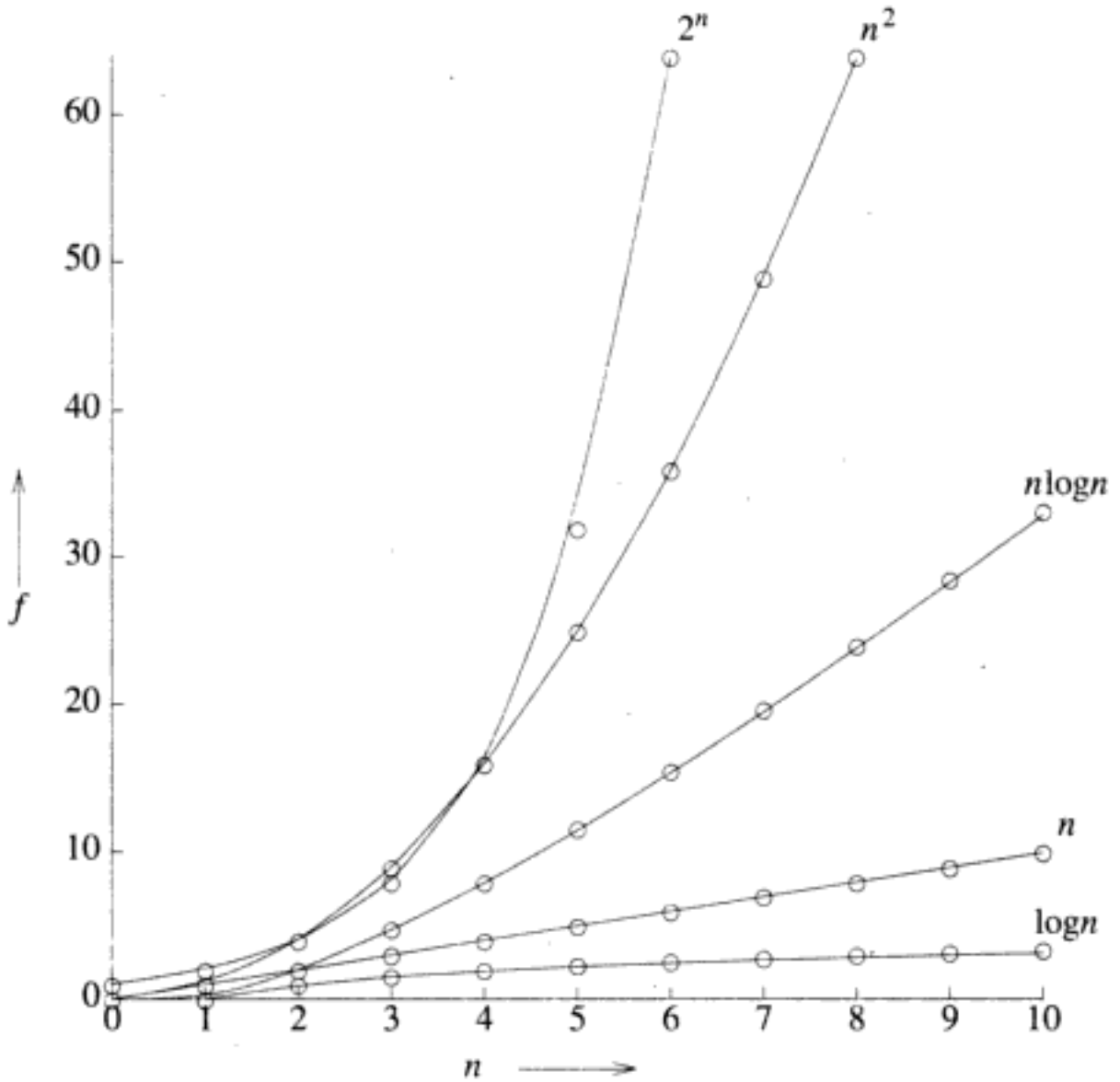


Figure 3.14 Plot of various functions

are feasible. Further, this is the case even if we could build a computer capable of executing 10^{12} instructions per second. In this case the computing times of Figure 3.15 would decrease by a factor of 1000. Now when $n = 100$, it would take 3.17 years to execute n^{10} instructions and $4 * 10^{10}$ years to execute 2^n instructions.

EXERCISES

19. Let A and B be two programs that perform the same task. Let $t_A(n)$ and $t_B(n)$, respectively, denote their run times. For each of the following pairs, find the range of n values for which program A is faster than program B.

Copyrighted material

	$f(n)$						
n	n	$n \log_2 n$	n^2	n^3	n^4	n^{10}	2^n
10	.01 μs	.03 μs	.1 μs	1 μs	10 μs	10 s	1 μs
20	.02 μs	.09 μs	.4 μs	8 μs	160 μs	2.84 h	1 ms
30	.03 μs	.15 μs	.9 μs	27 μs	810 μs	6.83 d	1 s
40	.04 μs	.21 μs	1.6 μs	64 μs	2.56 ms	121 d	18 m
50	.05 μs	.28 μs	2.5 μs	125 μs	6.25 ms	3.1 y	13 d
100	.10 μs	.66 μs	10 μs	1 ms	100 ms	3171 y	$4 * 10^{13}$ y
10^3	1 μs	9.96 μs	1 ms	1 s	16.67 m	$3.17 * 10^{13}$ y	$32 * 10^{283}$ y
10^4	10 μs	130 μs	100 ms	16.67 m	115.7 d	$3.17 * 10^{23}$ y	
10^5	100 μs	1.66 ms	10 s	11.57 d	3171 y	$3.17 * 10^{33}$ y	
10^6	1 ms	19.92 ms	16.67 m	31.71 y	$3.17 * 10^7$ y	$3.17 * 10^{43}$ y	

μs = microsecond = 10^{-6} seconds; ms = milliseconds = 10^{-3} seconds
s = seconds; m = minutes; h = hours; d = days; y = years

Figure 3.15 Run times on a 1,000,000,000 instruction per second computer

(a) $t_A(n) = 1000n$, $t_B(n) = 10n^2$.

(b) $t_A(n) = 2n^2$, $t_B(n) = n^3$.

(c) $t_A(n) = 2^n$, $t_B(n) = 100n$.

(d) $t_A(n) = 1000n \log_2 n$, $t_B(n) = n^2$.

20. Redo Figure 3.15 assuming a computer capable of doing 1 trillion instructions per second.

21. Suppose that using a certain program and computer, it is possible to solve problems of size up to $n = N$ in a "reasonable amount of time." Create a table that shows the largest value of n for which solutions can be found in reasonable time using the same program and a computer that is x times as fast. Do this exercise for $x = 10$, 100, 1000, and 1,000,000 and $t_A(n) = n$, n^2, n^3, n^5, and 2^n.

3.6 REFERENCES AND SELECTED READINGS

The following books provide asymptotic analyses for several programs: *Fundamentals of Computer Algorithms* by E. Horowitz, S. Sahni, and S. Rajasekaran, W. H. Freeman and Co., New York, NY, l998; *Introduction to Algorithms*, Second Edition, by T. Cormen, C. Leiserson, and R. Rivest, McGraw-Hill, New York, NY, 2002; and *Compared to What: An Introduction to the Analysis of Algorithms* by G. Rawlins, W. H. Freeman and Co., New York, NY, 1992.

Copyrighted material

5.1 DATA OBJECTS AND STRUCTURES

A data object is a set of *instances* or *values*. Some examples are

1. *boolean* = {*false*, *true*}
2. *digit* = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
3. *letter* = {A, B, C, $\cdots$, Z, a, b, $\cdots$, z}
4. *naturalNumber* = {0, 1, 2, $\cdots$}
5. *integer* = {0, $\pm$1, $\pm$2, $\pm$3, $\cdots$}
6. *string* = {a, b, $\cdots$, aa, ab, ac, $\cdots$}

boolean, *digit*, *letter*, *naturalNumber*, *integer*, and *string* are data objects. *true* and *false* are the instances of *boolean*, while 0, 1, $\cdots$, and 9 are the instances of *digit*. We may regard the individual instances of a data object as being either **primitive** (or **atomic**) or composed of instances of another (possibly the same) data object. In the latter case we use the term **element** to refer to the individual components of an instance of an object.

For example, each instance of the data object *NaturalNumber* can be regarded as atomic. In this case we are not concerned with a further decomposition of the instances of this data object. Another view is to regard each instance of a *naturalNumber* as being composed of several instances of the data object *digit*. In this view the number 675 comprises the digits 6, 7, and 5 (in that order).

The data object *string* is the set of all possible string instances. Each instance of a string is composed of characters. Some examples of instances are *good*, *a trip to Hawaii*, *going down hill*, and *abcabcdabcde*. The first string has the four elements *g*, *o*, *o*, and *d* (in that order). Each element is an instance of the data object *Letter*.

The instances of a data object as well as the elements that constitute individual instances are generally related in some way. For example, the natural number 0 is the smallest natural number; 1 is the next; and 2 is the next. In the natural number 675, the most significant digit is 6, the next is 7, and 5 is the least significant digit. In the string *good*, *g* is the first letter, *o* the second and third, and *d* the last.

In addition to interrelationships, a set of operations (or functions) is generally associated with any data object. These operations may transform one instance of an object into another instance of that object, or into an instance of another data object, or do both these transformations. The operation could simply create a new instance without transforming the instances from which the new one is created. For example, the operation *add* defined on the natural numbers creates a new natural number that is the sum of the two numbers to be added; the two numbers that get added are unaltered.

A **data structure** is a data object together with the relationships that exist among the instances and among the individual elements that compose an instance. These relationships are provided by specifying the operations of interest.

Copyrighted material

When we study data structures, we are concerned with the representation of data objects (actually of the instances) as well as the implementation of the operations of interest for the data objects. The representation of each data object should facilitate an efficient[1] implementation of the operations.

The most frequently used data objects together with their frequently used operations are already implemented in C++ as primitive data types. The data objects *integer* (`int`) and *boolean* (`bool`), defined above, fall into this category. All other data objects can be represented using the primitive data types and the grouping ability provided by the class, array, and pointer features of C++. Many of the data objects we shall study in this text (for example, linear list, stack, queue, and priority queue) have been implemented as classes in the STL.

5.2 THE LINEAR LIST DATA STRUCTURE

Each instance of the data structure **linear list** (or **ordered list**) is an ordered collection of elements. Each instance is of the form $(e_0, e_1, \cdots, e_{n-1})$ where n is a finite natural number; the e_i items are the elements of the list; the **index** of e_i is i; and n is the list **length** or **size**. The elements may be viewed as atomic, as their individual structure is not relevant to the structure of the list. When $n = 0$, the list is **empty**. When $n > 0$, e_0 is the **zeroth** (or **front**) element and e_{n-1} is the **last** element of the list. We say that e_0 **comes before** (or precedes) e_1, e_1 comes before e_2, and so on. Other than this precedence relation, no other structure exists in a linear list.

Some examples of linear lists are (1) an alphabetized (i.e., ordered by name) list of students in a class; (2) a list of exam scores in nondecreasing order; (3) an alphabetized list of members of Congress; and (4) a list of gold-medal winners in the Olympics men's basketball event ordered by year. With these examples in mind, we see the need to perform the following operations on a linear list:

- Create a linear list.
- Destroy a linear list.
- Determine whether the list is empty.
- Determine the size of the list.
- Find the element with a given index.
- Find the index of a given element.
- Delete, erase or remove an element given its index.
- Insert a new element so that it has a given index.

[1]The term *efficient* is used here in a very liberal sense. It includes performance efficiency as well as measures of the complexity of development and maintenance of associated software.

Copyrighted material

- Output the list elements in order, left to right.

5.2.1 The Abstract Data Type *linearList*

A linear list may be specified as an **abstract data type** (ADT) in which we provide a specification of the instances as well as of the operations that are to be performed (see ADT 5.1). The ADT specification is independent of any representation and programming language we have in mind. All representations of the ADT must satisfy the specification, and the specification becomes a way to validate the representation. In addition, all representations that satisfy the specification may be used interchangeably in applications of the data type. In ADT 5.1 we have omitted specifying operations to create and destroy instances of the data type. All ADT specifications implicitly include an operation to create an empty instance and, optionally, an operation to destroy an instance.

AbstractDataType *linearList*
{

instances

ordered finite collections of zero or more elements

operations

empty() :	return `true` if the list is empty, `false` otherwise
size() :	return the list size (i.e., number of elements in the list)
get(*index*):	return the *index*th element of the list
indexOf(x):	return the index of the first occurrence of x in the list, return -1 if x is not in the list
erase(*index*):	remove/delete the *index*th element, elements with higher index have their index reduced by 1
insert(*index*, x):	insert x as the *index*th element, elements with index $\geq$ *index* have their index increased by 1
output():	output the list elements from left to right

}

ADT 5.1 Abstract data type specification of a linear list

Copyrighted material

5.2.2 The Abstract Class LinearList

C++ supports two types of classes—abstract and concrete. An abstract class is a class that contains a member function for which no implementation has been specified. Such a function, called a *pure virtual function*, is specified using the zero initializer as in

```
virtual int myPureVirtualFunction(int x) = 0;
```

A concrete class contains no pure virtual function. Only concrete classes may be instantiated. That is, we may create an instance or object only of a concrete class. However, we can create pointers to objects of an abstract class.

Rather than use the informal English approach to specify an ADT as in ADT 5.1, we may use a C++ abstract class as in Program 5.1.

```
template<class T>
class linearList
{
   public:
      virtual ~linearList() {}
      virtual bool empty() const = 0;
                  // return true iff list is empty
      virtual int size() const = 0;
                  // return number of elements in list
      virtual T& get(int theIndex) const = 0;
                  // return element whose index is theIndex
      virtual int indexOf(const T& theElement) const = 0;
                  // return index of first occurence of theElement
      virtual void erase(int theIndex) = 0;
                  // remove the element whose index is theIndex
      virtual void insert(int theIndex, const T& theElement) = 0;
                  // insert theElement so that its index is theIndex
      virtual void output(ostream& out) const = 0;
                  // insert list into stream out
};
```

Program 5.1 Abstract class specification of a linear list

Although the specification of Program 5.1 is quite similar to that of ADT 5.1, this specification is programming-language dependent. In particular, many of the keywords we have used are defined only in C++. Since a class that derives from or extends an abstract class is itself abstract (and so cannot be instantiated) unless it provides an implementation for all pure virtual functions of the base class, by

Copyrighted material

requiring that every ADT implementation be derived from the abstract class for that ADT, we ensure a complete and consistent (i.e., with the same public functions) implementation of the ADT.

We provide a virtual destructor for our abstract class so that when a reference to a linear list goes out of scope, the default destructor for linear list isn't invoked; rather the destructor for the true data type of the referenced object is invoked.

EXERCISE

1. Let $L = (a, b, c, d)$ be a linear list. What is the result of each of the following operations?

   (a) $empty()$

   (b) $size()$

   (c) $get(0)$, $get(2)$, $get(6)$, $get(-3)$

   (d) $indexOf(a)$, $indexOf(c)$, $indexOf(q)$

   (e) $erase(0)$, $erase(2)$, $erase(3)$

   (f) $insert(0, e)$, $insert(2, f)$, $insert(3, g)$, $insert(4, h)$, $insert(6, h)$, $insert(-3, h)$

5.3 ARRAY REPRESENTATION

5.3.1 The Representation

In an **array representation**, we use an array to store the list elements. Although we can pack several list instances into a single array (see Section 5.5), it is easier to use a different array for each instance. Individual elements of an instance are located in the array using a mathematical formula.

Suppose we decide to use a one-dimensional `element` to store the elements of a linear list. The array `element` has positions (or locations) `element[0]` $\cdots$ `element[arrayLength-1]`, where `arrayLength` is the length or capacity of the array. Each array position can be used to store a single list element. We need to map the elements of the list to positions in the array. Where does the zeroth element reside? Where does the last element reside? The most natural mapping uses the formula

$$location(i) = i \tag{5.1}$$

Equation 5.1 states that the `i`th element of the list (if it exists) is stored in position `i` of the array. Figure 5.1(a) shows how the list [5, 2, 4, 8, 1] is stored in the array `element` using the mapping of Equation 5.1. The length of the array is 10, and the size of the list is 5.

Copyrighted material

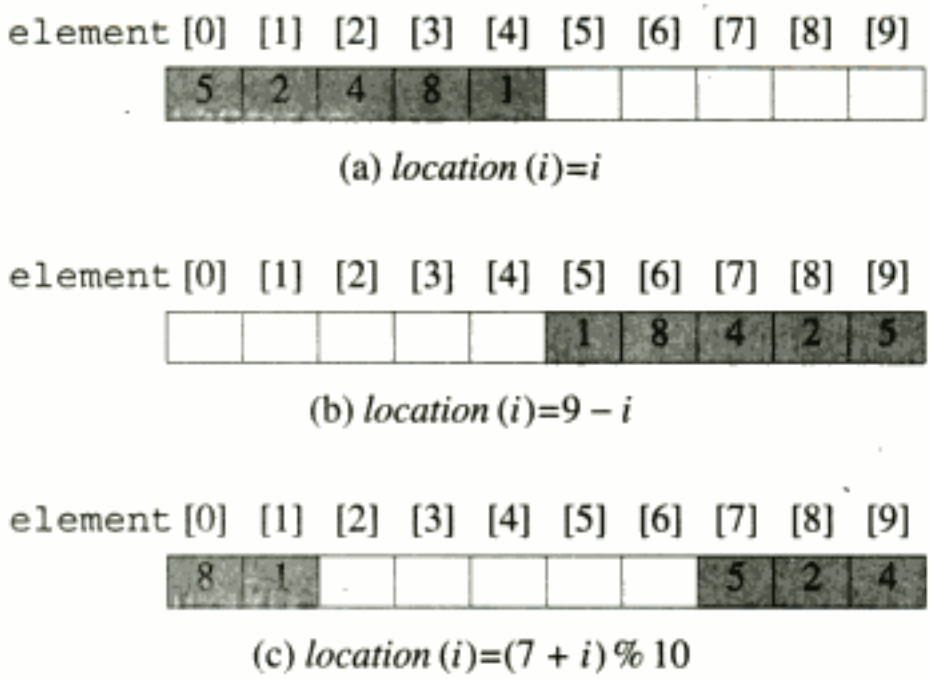


Figure 5.1 Different ways to map [5, 2, 4, 8, 1] into a one-dimensional array

Although Equation 5.1 is a natural choice for a formula to map list elements into array positions, other choices are possible. For example, the formula

$$location(i) = \texttt{arrayLength} - i - 1 \tag{5.2}$$

stores the list elements backwards beginning at the right end of the array `element`, and the formula

$$location(i) = (location(0) + i)\%\texttt{arrayLength} \tag{5.3}$$

stores elements beginning at any position in the array and wraps around to the front of the array, if necessary, to store the remaining elements. Figure 5.1(b) shows how the list [5, 2, 4, 8, 1] is stored when Equation 5.2 is used, and Figure 5.1(c) shows how this list is stored using Equation 5.3 and $location(0) = 7$. Equation 5.3 is used in Chapter 9 to map a queue into a one-dimensional array.

In our array representation of a linear list, we use a one-dimensional array `element` that holds the list elements as per Equation 5.1, a variable `listSize` that keeps track of the number of elements currently in the list, and a variable `arrayLength` that keeps track of the capacity of the array `element`. We may remove element e_i from the list by moving elements to its right down by 1. For example, to remove the element $e_1 = 2$ from the list of Figure 5.1(a), we have to move the elements $e_2 = 4$, $e_3 = 8$, and $e_4 = 1$, which are to the right of e_1, to positions 1, 2,

Copyrighted material

and 3 of the array `element`. Figure 5.2(a) shows the result. The shaded elements were moved.

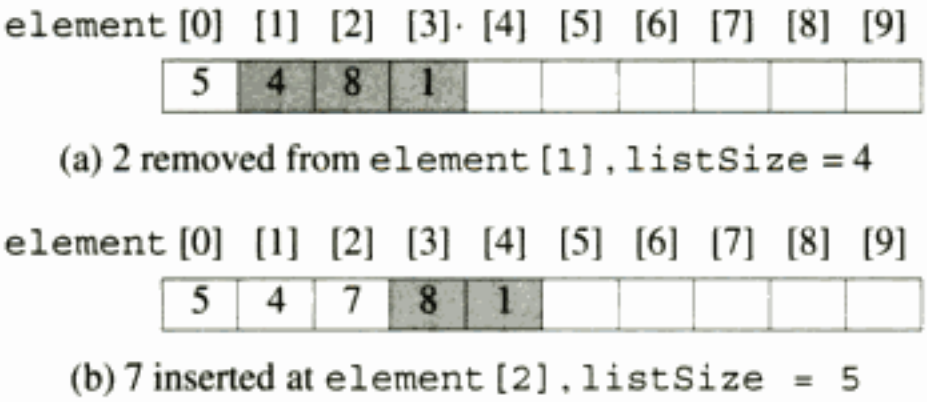


Figure 5.2 Removing and inserting an element

To insert an element so that it becomes element i of a list, we must first move the existing element e_i (if any) and all elements to its right one position right and then put the new element into position i of the array. For example, to insert 7 as the second element of the list of Figure 5.2(a), we first move elements e_2 and e_3 to the right by 1 and then put 7 into position 2 of the array. Figure 5.2(b) shows the result. The shaded elements were moved.

Before we can write an array class that implements the ADT *linearList*, we must decide on the data type of the array `element` and the length of this array. By making our linear list class a template class, we avoid having to make the first decision. For the second decision, we note that the array `element` must be large enough to hold the maximum number of elements that might be in the list at any time. This maximum number is often difficult to estimate. To overcome this hurdle, we can ask the user to provide an estimate and then dynamically increase the length of the array `element` in case the user underestimated.

5.3.2 Changing the Length of a One-Dimensional Array

To increase or decrease the length of a one-dimensional array `a` that contains elements in positions `a[0:n-1]`, we first define an array of the new length, then copy the `n` elements from `a` to the new array, and finally change the value of `a` so that it references the new array. Program 5.2 gives the method `changeLength1D`, which performs these three tasks.

It takes $\Theta(1)$ time to create an array of length m. Notice that the invocation of `new` to create the new array may cause a `bad_alloc` exception to be thrown. If `new` is successful in creating the new array, Program 5.2 spends $\Theta(\mathtt{n})$ time copying elements from the source array into the destination array. Therefore, the complexity of Program 5.2 is $O(\mathtt{n}) = O(\mathtt{n})$.

Copyrighted material

```
template<class T>
void changeLength1D(T*& a, int oldLength, int newLength)
{
   if (newLength < 0)
      throw illegalParameterValue("new length must be >= 0");

   T* temp = new T[newLength];               // new array
   int number = min(oldLength, newLength);  // number to copy
   copy(a, a + number, temp);
   delete [] a;                              // deallocate old memory
   a = temp;
}
```

Program 5.2 Changing the length of a one-dimensional array

When an array is used to represent a data structure whose size increases dynamically, the array length is often doubled whenever the array becomes full. This process is referred to as **array doubling**. When array doubling is used, the total time spent changing the array length is (in an asymptotic sense) no more than the total time spent inserting elements into the data structure (see Theorem 5.1).

5.3.3 The Class arrayList

Class Definition for arrayList

We define a C++ class `arrayList` that implements the ADT *linearList* using Equation 5.1. Program 5.3 gives the class header, the data members, and the function/method prototypes. Since `arrayList` is to be a concrete class that extends the abstract class `linearList`, it must provide an implementation of all the methods of the abstract class `linearList`. The class `arrayList` may, however, contain methods that are not declared in `linearList`. Our class, for example, contains the methods `capacity` and `checkIndex` that are not declared in `linearList`. The method `capacity`, gives the current length of the array `element` while the method `checkIndex` verifies that a specified element index is in the permissible range 0 through `listSize` – 1.

Constructor and Copy Constructor for arrayList

Program 5.4 gives the class constructor as well as the copy constructor. The constructor creates an array whose length is `initialCapacity`; the default value for this length is 10. Also, it sets `arrayLength` to `initialCapacity` and `listSize` to 0. The copy constructor makes a copy or clone of an object. This constructor is invoked, for example, when an object is passed by value to a function as well as

Copyrighted material

6.1 SINGLY LINKED LISTS AND CHAINS

6.1.1 The Representation

In a linked representation each element of an instance of a data object is represented in a cell or node. The nodes, however, need not be components of an array, and no formula is used to locate individual elements. Instead, each node keeps explicit information about the location of other relevant nodes. This explicit information about the location of another node is called a **link** or **pointer**.

Let $L = (e_0, e_1, \cdots, e_{n-1})$ be a linear list. In one possible linked representation for this list, each element e_i is represented in a separate node. Each node has exactly one link field that is used to locate the next element in the linear list. So the node for e_i links to that for e_{i+1}, $0 \leq i < n-1$. The node for e_{n-1} has no node to link to and so its link field is `NULL`. The variable `firstNode` locates the first node in the representation. Figure 6.1 shows the linked representation of the list $L = (e_0, e_1, ..., e_{n-1})$. Links are shown as arrows. To locate the element e_2, for example, we must start at `firstNode`; follow the pointer in `firstNode` to the next node; follow one more pointer to get to the node with e_2. In general, to locate the element with index *theIndex*, we must follow a sequence of *theIndex* pointers beginning at `firstNode`.

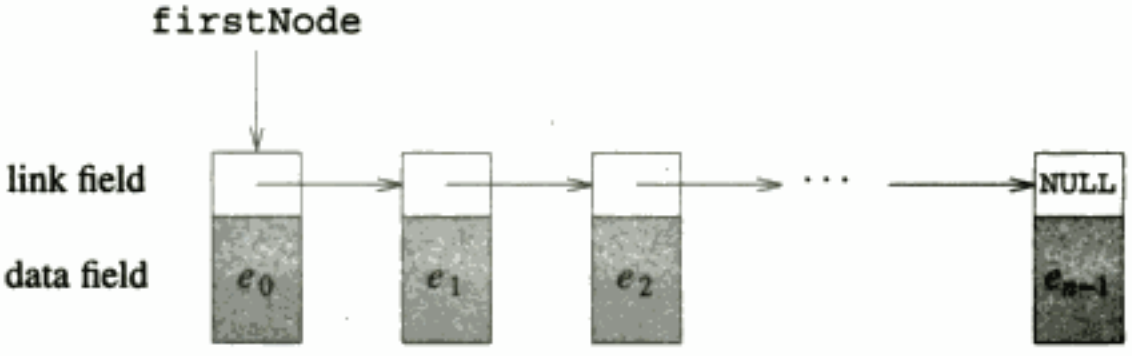


Figure 6.1 Linked representation of a linear list

Since each node in the linked representation of Figure 6.1 has exactly one link, the structure of this figure is called a **singly linked list**. Since the nodes are ordered from left to right with each node (other than the last one) linking to the next, and the last node has a `NULL` link, the structure is also called a **chain**.

To remove the element e_2 whose index is 2 from the chain of Figure 6.2, we do the following (note that e_2 is in the third node of the chain):

- Locate the second node (i.e., the node with e_1) in the chain.
- Link the second node to the fourth node.

Copyrighted material

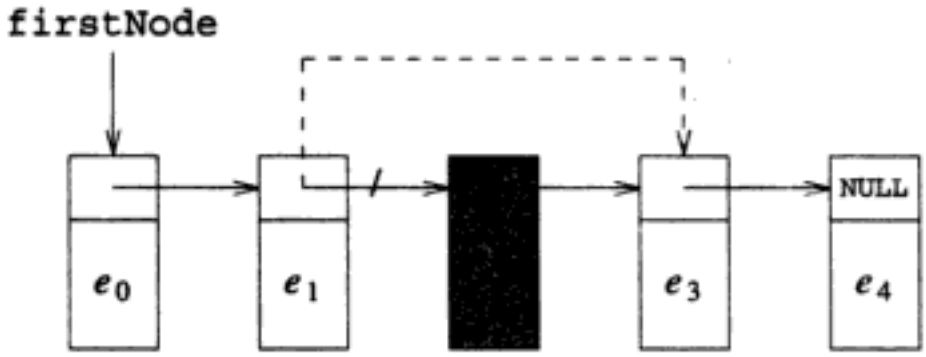


Figure 6.2 Removing e_2 from a 5-node chain

Notice that the removal of the third node from Figure 6.2 automatically decrements the index of the succeeding nodes by 1 (i.e., what were previously the fourth and fifth nodes of the chain become the third and fourth nodes). Because the nodes on a chain are always defined to be those nodes that can be reached following a sequence of pointers beginning at `firstNode`, we do not bother to change the pointer in the removed node (i.e., the former third node of the chain). Since the removed node is no longer reachable from `firstNode`, it is no longer part of the chain.

To insert a new element as the `index`th element in a chain, we need to first locate the `index-1`th element and then insert a new node just after it. Figure 6.3 shows the link changes needed for the two cases `index` = 0 and 0 < `index` ≤ `listSize`. Solid pointers exist prior to the insert, and those shown as broken (or dashed) lines exist following the insert.

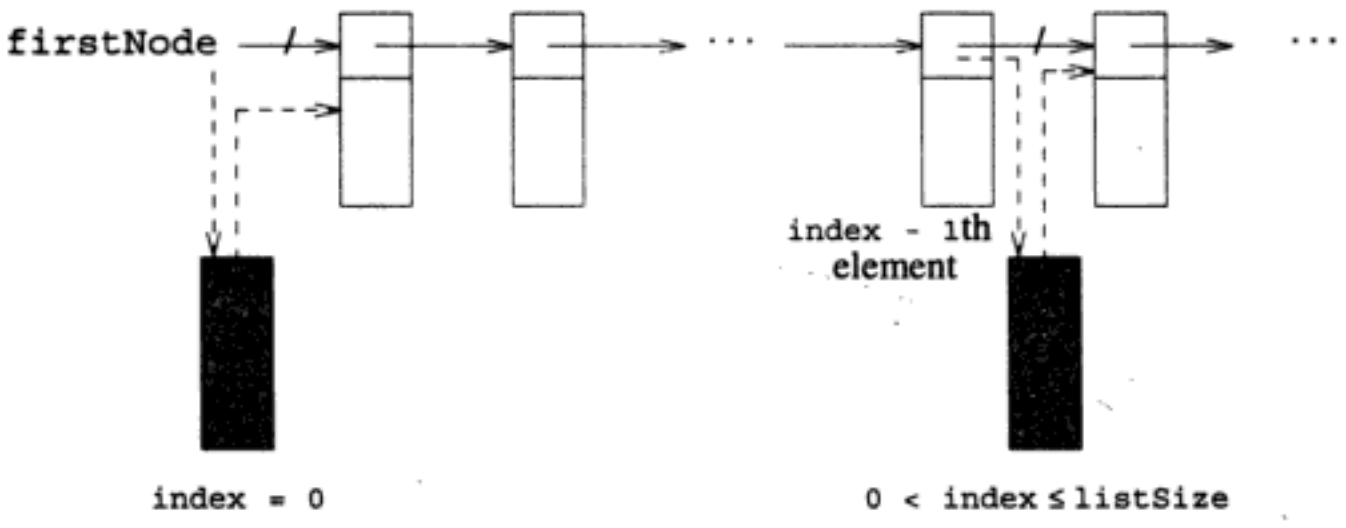


Figure 6.3 Insertion into a chain

Copyrighted material

6.1.2 The Struct chainNode

To represent a linear list as a chain, we define a struct **chainNode** and a class **chain**. Program 6.1 gives the struct **chainNode**, which defines the data type of the nodes used in Figure 6.1. The data member **element** holds a list element and is the data field of the node; the data member **next** holds a pointer to the next node in the chain and is the node's link field.

```
template <class T>
struct chainNode
{
   // data members
   T element;
   chainNode<T> *next;

   // methods
   chainNode() {}
   chainNode(const T& element)
      {this->element = element;}
   chainNode(const T& element, chainNode<T>* next)
      {this->element = element;
       this->next = next;}
};
```

Program 6.1 Struct definition for a chain node

Notice that two of the constructors of **chainNode** use the syntax **this->element** and **this->next** to access the data members of the constructed instance. This syntax is necessary to distinguish between the data members of the constructed instance and the formal parameters of the constructors because the data members and formal parameters have the same names.

6.1.3 The Class chain

Header, empty, and size

The class **chain** implements a linear list as a singly linked list of nodes in which the last node has the pointer **NULL**; that is, it implements a linear list as a chain of nodes. Program 6.2 gives the class header, data members and code for the **empty**, and **size** methods.

The data members are **firstNode** and **listSize**. **firstNode** is a pointer to the first node (i.e., the node for the zeroth element of the list) in the chain. When the chain has no first node, that is, when the chain is empty, **firstNode** is **NULL**.

Copyrighted material

```
template<class T>
class chain : public linearList<T>
{
   public:
      // constructor, copy constructor and destructor
      chain(int initialCapacity = 10);
      chain(const chain<T>&);
      ~chain();

      // ADT methods
      bool empty() const {return listSize == 0;}
      int size() const {return listSize;}
      T& get(int theIndex) const;
      int indexOf(const T& theElement) const;
      void erase(int theIndex);
      void insert(int theIndex, const T& theElement);
      void output(ostream& out) const;

   protected:
      void checkIndex(int theIndex) const;
            // throw illegalIndex if theIndex invalid
      chainNode<T>* firstNode;  // pointer to first node in chain
      int listSize;             // number of elements in list
};
```

Program 6.2 Header for the class `chain`

`listSize` gives the number of elements in the list, which equals the number of nodes in the chain.

Constructor and copy constructor

Program 6.3 gives the constructor and copy constructor for `chain`.

To create an empty chain, we need merely set the first node pointer `firstNode` to `NULL`. Unlike the case when a linear list is represented by an array, we do not allocate space for the expected maximum number of elements at the time the chain is created. Therefore, the user does not need to specify an estimate of this maximum size or provide an initial capacity for the list. Nevertheless, we have provided a constructor which has `initialCapacity` as a formal parameter in order to be compatible with the class `arrayList`. In particular, an application can create an array of type `linearList` and initialize the array components using either form of constructor as is shown below.

Copyrighted material

```
template<class T>
chain<T>::chain(int initialCapacity)
{// Constructor.
   if (initialCapacity < 1)
   {ostringstream s;
    s << "Initial capacity = " << initialCapacity << " Must be > 0";
    throw illegalParameterValue(s.str());
   }
   firstNode = NULL;
   listSize = 0;
}

template<class T>
chain<T>::chain(const chain<T>& theList)
{// Copy constructor.
   listSize = theList.listSize;

   if (listSize == 0)
   {// theList is empty
      firstNode = NULL;
      return;
   }

   // non-empty list
   chainNode<T>* sourceNode = theList.firstNode;
                    // node in theList to copy from
   firstNode = new chainNode<T>(sourceNode->element);
                    // copy first element of theList
   sourceNode = sourceNode->next;
   chainNode<T>* targetNode = firstNode;
                    // current last node in *this
   while (sourceNode != NULL)
   {// copy remaining elements
      targetNode->next = new chainNode<T>(sourceNode->element);
      targetNode = targetNode->next;
      sourceNode = sourceNode->next;
   }
   targetNode->next = NULL; // end the chain
}
```

Program 6.3 Constructor and copy constructor for `chain`

Copyrighted material

```
linearList<int>* list[10];
list[0] = new arrayList<int>(20);
list[1] = new arrayList<int>();
list[2] = new chain<int>(5);
list[3] = new chain<int>;
```

The complexity of the constructor is $\Theta(1)$. The copy constructor makes a clone of the chain **theChain** by copying the nodes of **theChain** one node at a time. The complexity of the copy constructor is $O($**theList.listSize**$)$.

The destructor

Program 6.4 gives the destructor for **chain**. The destructor deletes the nodes of the chain one by one. The strategy used in our destructor code is to repeatedly delete the first node in the chain until the remaining chain has no first node. Note that we must save the pointer to the second node in a variable such as **nextNode** before we delete the first node. The complexity of the destructor is $O($**listSize**$)$.

```
template<class T>
chain<T>::~chain()
{// Chain destructor. Delete all nodes in chain.
   while (firstNode != NULL)
   {// delete firstNode
      chainNode<T>* nextNode = firstNode->next;
      delete firstNode;
      firstNode = nextNode;
   }
}
```

Program 6.4 Destructor for **chain**

The method get

When an array representation is used, we locate a list element by evaluating a (usually simple) formula. To find the **theIndex**th element of a chain, however, we must start at the first node and follow the **next** links until we reach the desired node; that is, we must follow **theIndex** number of links. *We cannot access the desired node by performing arithmetic on the value of* **firstNode**. Program 6.5 gives the code. The method **checkIndex** is the same as that defined for **arrayList**. The complexity of **chain<T>::get** is $O($**theIndex**$)$, while that of **arrayList<T>::get** is $\Theta(1)$.

Copyrighted material

```
template<class T>
T& chain<T>::get(int theIndex) const
{// Return element whose index is theIndex.
 // Throw illegalIndex exception if no such element.
   checkIndex(theIndex);

   // move to desired node
   chainNode<T>* currentNode = firstNode;
   for (int i = 0; i < theIndex; i++)
      currentNode = currentNode->next;

   return currentNode->element;
}
```

Program 6.5 Method to return the `theIndex`th element

The method indexOf

Program 6.6 gives the code for the method `chain<T>::indexOf`. This code differs from the code for `arrayList<T>::indexOf` primarily in the mechanism used to go from one list element to the next. In the case of an array list, we go from one element to the next by performing some arithmetic on the location of the current element (when Equation 5.1 is used, we add 1 to the current location to get to the next location). When a chain is used, the only way to go from one node to the next is to follow the link or pointer in the current node. The complexity of `chain<T>::indexOf` is O(`listSize`).

The method erase

Program 6.7 gives the code for the erase operation. There are three cases to consider:

- `theIndex` $<$ 0 or `theIndex` $\geq$ `listSize`. In this case the erase operation fails because there is no `theIndex`th element to erase. This case implicitly covers the case when the chain is empty.
- The zeroth element is to be erased from a nonempty chain.
- An element other that the zeroth element is to be erased.

To get a feel for Program 6.7, manually try it on an initially empty list as well as on lists that contain at least one node. In addition, try out values of `theIndex` such as `theIndex` $<$ 0, `theIndex` $=$ 0 (erase the zeroth element), `theIndex` $=$ `listSize-1` (erase the last element), `theIndex` $\geq$ `listSize`, and 0 $<$ `theIndex` $<$ `listSsize-1` (erase an interior element).

Copyrighted material

```
template<class T>
int chain<T>::indexOf(const T& theElement) const
{// Return index of first occurrence of theElement.
 // Return -1 if theElement not in list.

   // search the chain for theElement
   chainNode<T>* currentNode = firstNode;
   int index = 0;  // index of currentNode
   while (currentNode != NULL &&
          currentNode->element != theElement)
   {
      // move to next node
      currentNode = currentNode->next;
      index++;
   }

   // make sure we found matching element
   if (currentNode == NULL)
      return -1;
   else
      return index;
}
```

Program 6.6 Method to return the index of the first occurrence of `theElement`

The complexity of `chain<T>::erase` is O(`theIndex`), whereas the complexity of `arrayList<T>::erase` is O(`listSize-theIndex`). Therefore, the linked implementation of a linear list is expected to perform better than the array implementation for erases that are done from near the front of the list.

The method insert

Insertion and erasing work in a similar way. To insert a new element as the `theEndex`th one in a chain, we need to first locate the `theIndex-1`th element and then insert a new node just after it. Program 6.8 gives the code. Its complexity is O(`theIndex`).

Outputting a chain

Program 6.9 gives the code for the `output` method as well as for the overloading of the stream insertion operator `<<`. The code for `chain<T>::output` differs from that for `arrayList<T>::output` primarily in its use of the `next` pointer to go from

Copyrighted material

```
template<class T>
void chain<T>::erase(int theIndex)
{// Delete the element whose index is theIndex.
 // Throw illegalIndex exception if no such element.
   checkIndex(theIndex);

   // valid index, locate node with element to delete
   chainNode<T>* deleteNode;
   if (theIndex == 0)
   {// remove first node from chain
      deleteNode = firstNode;
      firstNode = firstNode->next;
   }
   else
   {  // use p to get to predecessor of desired node
      chainNode<T>* p = firstNode;
      for (int i = 0; i < theIndex - 1; i++)
         p = p->next;

      deleteNode = p->next;
      p->next = p->next->next; // remove deleteNode from chain
   }
   listSize--;
   delete deleteNode;
}
```

Program 6.7 Erase the `theIndex`th element

one node to the next. The complexity of `chain<T>::output` is the same as that of `arrayList<T>::output`, O(`listSize`).

The member class `iterator`

By using `next` pointers, we can efficiently move from a node to its successor node in the chain. However, in a chain, there is no efficient way to move from a node to its predecessor. Therefore, for a chain, we define only a forward iterator. Recall that for an `arrayList` we defined a bidirectional iterator that enabled us to move from any list element to both its successor element and its predecessor element in $O(1)$ time. Program 6.10 gives the code for some of the methods of he forward iterator for a chain. The complete code may be found at the Web site for this book.

The methods `chain<T>::begin` and `chain<T>::end` are defined as

Copyrighted material

```
template<class T>
void chain<T>::insert(int theIndex, const T& theElement)
{// Insert theElement so that its index is theIndex.
   if (theIndex < 0 || theIndex > listSize)
   {// invalid index
      ostringstream s;
      s << "index = " << theIndex << " size = " << listSize;
      throw illegalIndex(s.str());
   }

   if (theIndex == 0)
      // insert at front
      firstNode = new chainNode<T>(theElement, firstNode);
   else
   {  // find predecessor of new element
      chainNode<T>* p = firstNode;
      for (int i = 0; i < theIndex - 1; i++)
         p = p->next;

      // insert after p
      p->next = new chainNode<T>(theElement, p->next);
   }
   listSize++;
}
```

Program 6.8 Insert `theElement` as the `theIndex`th element of the chain

```
iterator begin() {return iterator(firstNode);}
iterator end() {return iterator(NULL);}
```

The difference in run times between using the `get` method and the `iterator` method to access the linear list elements in left-to-right order is quite dramatic when the list is represented as a chain. The time needed to access the `i`th element using `get` is $\Theta(\texttt{i})$. Therefore, the `get` method to examine the list elements one at a time takes $\Theta(\texttt{listSize}^2)$ time, whereas the iterator method takes only $\Theta(\texttt{listSize})$ time.

6.1.4 Extensions to the ADT *linearList*

In some applications of linear lists, we wish to perform operations in addition to those that are part of the abstract data type *linearList* (ADT 5.1). So it is useful to

Copyrighted material

```
template<class T>
void chain<T>::output(ostream& out) const
{// Put the list into the stream out.
   for (chainNode<T>* currentNode = firstNode;
                      currentNode != NULL;
                      currentNode = currentNode->next)
      out << currentNode->element << "  ";
}

// overload <<
template <class T>
ostream& operator<<(ostream& out, const chain<T>& x)
   {x.output(out); return out;}
```

Program 6.9 The method output

extend the ADT to include additional functions such as *clear* (remove all elements from the list) and *push_back*(*theElement*) (insert *theElement* at the end of the list). Program 6.11 gives the abstract class that corresponds to the extended ADT.

6.1.5 The Class extendedChain

We will develop a class **extendedChain** that provides a linked implementation of the abstract class **extendedLinearList**. The easiest way to arrive at the class **extendedChain** is to derive it from **chain**.

To efficiently insert an element at the end of a chain, we add a new data member **lastNode** that points to the last node in the chain. Using this pointer, we can append an element to a chain in $\Theta(1)$ time. However, the addition of this new data member requires us to make changes in the implementation of the methods **erase** and **insert** because these methods may change the last node in the chain. When these methods change the last node, they must also update the new data member **lastNode**. Therefore, the class **extendedChain** will declare the data member **lastNode**; provide modified implementations of the methods **erase** and **insert**; define the remaining pure virtual functions of **linearList** as invocations of the corresponding methods in the class **chain**; and provide implementations for the new methods **clear** and **push_back**.

Program 6.12 gives the code for the methods **clear** and **push_back**. The full code for **extendedChain** may be found at the Web site for this book.

Copyrighted material

```
class iterator
{
   public:
      // typedefs required by C++ for a forward iterator omitted

      // constructor
      iterator(chainNode<T>* theNode = NULL)
         {node = theNode;}

      // dereferencing operators
      T& operator*() const {return node->element;}
      T* operator->() const {return &node->element;}

      // increment
      iterator& operator++()   // preincrement
                {node = node->next; return *this;}
      iterator operator++(int) // postincrement
             {iterator old = *this;
              node = node->next;
              return old;
             }

      // equality testing
      bool operator!=(const iterator right) const
            {return node != right.node;}
      bool operator==(const iterator right) const
            {return node == right.node;}
   protected:
      chainNode<T>* node;
};
```

Program 6.10 The class `chain<T>::iterator`

6.1.6 Performance Measurement

Memory Comparison

In an array implementation of a linear list, we typically use array doubling when the array gets full and array halving when the array occupancy falls below 25 percent (note, however, that the STL container class `vector` increases array length by a factor of 1.5 and never decreases array length; our array list classes use array doubling and do not decrease array length when usage falls below 25 percent of

Copyrighted material

```
template<class T>
class extendedLinearList : linearList<T>
{
   public:
      virtual ~extendedLinearList() {}
      virtual void clear() = 0;
                  // empty the list
      virtual void push_back(const T& theElement) = 0;
                  // insert theElement at end of list
};
```

Program 6.11 Abstract class for extended linear list

```
template<class T>
void extendedChain<T>::clear()
{// Delete all nodes in chain.
   while (firstNode != NULL)
   {// delete firstNode
      chainNode<T>* nextNode = firstNode->next;
      delete firstNode;
      firstNode = nextNode;
   }
   listSize = 0;
}

template<class T>
void extendedChain<T>::push_back(const T& theElement)
{// Insert theElement at the end of the chain.
   chainNode<T>* newNode = new chainNode<T>(theElement, NULL);
   if (firstNode == NULL)
      // chain is empty
         firstNode = lastNode = newNode;
   else
   {  // attach next to lastNode
      lastNode->next = newNode;
      lastNode = newNode;
   }
   listSize++;
}
```

Program 6.12 The methods `clear` and `push_back` of `extendedChain<T>`

Copyrighted material

array capacity). Therefore, a linear list with n elements may reside in an array whose length is between n and $4n$. So space for between n and $4n$ elements is needed. When a chain is used, exactly n nodes, each with two fields, are alloted to the list. Therefore, the chain representation uses space for n elements and n pointers. Let s be the number of bytes required by an element and assume that a pointer requires 4 bytes. Ignoring the space required by class data members such as `size` and `firstNode`, the array representation of a linear list requires between ns and $4ns$ bytes while a singly-linked representation requires $n(s+4)$ bytes. For most applications this difference in the space requirements will not be a deciding factor in selecting the representation to use.

Run-Time Comparison

For the time requirements we expect `chain<T>::get` to be much slower than `arrayList<T>::get`, because the complexity of `chain<T>::get` is O(`listSize`), whereas the complexity of the `arrayList<T>::get` is $\Theta(1)$. This expectation is borne out by experiment. We constructed a 50,000-node chain by making successive inserts at the left end of an initially empty chain. Then we measured the total time required to perform the 50,000 get operations `get(i)`, $0 \leq$ `i` $<$ 50000. `chain<T>::get` took 13.2 seconds to perform this sequence of operations, while `arrayList<T>::get` took 1.0 ms—not a commendable showing for the `chain` class. Things do not get any better when we compare the times for the `indexOf`, `insert` and `erase` methods.

Figures 6.4 and 6.5 show the times taken by `arrayList` and `chain` for operation sequences as described in Section 5.6. The time for 50,000 `indexOf` operations using `chain<T>::indexOf` is approximately 6 times that when `arrayList<T>::indexOf` is used. The average-case insert and erase times for the class `chain` are about 33 and 46 times that for for the corresponding methods of `arrayList`.

operation	`arrayList`	`chain`
`get`	1.0 ms	13.2 s
`indexOf`	2.3 s	13.0 s
best-case inserts	2.1 ms	45.1 ms
average inserts	1.5 s	49.3 s
worst-case inserts	2.5 s	12.9 s
best-case erases	2.0 ms	2.1 ms
average erases	1.5 s	68.8 s
worst-case erases	2.5 s	12.9 s

Times for 50,000 operations

Figure 6.4 Time taken by different linear list implementations

Even though the insert and erase sequences (insert and erase from the right

Copyrighted material

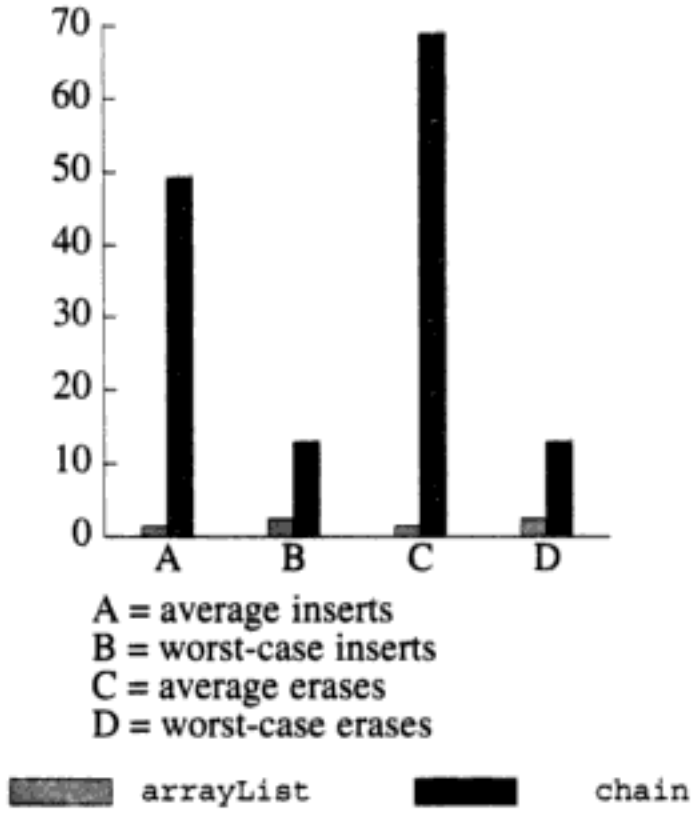


Figure 6.5 Average- and worst-case times, in seconds, for 50,000 operations

end of a chain) used in our worst-case test for chains cause our methods to do the maximum work, these sequences do not ensure maximum run time because of the cache effect (Section 4.5). In fact, you will notice that the measured worst-case times are smaller than the measured average times!

The average-case insert times were obtained by inserting into random positions of the linear list. Consequently, nodes that are adjacent in the chain are randomly located in memory. Hence when you move down the chain from left to right, you need to access random memory locations. This results in many cache misses. The same is true for the average-case erase experiment, which uses a randomly constructed chain. For the worst-case experiment, successive inserts are made at the right end of the chain. Since, for our experimental setup, successive calls to `new` return nodes that are adjacent in memory, nodes that are adjacent in the chain also are adjacent in memory. So when you move down the chain from left to right, you access adjacent memory; a memory access pattern that is favored by the cache management scheme. So the number of cache misses is reduced. This reduction in cache misses results in the anomalous run time measurements for worst-case inserts and erases relative to the average times.

The reported times for `chain<T>::get` and `chain<T>::indexOf` were obtained from a chain constructed by making inserts at the front of the chain (best-case

Copyrighted material

inserts). Since, in our experimental setup, successive calls to `new` return nodes that are adjacent in memory, adjacent nodes in the chain used to measure the times for the `get` and `indexOf` sequences occupy adjacent memory. So expect that performing the same sequence of `get` and `indexOf` operations in a chain created by making random inserts would take much more time. In fact, in a separate experiment, we determined this time to be 167 seconds and 165 seconds, respectively. So the time needed for our sequence of 50,000 `get`s in a randomly constructed chain is about 13 times that in the chain constructed for best-case inserts. This ratio is about the same for the `indexOf` operation. The linked representation gets a thumbs down as far as standard linear list operations are concerned!

Note that cache effects did not play a role in our comparison of best-case, average, and worst-case times for array representation (Section 5.6) because in all tests, the array elements are accessed from left to right and an array occupies a contiguous block of memory.

Are Pointers Any Good?

You are probably wondering whether you have just wasted a lot of time studying pointers. In Chapter 15 we develop balanced binary tree structures such as AVL and red-black trees. Indexed versions of these structures (e.g., the indexed AVL tree) may be used to represent a linear list (see Exercise 15.20). These structures use pointers and knock the socks off of `arrayList` as far as worst-case inserts and erases are concerned.

Despite the poor showing of chains in the linear list timing experiments we conducted, chains are more efficient than array linear list representations in several linear list applications. Section 6.5 gives a few of these applications. These applications require us to combine multiple lists into one or to remove and insert elements when the node just before the node to be removed or inserted is known because of other work done on the chain.

Two chains may be combined into one by linking the last node of one chain to the first node of the second. If we know both the first and last nodes of a chain, this combining is done in $O(1)$ time. To combine two array linear lists into one, we must copy the second into the array used by the first. This copying takes $\Theta($`size of second list`$)$ time. When we know the "just before node," the remove and insert operations of a chain run in $O(1)$ time; the complexity of these operations remains $O($`list size`$)$ when an array representation is used.

EXERCISES

1. Let `L` $= (a, b, c, d, e)$ be a linear list that is represented as a chain. Draw figures similar to Figure 6.1 showing the chain following each operation in the operation sequence: initial state, $insert(0, f)$, $insert(3, g)$, $insert(7, h)$, $erase(0)$, $erase(4)$.

Copyrighted material

2. Write code for the method `chain<T>::setSize(int theSize)` that makes the list size equal to `theSize`. If the original list size was less than the new one, `NULL` elements are added at the right end, and if the original size was more than the new one, the extra elements are removed from the right end. What is the complexity of your method? Test your code.

3. Write code for the method `chain<T>::set(theIndex, theElement)` that sets the element whose index is `theIndex` to `theElement`. Throw an exception in case `theIndex` is out of range. What is the complexity of your method? Test your code.

4. Write code for the method `chain<T>::removeRange(fromIndex, toIndex)` that removes all elements in the specified index range. What is the complexity of your method? Test your code.

5. Write code for the method `chain<T>::lastIndexOf(theElement)` that returns the index of the rightmost occurrence of `theElement`. A -1 is returned in case `theElement` is not in the list. What is the complexity of your method? Test your code.

6. Overload the operator `[]` so that the expression `x[i]` returns a reference to the `i`th element of the chain `x`. If the chain doesn't have an `i`th element, an `illegalIndex` exception is to be thrown. The statements `x[i] = y` and `y = x[i]` should work as expected. Test your code.

7. Overload the operator `==` so that the expression `x == y` returns true iff the two chains `x` are `y` are equal (i.e., the `i`th elements of both chains are equal for all `i`). Test your code.

8. Overload the operator `!=` so that the expression `x != y` returns true iff the two chains `x` are `y` are not equal (see Exercise 7). Test your code.

9. Overload the operator `<` so that the expression `x < y` returns true iff the chain `x` is lexically smaller than the chain `y` (see Exercise 7). Test your code.

10. Write code for the method `chain<T>::swap(theChain)`, which swaps the elements of the chains `*this` and `theChain`. What is the time complexity of your method? Test your code.

11. Write a method to convert an array linear list into a chain. Your method is a member of neither `arrayList` nor `chain`. Use the `get` method of `arrayList` and the `insert` method of `chain`. What is the time complexity of your method? Test the correctness of your code.

12. Write a method to convert a linear list that is an instance of `chain` into an equivalent list that is an instance of `arrayList`. Your method is a member of neither `arrayList` nor `chain`.

Copyrighted material

(a) First use only the `get` and `listSize` methods of `chain` and the `insert` method of `arrayList`. What is the time complexity of your method? Test the correctness of your code.

(b) Now use a chain iterator. What is the time complexity of the new code? Test your code using your own test data.

13. Add methods to `chain` to convert an `arrayList` to a `chain` and vice versa. Specifically, write a method `fromList(theList)` to convert the array linear list `theList` into a chain and another method `toList(theList)` to convert the chain `*this` into an array linear list `theList`. What is the time complexity of each method? Test the correctness of your code.

14. (a) Write code for the method `chain<T>::leftShift(i)` that shifts the list elements left by `i` positions. If `l` = [0, 1, 2, 3, 4], then `l.leftShift(2)` results in `l` = [2, 3, 4].

(b) What is the time complexity of your method?

(c) Test your code.

15. (a) Write code for the method `chain<T>::reverse`, which reverses the order of the elements in `*this`. Do the reversal in-place and do not allocate any new nodes.

(b) What is the complexity of your method?

(c) Test the correctness of your method by compiling and then executing the code. Use your own test data.

16. Write a nonmember method to reverse a chain. Use the member methods of `chain` to accomplish the reversal. What is the complexity of your method? Test the correctness of your method.

17. Let `a` and `b` be of type `extendedChain`.

(a) Write a method `meld` to create a new extended chain `c` that contains elements alternately from `a` and `b`, beginning with the first element of `a`. If you run out of elements in one of `a` and `b`, then append the remaining elements of the other extended chain to `c`. The complexity of your code should be linear in the lengths of the `a` and `b`. Note that `meld` is not a member method of the class `extendedChain`.

(b) Show that your code has linear complexity.

(c) Test the correctness of your method by compiling and then executing the code. Use your own test data.

18. Write code for the method `chain<T>::meld`. This method is similar to the method `meld` of Exercise 17. However, `a` and `b` as well as the result are of type `chain<T>`. You should use the same physical nodes used by the chains `a` and `b` to create the resulting melded chain. Following a call to `meld`, the input chains `a` and `b` are empty.

Copyrighted material

(a) Write the code for **meld**. The complexity of your code should be linear in the lengths of initial chains.

(b) Show that your code has linear complexity.

(c) Test the correctness of your code by compiling and then executing the code. Use your own test data.

19. Let **a** and **b** be of type **extendedChain**. Assume that the elements of **a** and **b** are of a type for which the relational operators **<**, **>**, **<=**, **>=**, **==**, and **!=** are defined. Further, assume that both **a** and **b** and are in sorted order (i.e., nondecreasing from left to right).

(a) Write a nonmember method **merge** to create a new sorted linear list **c** that contains all the elements in **a** and **b**.

(b) What is the complexity of your method?

(c) Test the correctness of your method by compiling and then executing the code. Use your own test data.

20. Redo Exercise 19 but this time write code for the method **chain<T>::merge**. You should use the same nodes as the two input chains use. Following the merge, the input chains are empty.

21. Let **c** be of type **extendedChain**.

(a) Write a nonmember method **split** to create two extended chains **a** and **b**. **a** contains all elements in odd positions of **c**, and **b** contains the remaining elements. Your method should not change the extended chain **c**.

(b) What is the complexity of your code?

(c) Test the correctness of your method by compiling and then executing the code. Use your own test data.

22. Write code for the method **chain<T>::split** that is similar to the method **split** of Exercise 21. However, the new method **split** destroys the input chain ***this** and uses its nodes to construct the chains **a** and **b**.

23. In a circular shift operation, the elements of a linear list are rotated clockwise by a given amount. For example, when the elements of **L** = [0, 1, 2, 3, 4] are shifted circularly by 2, the result is **L** = [2, 3, 4, 0, 1].

(a) Write code for the method **extendedChain<T>::circularShift(i)**, which performs a circular shift by **i** positions.

(b) Test your code.

Copyrighted material

24. Let **theChain** be a chain. Suppose that as we move to the right, we reverse the direction of the chain pointers; therefore, when we are at node **p**, the chain is split into two chains. One is a chain that begins at **p** and goes to the last node of **theChain**. The other begins at the node **l** that precedes **p** in **theChain** and goes back to the first node of **theChain**. Initially, **p** = **theChain.firstNode** and **l** = **NULL**.

    (a) Draw a chain with six nodes and show the configuration when **p** is at the third node and **l** is at the second.

    (b) Develop the class **moveLeftAndRightOnAChain**. The class constructor initializes the data members **l** and **p**. The public methods of **moveLeftAndRightOnAChain** are **moveRight**—move **l** and **p** one node right, **moveLeft**—move **l** and **p** one node left, **currentElement**—return the element at node **p**, and **previousElement**—return the element at node **l**.

    (c) Test your codes using suitable data.

25. Use the ideas of Exercise 24 to obtain a new version of the class **chain** of Program 6.2. The new version should permit you to move back and forth on a chain efficiently and to perform the methods of **linearList** even though the chain may be split into two chains as described in Exercise 24. For this version, add the data members **l** and **p** as in Exercise 24 and add the following public methods:

    (a) **reset**—Set **p** to **firstNode** and **l** to **NULL**.

    (b) **current()**—Return the element pointed to by **p**; throw an exception if the operation fails.

    (c) **atEnd**—Return **true** if **p** is at the last element of the list; return **false** otherwise.

    (d) **atFront**—Return **true** if **p** is at the first element of the list; return **false** otherwise.

    (e) **moveToNext**—Move **p** and **l** one position right; throw an exception if the operation fails.

    (f) **moveToPrevious**—Move **p** and **l** one position back; throw an exception if the operation fails.

    To implement the **insert**, **erase**, and **indexOf** methods efficiently, it will be useful to have another data member **currentElement** that gives you the index of the element to which **p** points (i.e., is it element 0, 1, 2, etc., of the list?). Test the correctness of your code using suitable test data.

26. Write code for the method **chain<T>::insertionSort**, which uses insertion sort (see Program 2.15) to reorder the chain elements into nondecreasing order.

Copyrighted material

Do not create new nodes or delete old ones. You may assume that the elements being sorted are of type for which the relational operators (<, >, etc.) are defined.

(a) What is the worst-case time complexity of your method? How much time does your method need if the elements are already in sorted order?

(b) Test the correctness of your method by compiling and then executing the code. Use your own test data.

27. Do Exercise 26 for the following sort methods (see Chapter 2 for descriptions):

(a) Bubble sort.

(b) Selection sort.

(c) Count or rank sort.

6.2 CIRCULAR LISTS AND HEADER NODES

Application codes that result from the use of chains can often be simplified and made to run faster by doing one or both of the following: (1) represent the linear list as a **singly linked circular list** (or simply **circular list**), rather than as a chain, and (2) add an additional node, called the **header node**, at the front. A circular list is obtained from a chain by linking the last node back to the first as in Figure 6.6(a). Figure 6.6(b) shows a nonempty circular list with a header node, and Figure 6.6(c) shows an empty circular list with a header node.

The use of header nodes is a very common practice when linked lists are used, as their presence generally leads to simpler and faster programs. Program 6.13 gives the constructor and `indexOf` methods for the class `circularListWithHeader`, which represents a linear list as a circular list with a header node. The constructor creates the configuration for an empty list (Figure 6.6(c)). The complexity of the constructor is $\Theta(1)$ and that of `indexOf` is $O($`listSize`$)$. Although `chain<T>::indexOf` and `circularListWithHeader<T<::indexOf` have the same complexity, the code for the latter method is slightly simpler. Since `circularListWithHeader<T>::indexOf` avoids the check `currentNode != NULL` that is made by `chain<T>::indexOf` on each iteration of its `while` loop, `circularListWithHeader<T>::indexOf` will run slightly faster than `chain<T>::indexOf` except possibly when we are looking for an element near the left end of the chain.

EXERCISES

28. Compare the run-time performance of the `indexOf` methods of Programs 6.6 and 6.13. Do this for both worst-case and average run times using linear lists of size 100; 1000; 10,000; and 100,000. Present your times in tabular form and in graph form.

Copyrighted material

CHAPTER 8

STACKS

BIRD'S-EYE VIEW

Stacks and queues are, perhaps, the most frequently used data structures. Both are restricted versions of the linear or ordered list data structure studied extensively in Chapters 5 and 6. In fact, stacks and queues are so widely used that the C++ STL provides the classes **stack** and **queue**, which are array implementations of these data structures. We will study stacks in this chapter and queues in the next. Even though C++ provides an implementation of a stack and a queue, we obtain our own stack and queue implementations just to learn how to implement these data structures.

The stack data structure is obtained from a linear list by restricting the insertions and removals to take place from the same end. As a result, a stack is a last-in-first-out (LIFO) structure. Since a stack is a special kind of linear list, it is natural to derive stack classes from corresponding linear list classes. Therefore, we may derive an array-stack class from any of the array linear list representations of Chapter 5; a linked-stack class may be derived from the class **chain** (Program 6.2). Although these derivations simplify the programming task, they result in code that incurs a significant run-time penalty. Since a stack is a very basic data structure that many applications employ, we also develop array- and linked-stack classes from scratch (i.e., not derived from any other class). These latter classes provide improved run-time performance over their derived counterparts.

Copyrighted material

Six application codes that make use of stacks are also developed. The first is a simple program to match left and right parentheses in an expression. The second is the classical Towers of Hanoi problem in which we move disks one at a time from a source tower to a destination tower using one intermediary tower; each tower operates as a stack. The third application uses stacks to represent shunting tracks in a railroad yard. The objective is to rearrange the cars in a train into the desired order. The fourth application is from the computer-aided design of circuits field. In this application we use a stack to determine whether a switch box can be feasibly routed. The fifth application revisits the offline equivalence class problem introduced in Section 6.5.4. A stack enables us to determine the equivalence classes in linear time. The final application considered in this chapter is the classical rat-in-a-maze problem in which we are to find a path from the entrance of a maze to its exit. You are urged to go through this application very carefully, as its treatment in this chapter illustrates many software-engineering principles. Additional stack applications appear in later chapters.

Copyrighted material

8.1 DEFINITION AND APPLICATIONS

Definition 8.1 *A* **stack** *is a linear list in which insertions (also called additions and pushes) and removals (also called deletions and pops) take place at the same end. This end is called the* **top***; the other end of the list is called the* **bottom**. ■

Figure 8.1(a) shows a stack with four elements. Suppose we wish to add element E to the stack of Figure 8.1(a). This element will have to be placed on top of element D, giving us the configuration of Figure 8.1(b). If we are to delete an element from the stack of Figure 8.1(b), it will be element E. Following the deletion, the configuration of Figure 8.1(a) results. If we perform three successive deletions on the stack of Figure 8.1(b), the stack of Figure 8.1(c) results.

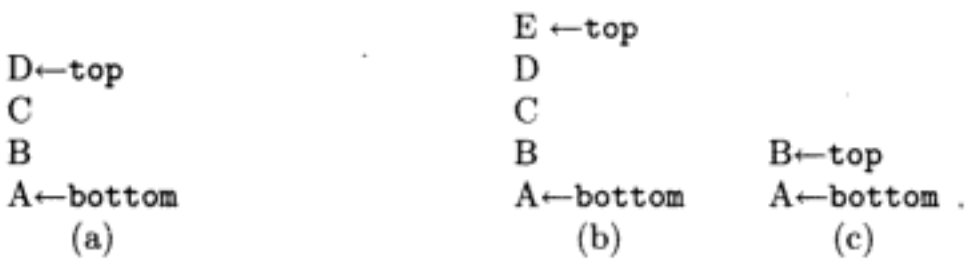


Figure 8.1 Stack configurations

From the preceding discussion, we see that a stack is a LIFO list. Lists of this type appear frequently in computing.

Example 8.1 [Stacks in the Real World]

- If you examine the paper tray of your printer (or copy machine), you will see that the next sheet that gets used is the one at the top; when you add a sheet to the paper tray, you add it to the top. So the paper tray maintains a stack of paper; the paper tray works in a LIFO manner. This LIFO behavior of the paper tray is quite convenient when you want to do that occasional letter on a preprinted letterhead sheet or you want to print the next page on a preprinted form—you simply put the letterhead sheet or form at the top of the paper tray and smile when the printer prints on the desired sheet.

- Walk into a cafeteria, and you'll see a stack of trays. When you get into the food line, you pick up a tray from the top of this stack (unless, of course, you spot a new tray not too far from the top); when the tray stack is replenished, trays are added at the top of the stack. So barring anomalous behavior (like picking up a new tray that is not at the stack top), the tray stack in a cafeteria operates just like the stack data structure we have defined—the tray stack works in a LIFO manner.

Copyrighted material

- The next time you are in a college bookstore at the start of a term, observe any pile of heavy text books. Each student who needs the book removes and purchases the book at the top of the pile. When the pile gets sufficiently low, a bookstore employee mysteriously appears and adds books to the top of the pile. The book pile works in a LIFO manner—the pile is a stack. ■

Example 8.2 [Recursion] How does your computer run a recursive method? Recursive methods are correctly executed using a **recursion stack**. When a method is invoked, a return address (i.e., the location of the program instruction to execute once the invoked method completes) and the values of all local variables and formal parameters of the invoked method are stored on the recursion stack. When a `return` is executed, the values of local variables and formal parameters are restored to their values prior to the method invocation (these prior values are at the top of the recursion stack) and program execution resumes from the return address, which is also at the top of the stack.

Suppose the recursive sum function (Program 1.31) is invoked from the function `outerFunction` using the statement

```
y = rSum(x,2);
```

This statement is compiled into code to invoke `rSum` and is followed by code to store the value returned by `rSum` into `y`. Let l_1 be the address of the first instruction in the code to store the returned value into `y`. The `return rSum` statement of Program 1.31 is compiled into code to invoke `rSum`, followed by code to add the returned value to `a[n - 1]`, followed by code to return the result of this addition. Let l_2 be the address of the first instruction in the code to add the returned value to `a[n - 1]`.

When the invocation `rSum(x,2)` is made from `outerFunction`, the return address (l_1) and the the values of the formal parameters and local variables of `rSum` are saved on the recursion stack as a tuple of the form

(return address, values of formal parameters, values of local variables)

Since `a` and `n` have unspecified values at this time, the tuple (l_1, *, *) (* denotes an unspecified value) is added to the recursion stack (note that `rSum` has no local variables) and the formal parameters of `rSum` are assigned their new values. The parameter `a` is assigned the value of `x`, which is a reference to element 0 of the array `x[]`, and `n` is assigned the value 2. Execution continues with the first instruction of `rSum`.

When `rSum` is invoked from within `rSum`, the tuple (l_2, x, 2) is added to the stack; the formal parameters of `rSum` are assigned their new values (x and 1); and we continue with the first instruction of `rSum`. The function `rSum` is invoked again from within `rSum`, and (l_2, x, 1) is added to the stack; the formal parameters are assigned their new values (x and 0); and we proceed to the first instruction of `rSum`. Now since `n` equals 0, the value 0 is to be returned by `rSum`. How do we know

Copyrighted material

whether we should return to l_1 or to l_2? This determination is made by removing the top tuple from the stack. The stack contents (from bottom to top) are

$$[(l_1, *, *), (l_2, x, 2), (l_2, x, 1)]$$

The top tuple $(l_2, x, 1)$ is removed from the stack, the values of the formal parameters and local variables of the function we are exiting (i.e., `rSum`) are reset (`a` is reset to x and `n` is reset to 1), and we continue with the instruction at l_2. The sum `0 + x[0]` is computed, and another return executed. At this time the recursion stack looks like this:

$$[(l_1, *, *), (l_2, x, 2)]$$

The top tuple $(l_2, x, 2)$ is removed from the stack, `a` is reset to x, `n` is reset to 2, the computed sum `0 + a[0]` is to be returned, and we continue with the instruction at l_2. This time `a[1]` is added to `0 + a[0]`, the top tuple $(l_1, *, *)$ is removed from the stack, `a` is set to *, `n` is set to *, the value `0 + a[0] + a[1]` is to be returned, and we continue at l_1. ■

EXERCISES

1. The following sequence of operations is done on an initially empty stack: push A, push B, pop, push T, push T, push U, pop, pop, push A, push D. Draw figures similar to those of Figure 8.1 to show the stack configuration after each operation.

2. Do Exercise 1 for the operation sequence: push S, push S, push T, push U. pop, pop, push A, push L, push G, pop, push C, push A, push B, pop, pop.

3. Identify three additional real-world applications of a stack.

4. Show the contents of the recursion stack following each invocation of and each return from the method `rSum` (Program 1.31). The initial invocation is `rSum(x,3)`.

5. Show the contents of the recursion stack following each invocation of and each return from the method `factorial` (Program 1.29). The initial invocation is `factorial(3)`.

6. Show the contents of the recursion stack following each invocation of and each return from the method `perm` (Program 1.32). The initial invocation is `perm(x, 0, 2)`.

Copyrighted material

8.2 THE ABSTRACT DATA TYPE

The ADT stack is specified in ADT 8.1. We have chosen the stack operation names to be the same as the method names used in the STL class `stack`.

AbstractDataType *stack*
{
instances
linear list of elements; one end is called the *bottom*; the other is the *top*;

operations
empty() : Return `true` if the stack is empty, return `false` otherwise;
size() : Return the number of elements in the stack;
top() : Return the top element of the stack;
pop() : Remove the top element from the stack;
push(x) : Add element x at the top of the stack;
}

ADT 8.1 The abstract data type stack

Program 8.1 gives the C++ abstract class that corresponds to the *stack* abstract data type.

8.3 ARRAY REPRESENTATION

Since a stack is a linear list with the restriction that additions and deletions take place at the same end, we may use any of the linear list representations of Section 5.3.3. When we identify the stack top with the right end of the array linear list, the `push` and `pop` operations correspond to the best case for linear list inserts and erases. Consequently, both operations take $O(1)$ time.

8.3.1 Implementation as a Derived Class

Program 8.2 gives the class `derivedArrayStack`, whose base classes are `arrayList` and `stack`.

The constructor for `derivedArrayStack` simply invokes the constructor for the base `arrayList`, which allocates a one-dimensional array whose capacity (length) is `initialCapacity`. The default value for `initialCapacity` is 10. The codes for the remaining methods of `derivedArrayStack` are also straightforward.

Copyrighted material

```
template<class T>
class stack
{
   public:
      virtual ~stack() {}
      virtual bool empty() const = 0;
                  // return true iff stack is empty
      virtual int size() const = 0;
                  // return number of elements in stack
      virtual T& top() = 0;
                  // return reference to the top element
      virtual void pop() = 0;
                  // remove the top element
      virtual void push(const T& theElement) = 0;
                  // insert theElement at the top of the stack
};
```

Program 8.1 The C++ abstract class `stack`

Complexity of `derivedArrayStack` Methods

The complexity of the constructor is $O(1)$ when `T` is a primitive data type and is $O($`initialCapacity`$)$ when `T` is a user-defined type. The complexity of `push` is $\Theta(1)$ except when we need to increase the capacity of the stack. In this latter case the complexity is $O($stack size$)$. The complexity of each of the remaining methods is $\Theta(1)$.

Comments on `derivedArrayStack`

The codes for `top` and `pop` check whether the stack is empty before they, respectively, invoke the base class methods `get` and `erase`. Since the invoked base-class methods will throw an exception when invoked with an empty stack, we can eliminate the stack empty check from `top` and `pop` without affecting the program outcome. However, since the `get` and `remove` methods of `arrayList` throw an exception of type `illegalIndex`, the user of our derived stack class will be bewildered upon having an exception of this type thrown when he/she invokes `top` and `pop`. An alternative is to replace the check for an empty stack by a `try-catch` construct in which the `catch` block catches the exception thrown by the base-class method and throws a new and meaningful exception in its place. Program 8.3 shows the code for the method `top` when we use this alternative. The corresponding class is called `derivedArrayStackWithCatch`.

The derivation from `arrayList` has the access modifier `private`. Consequently,

Copyrighted material

```
template<class T>
class derivedArrayStack : private arrayList<T>,
                          public stack<T>
{
   public:
      derivedArrayStack(int initialCapacity = 10)
        : arrayList<T> (initialCapacity) {}
      bool empty() const
           {return arrayList<T>::empty();}
      int size() const
          {return arrayList<T>::size();}
      T& top()
         {
            if (arrayList<T>::empty())
               throw stackEmpty();
            return get(arrayList<T>::size() - 1);
         }
      void pop()
           {
              if (arrayList<T>::empty())
                 throw stackEmpty();
              erase(arrayList<T>::size() - 1);
           }
      void push(const T& theElement)
           {insert(arrayList<T>::size(), theElement);}
};
```

Program 8.2 An array stack class derived from `arrayList`

```
T& top()
   {
      try {return get(arrayList<T>::size() - 1);}
      catch (illegalIndex)
         {throw stackEmpty();}
   }
```

Program 8.3 Implementation of `top` using the `try-catch` construct

the public and protected methods and data members of `arrayList` are accessible only from within the code for `derivedArrayStack`. In particular, users of the de-

Copyrighted material

fined stack class are unable to access `arrayList` methods such as `get`, `insert`, and `erase`. Hence the LIFO stack discipline is enforced on instances of type `derivedArrayStack`.

An alternative, but very similar, implementation of an array stack would use a data member `stack` of type `arrayList` and define the stack methods in terms of operations on the linear list `stack`. The code would be quite similar to that of Program 8.2. Alternatively, the data member `stack` could be an array of type `T`, and the code for the `stack` methods would not employ any method of `linearList`. We explore this alternative in the next subsection.

8.3.2 The Class arrayStack

When we obtain a stack class by extending a linear list class as was done in Program 8.2, we pay a performance penalty. For example, whenever we add an element to a stack, the `push` method invokes `arrayList<T>::insert`, which does an index check, a possible array doubling, and a `copy_backward` prior to actually inserting the new element. The index check and `copy_backward` are unnecessary because when we add an element to a stack, the element is always added to the right end of the linear list.

One way to arrive at a faster implementation of an array stack is to develop a class that employs an array `stack` to hold the stack elements. Program 8.4 gives the class `arrayStack` that does just this. The bottom element of the stack is stored in `stack[0]`, and the top element in `stack[top]`. The codes for the methods of `arrayStack` may be arrived at from those of `arrayList` by eliminating the redundant code in `arrayList`. The asymptotic complexity of each method of `arrayStack` is the same as that of the corresponding method of `derivedArrayStack`.

8.3.3 Performance Measurement

Even though the array stack classes `arrayStack` (`AS`), `derivedArrayStack` (`DAS`), and the C++ STL container class `stack` (STL) implement all methods of the stack ADT so as to have the same asymptotic complexity, the observed performance of the methods is expected to be different for each class.

Define an n-sequence to be a sequence of n `push` operations followed by an alternating sequence of n `top` and n `pop` operations. Figure 8.2 gives the measured times to perform a 50,000,000-sequence, and Figure 8.3 shows these time as a bar chart. For all stack classes except the STL stack class, we obtained the run times for the cases: (1) start with a stack having the default capacity and (2) start with a stack whose initial capacity is 50,000,000. We did not try (2) for the STL stack class because this class does not have a constructor that allows us to specify an initial capacity.

The STL class `stack` took 2 times the time taken by `arrayStack` to perform a 50,000,000-sequence when the initial stack capacity is the default capacity. The performance ratio jumps to 3.7 when we start with an `arrayStack` whose capacity

Copyrighted material

```
template<class T>
class arrayStack : public stack<T>
{
   public:
      arrayStack(int initialCapacity = 10);
      ~arrayStack() {delete [] stack;}
      bool empty() const {return stackTop == -1;}
      int size() const
          {return stackTop + 1;}
      T& top()
         {
            if (stackTop == -1)
               throw stackEmpty();
            return stack[stackTop];
         }
      void pop()
           {
              if (stackTop == -1)
                 throw stackEmpty();
              stack[stackTop--].~T();  // destructor for T
           }
      void push(const T& theElement);
   private:
      int stackTop;          // current top of stack
      int arrayLength;       // stack capacity
      T *stack;              // element array
};
```

Program 8.4 The class `arrayStack` (continues)

	`initialCapacity`	
Class	default	50,000,000
`arrayStack`	2.7	1.5
`derivedArrayStack`	7.5	6.3
`stack`	5.6	-

Times are in seconds

Figure 8.2 Time taken by different array stack implementations

Copyrighted material

```
template<class T>
arrayStack<T>::arrayStack(int initialCapacity)
{// Constructor.
   if (initialCapacity < 1)
   {ostringstream s;
    s << "Initial capacity = " << initialCapacity << " Must be > 0";
    throw illegalParameterValue(s.str());
   }
   arrayLength = initialCapacity;
   stack = new T[arrayLength];
   stackTop = -1;
}

template<class T>
void arrayStack<T>::push(const T& theElement)
{// Add theElement to stack.
   if (stackTop == arrayLength - 1)
      {// no space, double capacity
         changeLength1D(stack, arrayLength, 2 * arrayLength);
         arrayLength *= 2;
      }

   // add at stack top
   stack[++stackTop] = theElement;
}
```

Program 8.4 The class **arrayStack** (concluded)

is 50,000,000 (this jump is due mainly to the fact that **stack** does not allow you to specify an initial capacity and so array resizing cannot be avoided). It is more appropriate to compare the performance of the STL class with that of **derived-ArrayStack** because both of these classes derive from other concrete linear list classes. The STL class **stack** derives from the STL class **deque** (Exercise 9.9) while **derivedArrayStack** derives from **arrayList**. **stack** has a better performance than **derivedArrayStack** primarily because **deque** does no index checking while **array-List** does index checking.

An interesting (and expected) observation is that the time spent resizing the array (this is just the difference in the time taken when the initial capacity is 10 and when it is 500,000) is approximately the same for both of stack implementations developed in this section (approximately 1.2 seconds). The time spent on array resizing is about 44 percent of the total time taken by **arrayStack** when we start

Copyrighted material

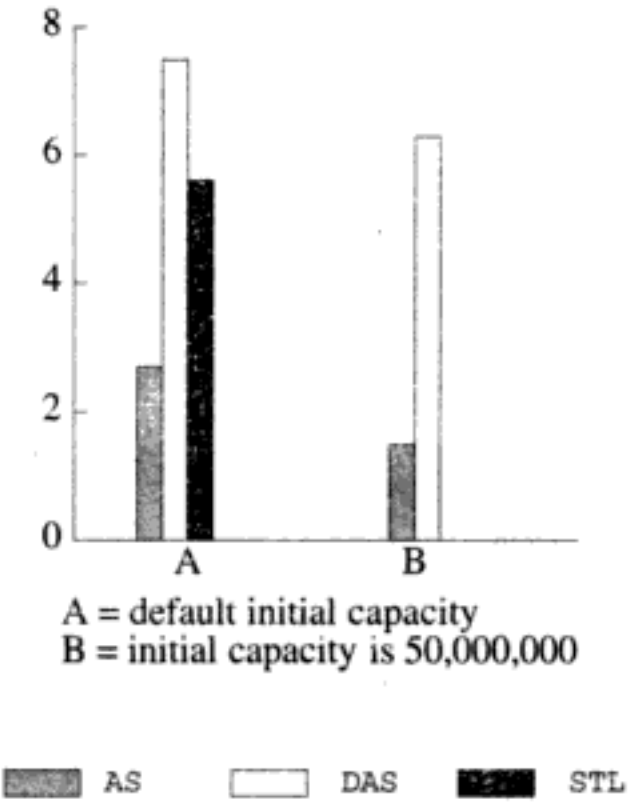


Figure 8.3 Stack run times in seconds

with an array having the default length 10.

EXERCISES

7. (a) Extend the stack ADT by adding functions to
   i. Input a stack.
   ii. Convert a stack into a string suitable for output.
   iii. Split a stack in two. The first contains the bottom half elements, and the second the remaining elements.
   iv. Combine two stacks by placing all elements of the second stack on top of those in the first. The relative order of elements from the second stack is unchanged.

   (b) Define the abstract class `extendedStack` that extends the abstract class `stack` (Program 8.1) and includes the methods that correspond to the functions of (a).

   (c) Develop the concrete classes `extendedDerivedArrayStack` and `extendedArrayStack` whose base class is `extendedStack` and which also are, respectively, derived from `derivedArrayStack` and `arrayStack`.

Copyrighted material

(d) Test the correctness of your codes.

8. Consider the class **arrayStack**. Show that even though it is possible for an individual **push** operation to take Θ(stack size) time, the time taken by any sequence of n stack operations is $O(n)$.

9. Consider the class **arrayStack**.

   (a) As implemented in Program 8.4, the capacity of a stack (i.e., the length of the array **stack**) can increase but not decrease. To use space more efficiently, modify the implementation of **pop** so that you decrease the capacity to one-half of the current capacity whenever a **pop** reduces the stack occupancy below one-fourth of capacity.

   (b) Show that even though it is possible for an individual **push** and **pop** operation to take Θ(capacity) time, the time taken by any sequence of n stack operations is $O(n)$.

10. Develop the concrete class **stackWithArrayList** that derives from the abstract class **stack**. This class has the single data member **list** whose datatype is **arrayList<T>**. Comment on the relative merits of the classes **derivedArrayStack** and **stackWithArrayList**.

11. Develop the C++ class **twoStacks** in which a single array is used to represent two stacks. Peg the bottom of one stack at position 0 and the bottom of the other at position **arrayLength-1**. The two stacks grow toward the middle of the array (see Figure 8.4). Your class must contain methods to perform all operations of the ADT *stack* on each of the two stacks. Further the complexity of each method should be $O(1)$ excluding the time for array resizing.

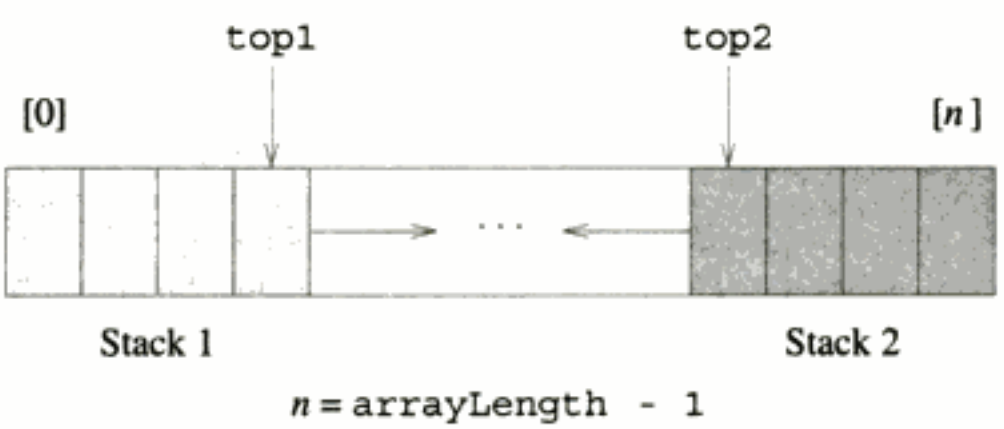


Figure 8.4 Two stacks in an array

Copyrighted material

8.4 LINKED REPRESENTATION

When using a chain to represent a stack, we must decide which end of the chain corresponds to the stack top. If we associate the right end of the chain with the stack top, then the stack operations `top`, `push`, and `pop` are implemented using invocations to the chain methods `get(size() - 1)`, `insert(size(), theElement)`, and `erase(size() - 1)`. Each of these chain operations takes $O($`size()`$)$ time. On the other hand, if we associate the left end of the chain with the stack top, then the chain operations to use are `get(0)`, `insert(0, theElement)`, and `erase(0)`. Each of these operations takes $\Theta(1)$ time. This analysis shows that we should use the left end of the chain to represent the stack top.

8.4.1 The Class `derivedLinkedStack`

The code for the class `derivedLinkedStack`, which derives from `chain` (Program 6.2) and which implements the abstract class `stack`, may be obtained from the code for `derivedArrayStack` (Program 8.2) by replacing the clause `private arrayList<T>` with the clause `private chain<T>`; replacing all occurrences of the name `derivedArrayStack` with the name `derivedLinkedStack`; and changing the index actual parameter to all uses of the methods `get`, `insert`, and `erase` to 0 so that these operations take place at the left end of the chain. What could be easier? By using the object-oriented programming principles of information hiding and encapsulation, we have greatly simplified program development. The complexity of each method of `derivedLinkedStack` (including the constructor and `push` methods) is $\Theta(1)$.

8.4.2 The Class `linkedStack`

As was the case with the class `arrayStack` (Program 8.4), we can improve the runtime performance by customizing the code and not deriving our linked-stack class from the class `chain`. Program 8.5 gives the customized code.

8.4.3 Performance Measurement

`derivedLinkedStack` and `linkedStack` took 41 and 40.5 seconds, respectively, to perform a 50,000,000-sequence (these times were obtained by measuring the times for a 10,000,000-sequence and multiplying by 5). Comparing with the times reported in Figure 8.2, we see that `linkedStack` takes 15 times the time `arrayStack` takes when started with a capacity of 10 and 27 times the time taken when the array stack's initial capacity is 50,000,000.

EXERCISES

12. In some stack applications the elements to be put on a stack are already in nodes of type `chainNode`. For these applications it is desirable to have the

Copyrighted material

```
template<class T>
class linkedStack : public stack<T>
{
   public:
      linkedStack(int initialCapacity = 10)
            {stackTop = NULL; stackSize = 0;}
      ~linkedStack();
      bool empty() const
           {return stackSize == 0;}
      int size() const
          {return stackSize;}
      T& top()
         {
            if (stackSize == 0)
               throw stackEmpty();
            return stackTop->element;
         }
      void pop();
      void push(const T& theElement)
           {
              stackTop = new chainNode<T>(theElement, stackTop);
              stackSize++;
           }
   private:
      chainNode<T>* stackTop;  // pointer to stack top
      int stackSize;           // number of elements in stack
};
```

Program 8.5 Customized linked stack (continues)

methods `pushNode(chainNode* theNode)`, which adds `theNode` to the top of the stack (notice that no call to `new` is made), and `popNode`, which removes and returns the top node of the stack.

(a) Write code for these methods

(b) Test your code.

(c) Compare the performance of a 10,000,000-sequence that uses `pushNode` and `popNode` with a 10,000,000-sequence that uses `push` and `pop`.

13. Develop the concrete class `extendedLinkedStack` that derives from both `linkedStack` and the abstract class `extendedStack` (see Exercise 7).

Copyrighted material

```
template<class T>
linkedStack<T>::~linkedStack()
{// Destructor.
   while (stackTop != NULL)
   {// delete top node
      chainNode<T>* nextNode = stackTop->next;
      delete stackTop;
      stackTop = nextNode;
   }
}

template<class T>
void linkedStack<T>::pop()
{// Delete top element.
   if (stackSize == 0)
      throw stackEmpty();

   chainNode<T>* nextNode = stackTop->next;
   delete stackTop;
   stackTop = nextNode;
   stackSize--;
}
```

Program 8.5 Customized linked stack (concluded)

14. Compare the performance of the array stack classes used in Figure 8.2 and the linked classes **derivedLinkedStack** and **linkedStack**. Do this by performing an alternating sequence of 10,000,000 **push** and **pop** operations. For the array classes start with the default initial capacity. Does array doubling occur in your experiment? Why?

8.5 APPLICATIONS

8.5.1 Parenthesis Matching

Problem Description

In this problem we are to match the left and right parentheses in a character string. For example, the string `(a*(b+c)+d)` has left parentheses at positions 0 and 3 and right parentheses at positions 7 and 10. The left parenthesis at position 0 matches the right at position 10, while the left parenthesis at position 3 matches the right parenthesis at position 7. In the string `(a+b))(`, the right parenthesis at position

Copyrighted material

5 has no matching left parenthesis, and the left parenthesis at position 6 has no matching right parenthesis. Our objective is to write a C++ program that inputs a string and outputs the pairs of matched parentheses as well as those parentheses for which there is no match. Notice that the parenthesis matching problem is equivalent to the problem of matching braces ({ and }) in a C++ program.

Solution Strategy

We observe that if we scan the input expression from left to right, then each right parenthesis is matched to the most recently seen unmatched left parenthesis. This observation motivates us to save the position of left parentheses on a stack as they are encountered in a left-to-right scan. When a right parenthesis is encountered, it is matched to the left parenthesis (if any) at the top of the stack. The matched left parenthesis is deleted from the stack.

C++ Implementation

Program 8.6 gives the C++ code.

Complexity

The time complexity of Program 8.6 is $O(n)$ where n is the length of the input expression. To see this, note that the program performs $O(n)$ **push** and $O(n)$ **pop** operations. Even though the complexity of an individual **push** operation is O(capacity) (because of array doubling), the complexity of $O(n)$ **push** operations is $O(n)$. The complexity of each **pop** operation is $O(1)$. Therefore, the complexity of $O(n)$ **pop** operations is $O(n)$.

8.5.2 Towers of Hanoi

Problem Description

The Towers of Hanoi problem is fashioned after the ancient Tower of Brahma ritual. According to legend, when the world was created, there was a diamond tower (tower 1) with 64 golden disks (Figure 8.5). The disks were of decreasing size and were stacked on the tower in decreasing order of size from bottom to top. Next to this tower are two other diamond towers (towers 2 and 3). Since the time of creation, Brahman priests have been attempting to move the disks from tower 1 to tower 2, using tower 3 for intermediate storage. As the disks are very heavy, they can be moved only one at a time. In addition, at no time can a disk be on top of a smaller disk. According to legend, the world will come to an end when the priests have completed their task.

In the Towers of Hanoi problem, we are given n disks and three towers. The disks are initially stacked on tower 1 in decreasing order of size from bottom to top. We are to move the disks to tower 2, one disk at a time, such that no disk is ever

Copyrighted material

```
void printMatchedPairs(string expr)
{// Parenthesis matching.
   arrayStack<int> s;
   int length = (int) expr.size();

   // scan expression expr for ( and )
   for (int i = 0; i < length; i++)
      if (expr.at(i) == '(')
         s.push(i);
      else
         if (expr.at(i) == ')')
     try
         {// remove location of matching '(' from stack
            cout << s.top() << ' ' << i << endl;
            s.pop();  // unstack match
         }
         catch (stackEmpty)
         {// stack was empty, no match exists
            cout << "No match for right parenthesis"
                 << " at " << i << endl;
         }

   // remaining '(' in stack are unmatched
   while (!s.empty())
   {
      cout << "No match for left parenthesis at "
           << s.top() << endl;
      s.pop();
   }
}
```

Program 8.6 Program to output matched parentheses

on top of a smaller one. You may wish to attempt a solution to this problem for n = 2, 3, and 4 before reading further.

Solution Strategy

A very elegant solution results from the use of recursion. To get the largest disk to the bottom of tower 2, we move the remaining $n-1$ disks to tower 3 and then move the largest to tower 2. Now we are left with the task of moving the $n-1$ disks from tower 3 to tower 2. To perform this task, we can use towers 1 and 2. We can

Copyrighted material

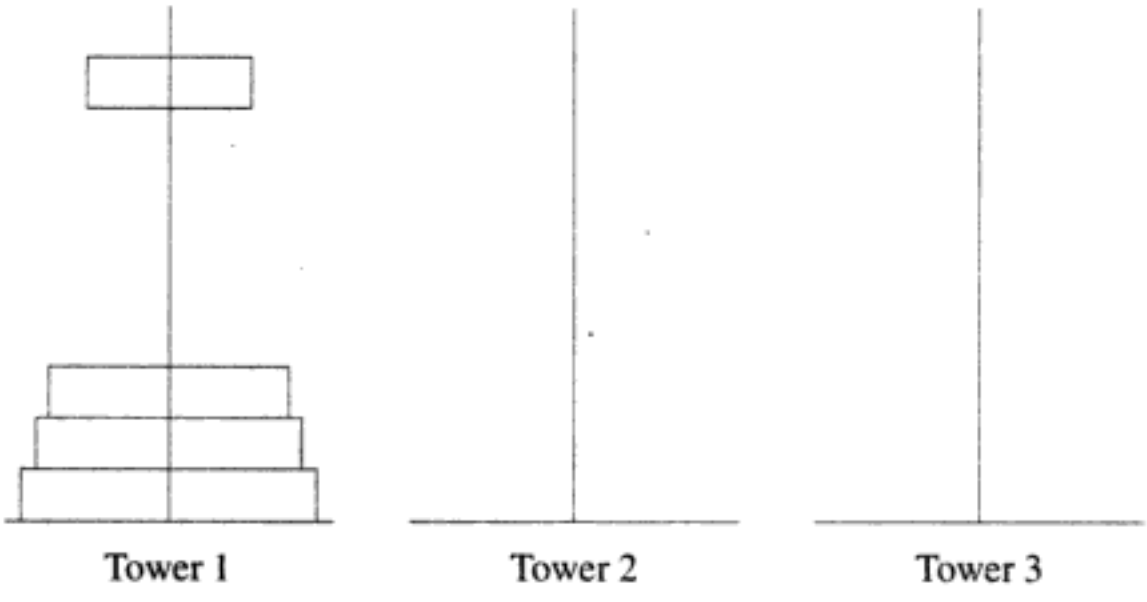


Figure 8.5 Towers of Hanoi

safely ignore the fact that tower 2 has a disk on it because this disk is larger than the disks being moved from tower 3. Therefore, we can place any disk on top of the disk on tower 2.

First Implementation

Program 8.7 gives recursive C++ code for this solution. The initial invocation is `towersOfHanoi(n, 1, 2, 3)`. The correctness of Program 8.7 is easily established.

```
void towersOfHanoi(int n, int x, int y, int z)
{// Move the top n disks from tower x to tower y.
 // Use tower z for intermediate storage.
   if (n > 0)
   {
      towersOfHanoi(n-1, x, z, y);
      cout << "Move top disk from tower " << x
           << " to top of tower " << y << endl;
      towersOfHanoi(n-1, z, y, x);
   }
}
```

Program 8.7 Recursive method for Towers of Hanoi

Copyrighted material

Complexity

The time taken by Program 8.7 is proportional to the number of lines of output generated, and the number of lines output is equal to the number of disk moves performed. Examining Program 8.7, we obtain the following recurrence for the number of moves, $moves(n)$:

$$moves(n) = \begin{cases} 0 & n = 0 \\ 2moves(n-1) + 1 & n > 0 \end{cases}$$

This recurrence may be solved by using the substitution method of Chapter 2 (see Example 2.20). The result is $moves(n) = 2^n - 1$. We can show that $2^n - 1$ is, in fact, the least number of moves in which the disks can be moved. Since $n = 64$ in the Tower of Brahma, the Brahman priests will need quite a few years to finish their task. From the solution to the above recurrence, we conclude that the time complexity of `towersOfHanoi` is $\Theta(2^n)$ provided the method runs to completion.

Second Implementation

The output from Program 8.7 gives us the disk-move sequence needed to move the disks from tower 1 to tower 2. Suppose we wish to show the state (i.e., the disks together with their order bottom to top) of the three towers following each move. To show this state, we must store the state of the towers in memory and change the state of each as disks are moved. Following each move, we can output the tower states to an output device such as the computer screen, printer, or video recorder.

Since disks are removed from each tower in a LIFO manner, each tower may be represented as a stack. The three towers together contain exactly n disks at any time. Using linked stacks, we can get by with space for n elements. If array stacks are used, towers 1 and 2 must have a capacity of n disks each, while tower 3 must have a capacity of $n-1$. Therefore, we need space for a total of $3n-1$ disks. As our earlier analysis has shown, the time complexity of the Towers of Hanoi problem is exponential in n. So using a reasonable amount of computer time, the problem can be solved only for small values of n (say $n \leq 30$). For these small values of n, the difference in space required by the array and linked representations is sufficiently small that either may be used. Since the array implementations of a stack run faster than the linked implementations, we use an array implementation.

The code of Program 8.8 uses array stacks. `towersOfHanoi(n)` is just a preprocessor for the recursive method `moveAndShow`, which is modeled after the method of Program 8.7. The preprocessor creates the three stacks `tower[1:3]` that will store the states of the three towers. The disks are numbered 1 (smallest) through n (largest). Since the disks are modeled as integers, the data type for the stack elements is `int`. The initial configuration has all n disks in `tower[1]`; the remaining two towers have no disk. After constructing this initial configuration, the preprocessor invokes the method `moveAndShow`.

Copyrighted material

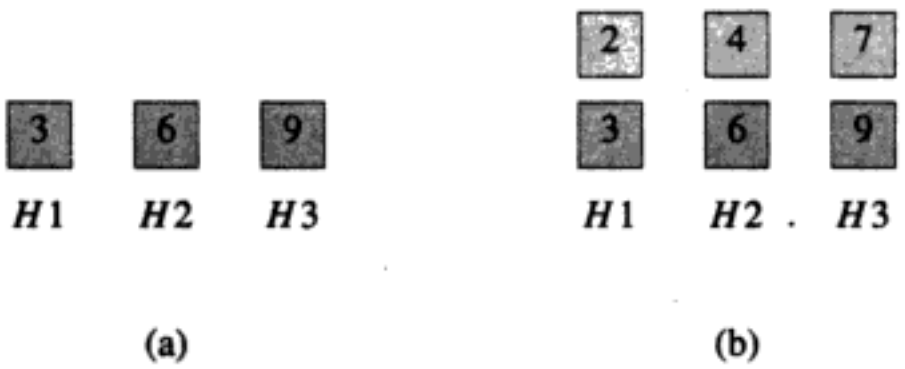


Figure 8.7 Track states

track are not in increasing order from top to bottom, the rearrangement cannot be completed. The current state of the holding tracks is shown in Figure 8.7(a).

Car 2 is considered next. It can be moved into any of the holding tracks while satisfying the requirement that car labels in any holding track be in increasing order, but moving it to $H1$ is preferred. If car 2 is moved to $H3$, then we have no place to move cars 7 and 8. If we move it to $H2$, then the next car, car 4, will have to be moved to $H3$ and we will have no place for cars 5, 7, and 8. *The least restrictions on future car placement arise when the new car u is moved to the holding track that has at its top a car with smallest label v such that $v > u$.* We will use this **assignment rule** to select the holding track.

When car 4 is considered, the cars at the top of the three holding tracks are 2, 6, and 9. Using our assignment rule, car 4 is moved to $H2$. Car 7 is then moved to $H3$. Figure 8.7(b) shows the current state of the holding tracks. The next car, car 1, is moved to the output track. It is now time to move car 2 from $H1$ to the output track. Next car 3 is moved from $H1$, and then car 4 is moved from $H2$. No other cars can be moved to the output at this time.

The next input car, car 8, is moved to $H1$. Then car 5 is moved from the input track to the output track. Following this move, car 6 is moved from $H2$. Then car 7 is moved from $H3$, car 8 from $H1$, and car 9 from $H3$.

While three holding tracks are sufficient to rearrange the cars from the initial ordering of Figure 8.6(a), other initial arrangements may need more tracks. For example, the initial arrangement 1, n, $n-1$, ..., 2 requires $n-1$ holding tracks.

C++ Implementation

We use k array stacks, `track[1:k]`, to represent the k holding tracks. Array stacks are used because they are faster than linked stacks. Program 8.9 gives the global variables we use.

The function `railroad` (Program 8.10) determines a sequence of moves that results in rearranging cars with initial ordering `inputOrder[1:theNumberOfCars]`

Copyrighted material

9.1 DEFINITION AND APPLICATIONS

Definition 9.1 *A* **queue** *is a linear list in which insertions (also called additions and puts) and deletions (also called removals) take place at different ends. The end at which new elements are added is called the* **back** *or* **rear** *, and that from which old elements are deleted is called the* **front**. ■

A queue with three elements is shown in Figure 9.1(a). The first element we delete from the queue of Figure 9.1(a) is A. Following the deletion, the configuration of Figure 9.1(b) results. To add element D to the queue of Figure 9.1(b), we must place it after element C. The new configuration is shown in Figure 9.1(c).

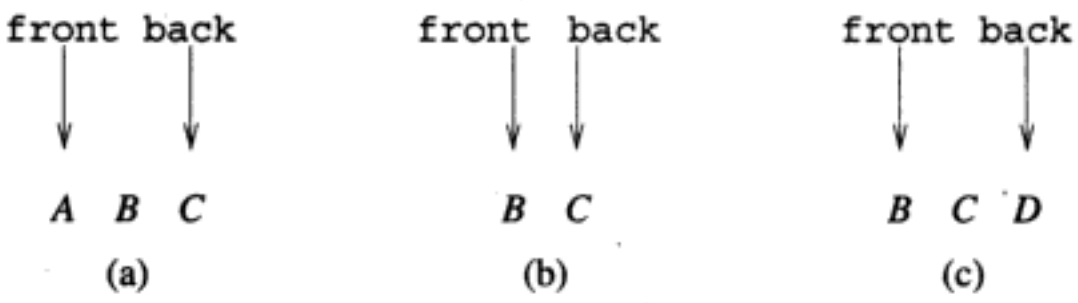


Figure 9.1 Sample queues

So a queue is a FIFO list, whereas a stack is a LIFO list.

Example 9.1 [Queues in the Real World]

- Although the stack of trays in a cafeteria works in a LIFO manner (see Example 8.1), the food line you stand in works in a FIFO manner. Customers exit at the checkout register in the order in which they entered the food line—the food line is a queue. Most other lines you find yourself in—the check-out line at a store, the line in front of a bank teller, the line of cars waiting at a car wash, the line at the postal center—also work in a FIFO manner.

- In a soda vending machine, you have a column of soda cans for each variety of soda dispensed. In each column cans are piled one on top of another. When you buy a can of soda, the can at the bottom of a column is given; when the stock is replenished, cans are added to the top of the column. The vending machine dispenses soda cans in a FIFO manner; the machine maintains a queue for each variety of soda dispensed.

- In a distributed system a single queue often feeds a bank of queues. A distributed system that has m servers has a bank of m server queues, one for each server. In addition, there is a queue called the broker or trader queue. Requests for service are first queued in the broker queue; a broker (or trader

Copyrighted material

or dispatcher) examines the service requests in the broker queue in a FIFO order and sends each request to the queue for the most appropriate server; servers handle the service requests in their queues in a FIFO order. Two specific examples follow.

1. In a distributed file system, computer files are replicated across file servers so as to provide a better level of service. All requests for a file are first queued in a broker queue; a broker dispatches each request to the least loaded server that has a copy of the requested file; the dispatched request waits its turn in the queue for the file server to which it is dispatched.
2. Voter polling stations provide a rather interesting application of queues. When you arrive at the polling station, you join a broker queue. When you get to the front of the broker queue, a polling volunteer directs you to a server queue based on the first letter of your surname. At the head of each server queue is a volunteer who checks your ID, gets your signature, and issues a ballot card. Once you have a card, you get into another queue and wait for a booth where you can punch holes in the ballot card to select your desired candidates. ■

EXERCISES

1. The following sequence of operations is done on an initially empty queue: add X, add Y, remove, add D, add A, remove, add T, add A. Draw figures similar to those of Figure 9.1 to show the queue configuration after each operation.
2. Identify three additional real-world applications of a queue. Do not include applications that involve people in a single line or vending machines. Distributed systems that involve people are okay.
3. Identify three real-world applications in which a stack is used sometimes and a queue is used at other times. For example, if you examine napkin dispensers, you will notice that some work as a stack and others work as a queue.
4. Which applications from Section 8.5 can use a queue instead of a stack without affecting the correctness of the program?

9.2 THE ABSTRACT DATA TYPE

The ADT queue is specified in ADT 9.1. The ADT function names are the same as those used in the C++ STL container class **queue**.

Program 9.1 gives the C++ abstract class that corresponds to ADT 9.1.

Copyrighted material

AbstractDataType *queue*
{
instances
ordered list of elements; one end is called the front; the other is the back;

operations
empty() : Return **true** if the queue is empty, return **false** otherwise;

size() : Return the number of elements in the queue;

front() : Return the front element of the queue;

back() : Return the back element of the queue;

pop() : Remove an element from the front of the queue;

push(x) : Add element x at the back of the queue;
}

ADT 9.1 The abstract data type queue

```
template<class T>
class queue
{
   public:
      virtual ~queue() {}
      virtual bool empty() const = 0;
                  // return true iff queue is empty
      virtual int size() const = 0;
                  // return number of elements in queue
      virtual T& front() = 0;
                  // return reference to the front element
      virtual T& back() = 0;
                  // return reference to the back element
      virtual void pop() = 0;
                  // remove the front element
      virtual void push(const T& theElement) = 0;
                  // add theElement at the back of the queue
};
```

Program 9.1 The abstract class queue

Copyrighted material

9.3 ARRAY REPRESENTATION

9.3.1 The Representation

Suppose that the queue elements are mapped into an array `queue` using Equation 9.1.

$$location(i) = i \tag{9.1}$$

This equation worked well for the array representation of linear lists and stacks. Element i of the queue is stored in `queue[i]`, $i \geq 0$. Let `arrayLength` be the length or capacity of the array `queue` and let `queueFront` and `queueBack`, respectively, be the locations of the front and back elements of the queue. When Equation 9.1 is used, `queueFront` equals 0 and the queue size is `queueBack+1`. An empty queue has `queueBack` $= -1$. Using Equation 9.1, the queues of Figure 9.1 are represented as in Figure 9.2.

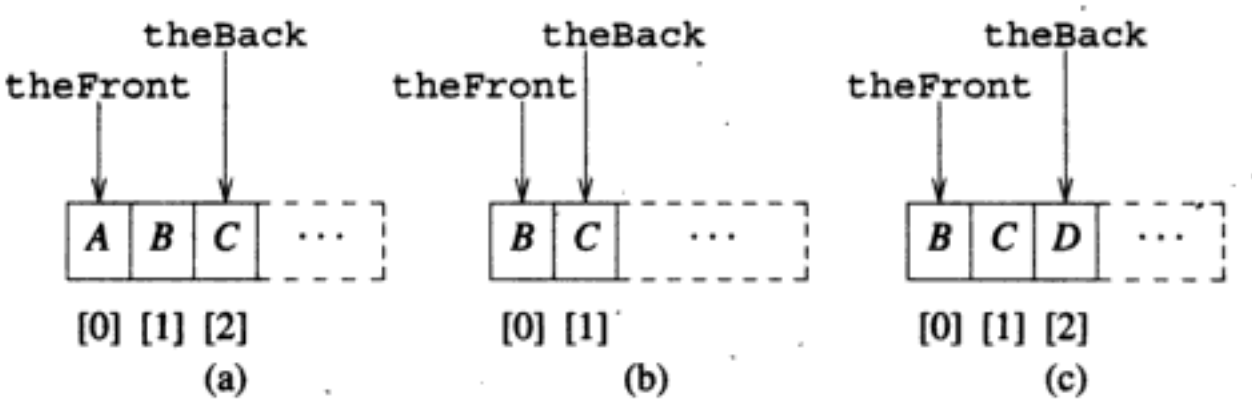


Figure 9.2 Queues of Figure 9.1 using Equation 9.1

To push an element into a queue, we need to increase `queueBack` by 1 and place the new element at `queue[queueBack]`, which means that a `push` operation requires $\Theta(1)$ time. To pop an element, we must slide the elements in positions 1 through `queueBack` one position down the array. Sliding the elements takes $\Theta(n)$ time where n is the number of elements in the queue following the pop.

We can pop an element in $\Theta(1)$ time if we use Equation 9.2 instead of Equation 9.1.

$$location(i) = location(\text{front element}) + i \tag{9.2}$$

Equation 9.2 does not require us to shift the queue one position left each time an element is popped from the queue. Instead, we simply increase *location*(front element) by 1. Figure 9.3 shows the representation of the queues of Figure 9.1 that results

Copyrighted material

when Equation 9.2 is used. Notice that `queueFront` = *location*(front element), `queueBack` = *location*(last element), and an empty queue has `queueBack` < `queueFront`.

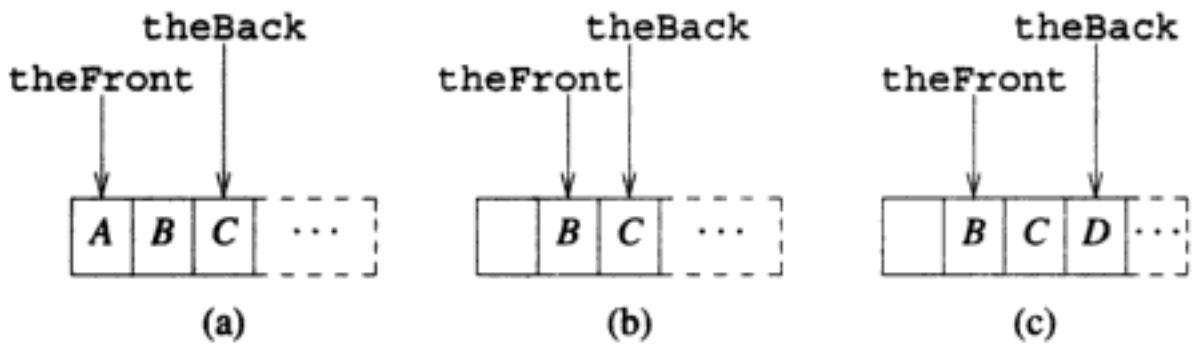


Figure 9.3 Queues of Figure 9.1 using Equation 9.2

As Figure 9.3(b) shows, each pop operation causes `queueFront` to move right by 1. Hence there will be times when `queueBack` = `arrayLength` - 1 and `queueFront` > 0. At these times the number of elements in the queue is less than `arrayLength`, and there is space for additional elements at the left end of the array. To continue inserting elements into the queue, we can shift all elements to the left end of the queue (as in Figure 9.4) and create space at the right end. This shifting increases the worst-case time for a push operation from $\Theta(1)$, when Equation 9.1 is used, to $\Theta($`arrayLength`$)$. So the trade-off for improved efficiency of the pop operation is a loss of efficiency for the push operation.

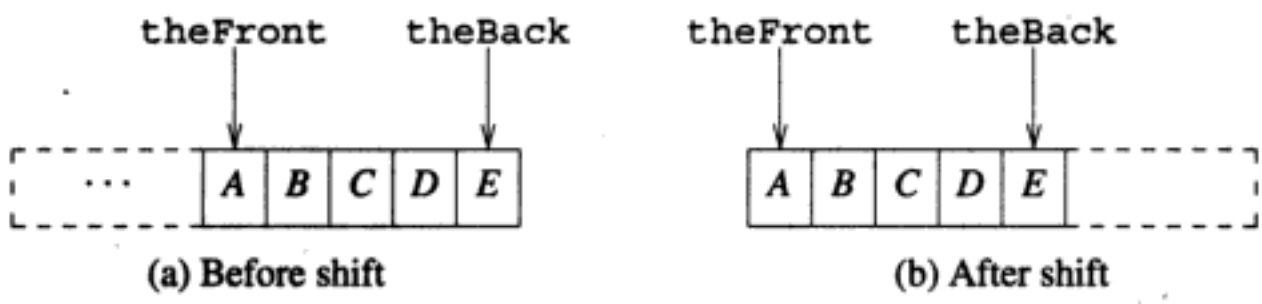


Figure 9.4 Shifting a queue

The worst-case push and pop times (assuming no array resizing is needed) become $\Theta(1)$ when we permit the queue to wrap around the end of the array. At this time it is convenient to think of the array positions as arranged in a circle (Figure 9.5) rather than in a straight line (Figure 9.4).

When the array is viewed as a circle, each array position has a next and a previous position. The position next to position `arrayLength` − 1 is 0, and the

Copyrighted material

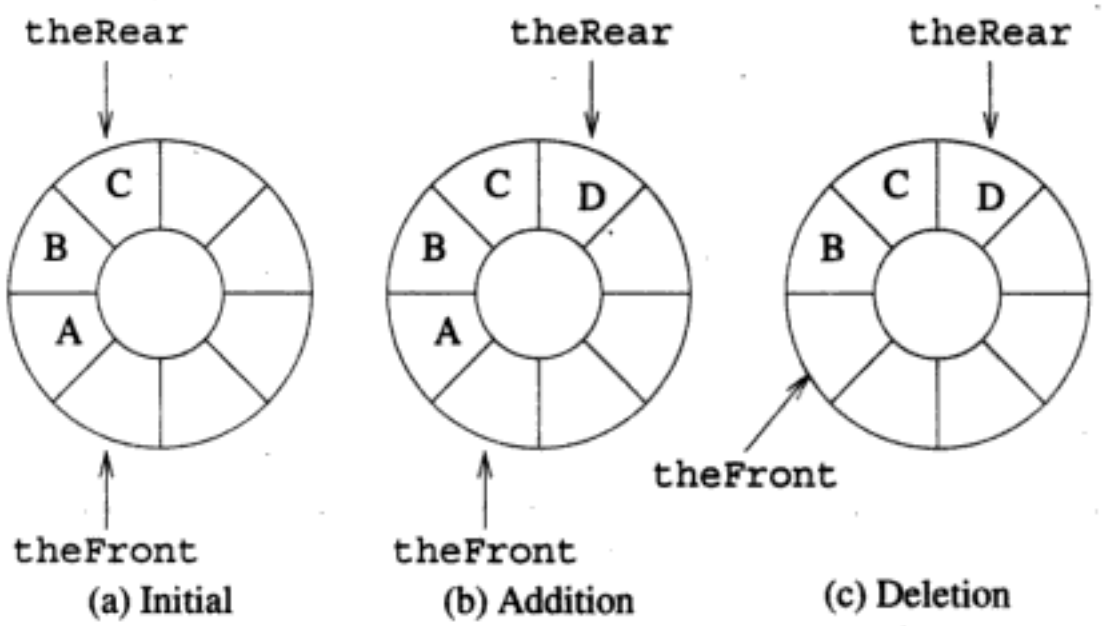


Figure 9.5 Circular queues

position that precedes 0 is `arrayLlength` − 1. When the back of the queue is at `arrayLength` − 1, the next element is put into position 0. The circular array representation of a queue uses the following mapping function:

$$location(i) = (location(\text{front element}) + i)\%\texttt{arrayLength} \tag{9.3}$$

In Figure 9.5 we have changed the convention for the variable `queueFront`. This variable now points one position counterclockwise from the location of the front element in the queue. The convention for `queueBack` is unchanged. This change simplifies the codes.

Pushing an element into the queue of Figure 9.5(a) results in the queue of Figure 9.5(b). Popping an element from the queue of Figure 9.5(b) results in the queue of Figure 9.5(c).

A queue is empty iff `queueFront` = `queueBack`. The initial condition `queueFront` = `queueBack` = 0 defines an initially empty queue. If we push elements into the queue of Figure 9.5(b) until the number of elements in the array `queue` equals `arrayLength` (i.e., the queue becomes full), we obtain the configuration of Figure 9.6. This configuration has `queueFront` = `queueBack`, which is the same condition as when the queue is empty! Therefore, we cannot distinguish between an empty and a full queue. To avoid this difficulty, we will not permit a queue to get full. Before pushing an element into a queue, we verify whether this push will cause the queue to get full. If so, we double the length of the array `queue` and

Copyrighted material

then proceed with the push. Using this strategy, the array **queue** can have at most **arrayLength-1** elements in it.

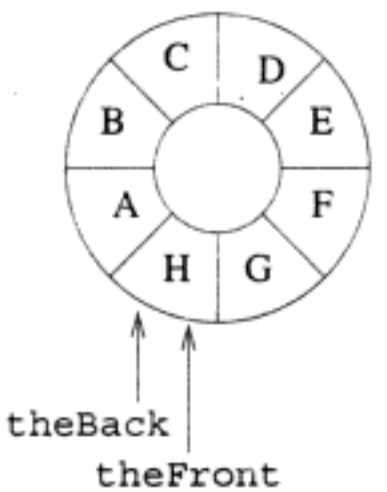


Figure 9.6 A circular queue with **arrayLength** elements

9.3.2 The Class arrayQueue

The class **arrayQueue** uses Equation 9.3 to map a queue into a one-dimensional array **queue**. The data members of **arrayQueue** are **queueFront**, **queueBack**, and **queue**; the codes for all methods other than **push** and **pop** are similar to those for the corresponding methods of **arrayStack**. These similar codes are available from the Web site. The method **push** (Program 9.2) uses customized code (Program 9.3) to double the array length.

To visualize array doubling when a circular queue is used, it is better to flatten out the array as in Figure 9.7(a). This figure shows a queue with seven elements in an array whose length is 8. Figure 9.7(b) shows the the flattened view of the same queue. Figure 9.7(c) shows the array after array doubling by **changeLength1D** (Program 5.2).

To get a proper circular queue configuration, we must slide the elements in the right segment (i.e., elements A and B) to the right end of the array as in Figure 9.7(d). The array doubling copies **arrayLength** (this is the capacity of the array **queue** prior to array doubling) elements, and when the second segment is slid right, up to **arrayLength-2** additional elements are copied. The number of elements copied can be limited to **arrayLength-1** by customizing the array doubling code. Figure 9.7(e) shows an alternative configuration for the array after doubling. This configuration may be obtained as follows:

- Create a new array **newQueue** of twice the length.

Copyrighted material

```
template<class T>
void arrayQueue<T>::push(const T& theElement)
{// Add theElement to queue.

   // increase array length if necessary
   if ((queueBack + 1) % arrayLength == queueFront)
   {// double array length
      // code to double array size comes here
   }

   // put theElement at the queueBack of the queue
   queueBack = (queueBack + 1) % arrayLength;
   queue[queueBack] = theElement;
}
```

Program 9.2 Pushing an element into a queue

- Copy the second segment (i.e., the elements `queue[queueFront+1]` through `queue[arrayLength - 1]`) to positions in `newQueue` beginning at 0.
- Copy the first segment (i.e., the elements `queue[0]` through `queue[queueBack]`) to positions in `newQueue` beginning at `arrayLength-queueFront-1`.

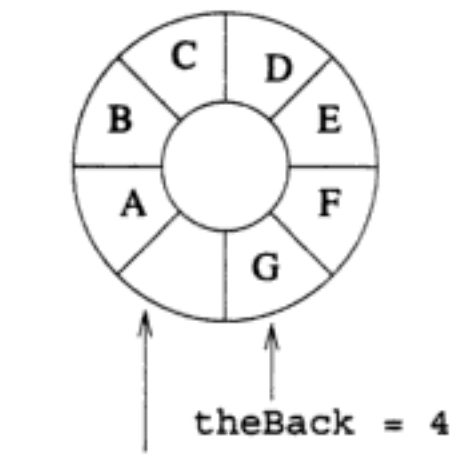
The code of Program 9.3 obtains the configuration of Figure 9.7(e). The segment copying is done using the STL function `copy`. Program 9.4 gives the code for the `pop` method.

The complexity of the queue constructor is $O(1)$ when the `T` is a primitive datatype and is $O($`initialCapacity`$)$ when `T` is a user-defined type. The complexity of `empty`, `size`, `front`, `back`, and `pop` is $\Theta(1)$; and the complexity of `push` is $\Theta(1)$ when no array doubling is done and is Θ(queue size) when array doubling is done. From the analysis used to establish Theorem 5.1, it follows that the complexity of m invocations of `push` is $O(m)$.

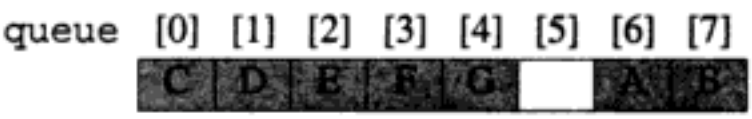
EXERCISES

5. (a) Extend the queue ADT by adding functions to
   i. Input a queue.
   ii. Output a queue.
   iii. Split a queue into two queues. The first of the resulting queues contains the first, third, fifth, $\cdots$ elements of the original queue; the second contains the remaining elements.

Copyrighted material

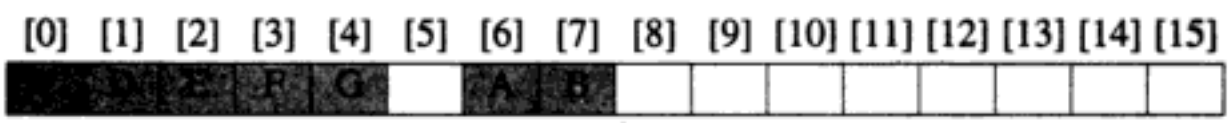


(a) A full circular queue



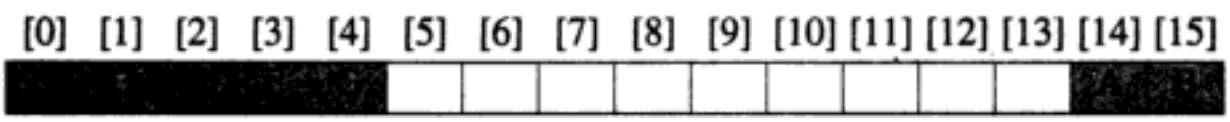
theFront = 5, theBack = 4

(b) flattened view of circular full queue



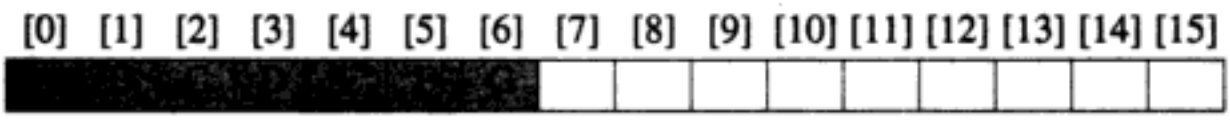
theFront = 5, theBack = 4

(c) after array doubling



theFront = 13, theBack = 4

(d) after shifting right segment



theFront = 15, theBack = 6

(e) alternative configuration

Figure 9.7 Doubling array length

iv. Combine two queues by selecting elements alternately from the two queues beginning with queue 1. When a queue exhausts, append the remaining elements from the other queue to the combined queue. The relative order of elements from each queue is unchanged.

Copyrighted material

```
// allocate a new array
T* newQueue = new T[2 * arrayLength];

// copy elements into new array
int start = (theFront + 1) % arrayLength;
if (start < 2)
   // no wrap around
   copy(queue + start, queue + start + arrayLength - 1, newQueue);
else
{  // queue wraps around
   copy(queue + start, queue + arrayLength, newQueue);
   copy(queue, queue + theBack + 1, newQueue + arrayLength - start);
}

// switch to newQueue and set theFront and theBack
theFront = 2 * arrayLength - 1;
theBack = arrayLength - 2;   // queue size arrayLength - 1
arrayLength *= 2;
queue = newQueue;
```

Program 9.3 Doubling the length of the array queue

```
void pop()
     {// remove queueFront element
        if (queueFront == queueBack)
           throw queueEmpty();
        queueFront = (queueFront + 1) % arrayLength;
        queue[queueFront].~T();  // destructor for T
     }
```

Program 9.4 Popping an element from a queue

(b) Define the abstract class **extendedQueue** that derives from the abstract class **queue** and includes methods that correspond to the functions of (a).

(c) Develop code for the concrete class **extendedarrayQueue** that derives from the classes **arrayQueue** and **extendedQueue**.

(d) Test your code.

6. Develop the concrete class **slowArrayQueue** that derives from **queue** and use

Copyrighted material

the mapping of Equation 9.2 Test your code and compare its performance with that of `arrayQueue`.

7. Modify the representation used in the class `arrayQueue` so that a queue can hold as many elements as the length of the array `queue`. For this modification replace the variable `queueBack` with the variable `queueSize`, which equals the size of the queue. Use the convention that the front element is at `queue[queueFront]`. Note that the location of the back element may be computed from `queueSize` and `queueFront`. Test the correctness of your modified code.

8. Modify the representation used in the class `arrayQueue` so that a queue can hold as many elements as the length of the array `queue`. For this modification introduce another data member `lastOp` that keeps track of the last operation (from among the operations `push` and `pop`) performed on the queue. Notice that if the last operation performed was `push`, the queue cannot be empty. Also, if the last operation was `pop`, the queue cannot be full. So `lastOp` can be used to distinguish between an empty and full queue when `queueFront = queueBack`. Test the correctness of your modified code.

9. A **deque** (pronounced *deck*) is an ordered list to/from which we can make pushes and pops at/from either end. Therefore, we can call it a double-ended queue.

   (a) Define the ADT *deque*. Include the operations *empty*, *size*, *front*, *back*, *push_front*, *push_back*, *pop_front*, and *pop_back*.

   (b) Define an abstract C++ class `deque` that includes methods for each function of the ADT *deque*.

   (c) Use Equation 9.3 to represent a deque. Develop a concrete C++ class `arrayDeque` that derives from `deque`. Note that the C++ STL has a concrete class `deque` that is an array implementation of the data structure *deque*.

   (d) Test your code using suitable test data.

10. (a) Develop the concrete class `dequeStack` that derives from `stack` (Program 8.1) and `arrayDeque` (see Exercise 9).

    (b) What is the time complexity of each method of `dequeStack`?

    (c) Comment on the expected performance of the methods of `dequeStack` relative to their counterparts in `arrayStack`.

11. (a) Develop the concrete class `dequeQueue` that derives from `queue` (Program 9.1) and `arrayDeque` (see Exercise 9).

    (b) What is the time complexity of each method of `dequeQueue`?

    (c) Comment on the expected performance of the methods of `dequeQueue` relative to their counterparts in `arrayQueue`.

Copyrighted material

9.4 LINKED REPRESENTATION

A queue, like a stack, can be represented as a chain. We need two variables, `queueFront` and `queueBack`, to keep track of the two ends of a queue. There are two possibilities for binding these two variables to the two ends of a chain. The nodes can be linked from front to back (Figure 9.8(a)) or from back to front (Figure 9.8(b)). The relative difficulty of performing pushes and pops determines the direction of linkage. Figures 9.9 and 9.10, respectively, illustrate the mechanics of a push and a pop. We can see that both linkage directions are well suited for pushes, but the front-to-back linkage is more efficient for pops. Hence we will link the nodes in a queue from front to back.

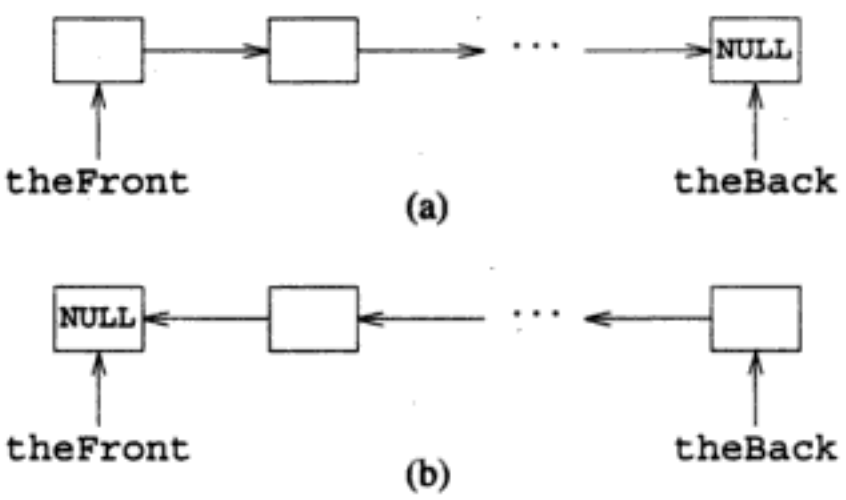


Figure 9.8 Linked queues

We can use the initial values `queueFront` = `queueBack` = `NULL` and the boundary value `queueFront` = `NULL` iff the queue is empty. The class `linkedQueue` may be defined as a derived class of `extendedChain` (Program 6.12). Exercise 12 considers this way of developing `linkedQueue`. In this section we develop the class `linkedQueue` from scratch.

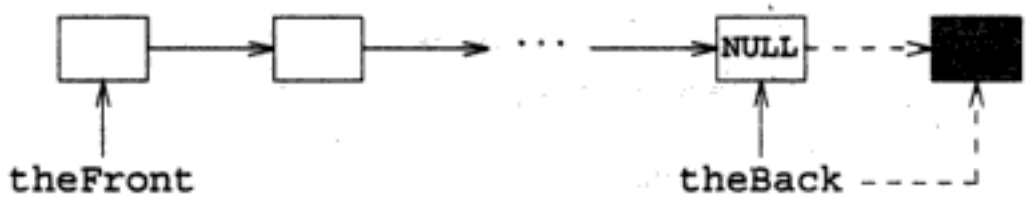
Program 9.5 gives the `push` and `pop` methods of `linkedQueue`. You should run through these codes by hand using an empty queue, a queue with one element, and a queue with many elements as examples. The complexity of each of the linked queue methods is $\Theta(1)$.

EXERCISES

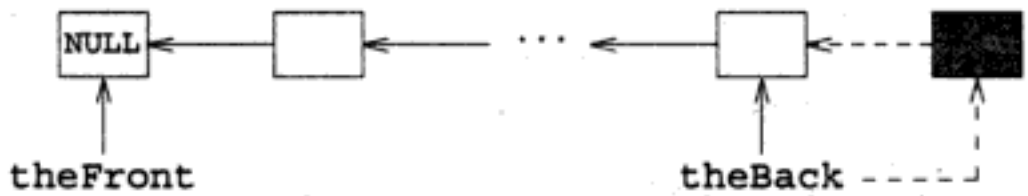
12. Develop the class `linkedQueueFromExtendedChain`, which implements a linked queue by deriving from `extendedChain` (Section 6.1.5) and `queue`.

13. Do Exercise 5 using a linked queue.

Copyrighted material

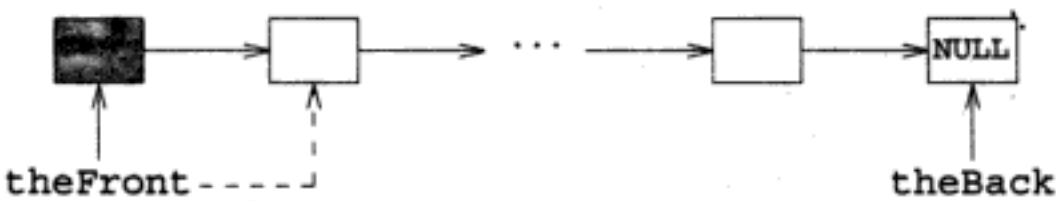


(a) Pushing into Figure 9.8(a)

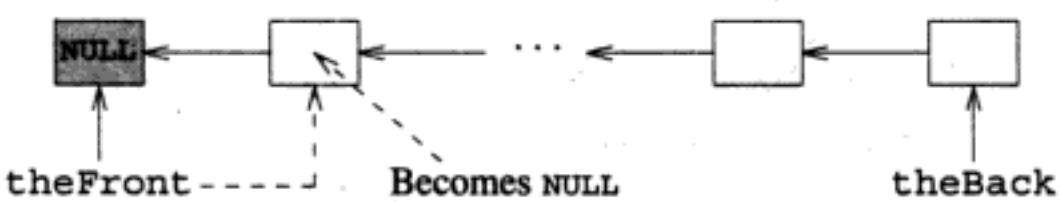


(b) Pushing into Figure 9.8(b)

Figure 9.9 Pushing an element into a linked queue



(a) Popping from Figure 9.8(a)



(b) Popping from Figure 9.8(b)

Figure 9.10 Popping an element from a linked queue

14. Compare the performance of **arrayQueue** and **linkedQueue** by performing a sequence of 1,000,000 **push** operations followed by 1,000,000 **pop** operations.

15. In some queue applications the elements to be put on a queue are already in nodes of type **chainNode**. For these applications it is desirable to have the methods **pushNode(chainNode theNode)**, which adds **theNode** at the back of the queue (notice that no call to **new** is made), and **popNode**, which removes and returns the front node of the queue.

    (a) Write code for these methods.

Copyrighted material

```
template<class T>
void linkedQueue<T>::push(const T& theElement)
{// Add theElement to back of queue.

   // create node for new element
   chainNode<T>* newNode = new chainNode<T>(theElement, NULL);

   // add new node to back of queue
   if (queueSize == 0)
      queueFront = newNode;        // queue empty
   else
      queueBack->next = newNode;  // queue not empty
   queueBack = newNode;

   queueSize++;
}

template<class T>
void linkedQueue<T>::pop()
{// Delete front element.
   if (queueFront == NULL)
      throw queueEmpty();

   chainNode<T>* nextNode = queueFront->next;
   delete queueFront;
   queueFront = nextNode;
   queueSize--;
}
```

Program 9.5 The `push` and `pop` methods of `linkedQueue`

(b) Test your code.

(c) Compare the time required by a sequence of 1,000,000 `push` operations followed by 1,000,000 `pop` operations with that required by 1,000,000 `pushNodes` followed by 1,000,000 `popNodes`.

16. See Exercise 9 for the definition of a deque.

(a) Develop a concrete C++ class `doublyLinkedDeque` that derives from the abstract class `deque` of Exercise 9 and uses a doubly linked list. Your class should not derive from any other class.

(b) What is the complexity of each method of your class?

Copyrighted material

(c) Test your code using suitable test data.

17. Do Exercise 16 using a singly linked list (i.e., a chain) rather than a doubly linked list. Name your class `linkedDeque`.

18. Do Exercise 16 using a singly linked circular list rather than a doubly linked list. Name your class `circularDeque`.

9.5 APPLICATIONS

9.5.1 Railroad Car Rearrangement

Problem Description and Solution Strategy

We will reconsider the railroad car rearrangement problem of Section 8.5.3. This time the holding tracks lie between the input and output track as in Figure 9.11. These tracks operate in a FIFO manner and so may be regarded as queues. As in the case of Section 8.5.3, moving a car from a holding track to the input track or from the output track to a holding track is forbidden. All car motion is in the direction indicated by the arrowheads of Figure 9.11.

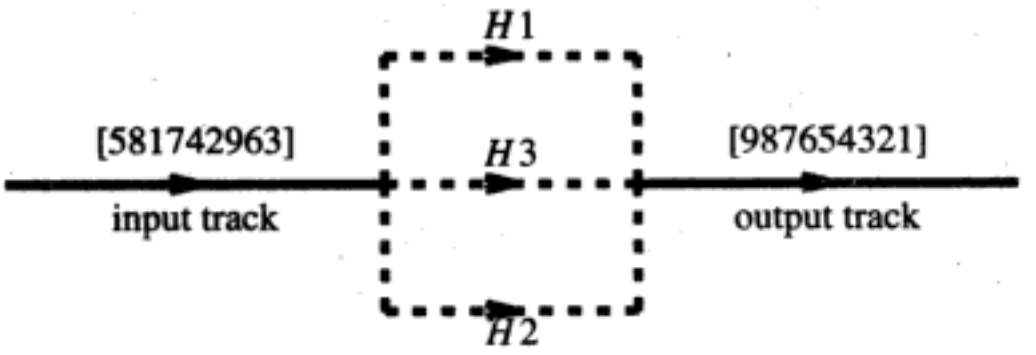


Figure 9.11 A three-track example

We reserve track Hk for moving cars directly from the input track to the output track. So only $k-1$ tracks are available to hold cars that are not ready to be output.

Consider rearranging nine cars that have the initial ordering 5, 8, 1, 7, 4, 2, 9, 6, 3. Assume that $k = 3$ (Figure 9.11). Car 3 cannot be moved directly to the output track, as cars 1 and 2 must come before it. So car 3 is moved to $H1$. Car 6 can be placed behind car 3 in $H1$, as car 6 is to be output after car 3. Car 9 can now be placed after car 6 in $H1$. Car 2 cannot be placed after car 9, as car 2 is to be output before car 9. So it is placed at the front of $H2$. Car 4 can now be placed after car 2 in $H2$, and car 7 can be placed after it. Car 1 can be moved to the output using $H3$. Next car 2 is moved from $H2$ to the output. Then car 3 is moved from $H1$ to the output, and car 4 is moved from $H2$ to the output. Car 5 is to be output next.

Copyrighted material

It is still in the input track. So car 8 is moved from the input track to $H2$. Then car 5 is moved to the output track. Now cars 6, 7, 8, and 9 are moved from their holding tracks to the output track.

When a car is to be moved to a holding track, we can use the following track selection method. *Move car c to a holding track that contains only cars with a smaller label; if several such tracks exist, select one with the largest label at its left end; otherwise, select an empty track (if one remains).*

First Implementation

We can implement the car rearrangement algorithm by using queues for the $k-1$ holding tracks that can hold cars. We can model the code after Programs 8.9 through 8.12. Program 9.6 gives the new code for `outputFromHoldingTrack`, and Program 9.7 gives the new code for `putInHoldingTrack`. The method `railroad` of Program 8.10 needs to be modified. The changes are (1) decrease the number of tracks by 1 and (2) change the type of `track` to `arrayQueue`. The time needed to perform the rearrangement is $O(\texttt{numberOfCars} * \texttt{k})$. We can use AVL trees (see Chapter 15) to reduce this time to $O(\texttt{numberOfCars} * \log \texttt{k})$.

```
void outputFromHoldingTrack()
{// output the smallest car from the holding tracks
   // pop smallestCar from itsTrack
   track[itsTrack].pop();
   cout << "Move car " << smallestCar << " from holding track "
        << itsTrack << " to output track" << endl;

   // find new smallestCar and itsTrack by checking all queue fronts
   smallestCar = numberOfCars + 2;
   for (int i = 1; i <= numberOfTracks; i++)
      if (!track[i].empty() && track[i].front() < smallestCar)
      {
         smallestCar = track[i].front();
         itsTrack = i;
      }
}
```

Program 9.6 Function to output a railroad car

Second Implementation

To animate the progress of the rearrangement algorithm, it is useful to keep the queues used in our first implementation. In an animation we wish to move all cars

Copyrighted material

```
bool putInHoldingTrack(int c)
{// Put car c into a holding track.
 // Return false iff there is no feasible holding track for this car.

   // find best holding track for car c
   // initialize
   int bestTrack = 0,  // best track so far
       bestLast =  0;  // last car in bestTrack

   // scan tracks
   for (int i = 1; i <= numberOfTracks; i++)
      if (!track[i].empty())
      {// track i not empty
          int lastCar = track[i].back();
          if (c > lastCar && lastCar > bestLast)
          {
             // track i has bigger car at its rear
             bestLast = lastCar;
             bestTrack = i;
          }
      }
      else // track i empty
         if (bestTrack == 0)
            bestTrack = i;

   if (bestTrack == 0)
      return false; // no feasible track

   // add c to bestTrack
   track[bestTrack].push(c);
   cout << "Move car " << c << " from input track "
        << "to holding track " << bestTrack << endl;

   // update smallestCar and itsTrack if needed
   if (c < smallestCar)
   {
       smallestCar = c;
       itsTrack = bestTrack;
   }

   return true;
}
```

Program 9.7 Function to put a railroad car into a holding track

Copyrighted material

11.1 TREES

So far in this text we have seen data structures for linear and tabular data. These data structures are generally not suitable for the representation of hierarchical data. In hierarchical data we have an ancestor-descendant, superior-subordinate, whole-part, or similar relationship among the data elements.

Example 11.1 [Joe's Descendants] Figure 11.1 shows Joe and his descendants arranged in a hierarchical manner, beginning with Joe at the top of the hierarchy. Joe's children (Ann, Mary, and John) are listed next in the hierarchy, and a line or edge joins Joe and his children. Ann has no children, while Mary has two and John has one. Mary's children are listed below her, and John's child is listed below him. There is an edge between each parent and his/her children. From this hierarchical representation, it is easy to identify Ann's siblings, Joe's descendants, Chris's ancestors, and so on. ■

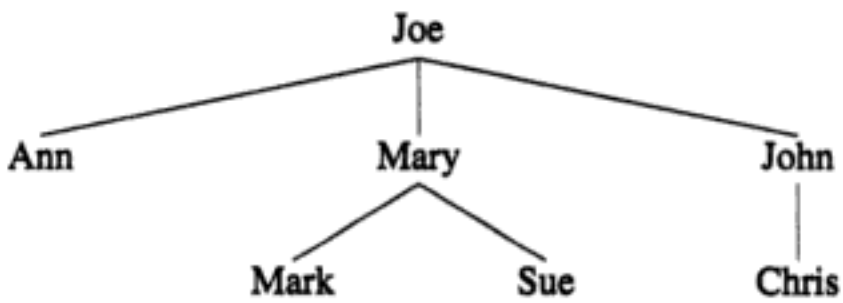


Figure 11.1 Descendants of Joe

Example 11.2 [Corporate Structure] As an example of hierarchical data, consider the administrative structure of the corporation of Figure 11.2. The person (in this case the president) highest in the hierarchy appears at the top of the diagram. Those who are next in the hierarchy (i.e., the vice presidents) are shown below the president and so on. The vice presidents are the president's subordinates, and the president is their superior. Each vice president, in turn, has his/her subordinates who may themselves have subordinates. In the diagram we have drawn a line or edge between each person and his/her direct subordinates or superior. ■

Copyrighted material

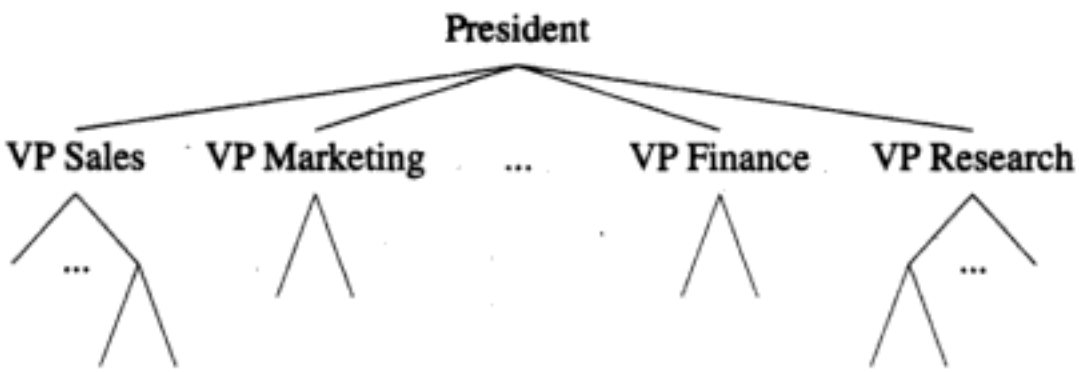


Figure 11.2 Hierarchical administrative structure of a corporation

Example 11.3 [Governmental Subdivisions] Figure 11.3 is a hierarchical drawing of the branches of the federal government. At the top of the hierarchy, we have the entire federal government. At the next level of the hierarchy, we have drawn its major subdivisions (i.e., the different departments). Each department may be further subdivided. These subdivisions are drawn at the next level of the hierarchy. For example, the Department of Defense has been subdivided into the Army, Navy, Air Force, and Marines. A line runs between each element and its components. The data of Figure 11.3 are an example of whole-part relationships. ■

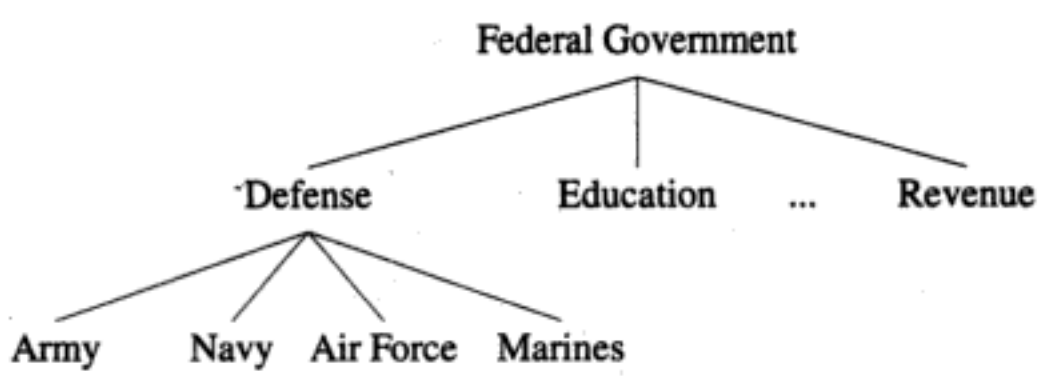


Figure 11.3 Modules of the federal government

Example 11.4 [Software Engineering] For another example of hierarchical data, consider the software-engineering technique referred to as modularization. In modularization we decompose a large and complex task into a collection of smaller, less complex tasks. The objective is to divide the software system into many functionally independent parts or **modules** so that each can be developed relatively independently. This decision reduces the overall software development time, as it is much easier to solve several small problems than one large one. Additionally, different

Copyrighted material

programmers can develop different modules at the same time. If necessary, each module may be further decomposed so as to obtain a hierarchy of modules as shown by the tree of Figure 11.4. This tree represents a possible modular decomposition of a text processor.

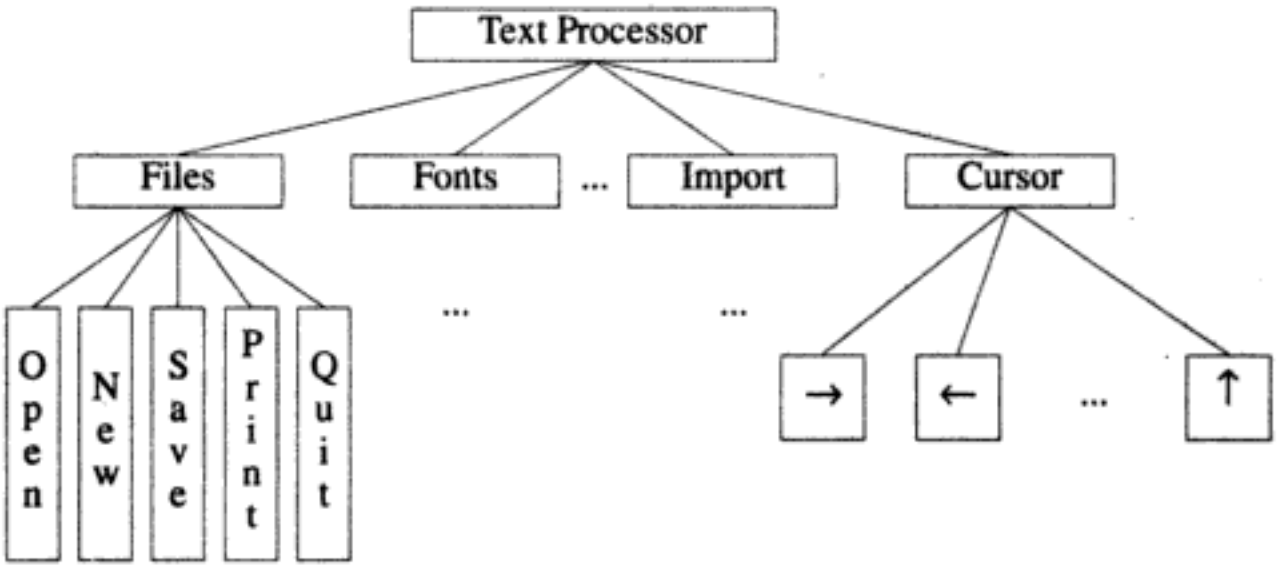


Figure 11.4 Module hierarchy for text processor

At the top level the text processor has been split into several modules. Only four are shown in the figure. The Files module performs functions related to text files such as opening an existing file, opening a new file, saving a file, printing a file, and exiting from the text processor (exiting requires saving files if the user so desires). Each function is represented by a module at the next level of the hierarchy. The Fonts module handles all functions related to the font in use. These functions include changing the font, its size, color, and so on. If modules for these functions were shown in the figure, they would appear below the module Fonts. The Import module handles functions associated with the import of material such as graphics, tables, and text in a format not native to this text processor. The Cursor module handles the movement of the cursor on the screen. Its subordinate modules correspond to various cursor motions. Programmers can carry out the specification, design, and development of each module in a relatively independent manner after the interfaces are fully specified.

When the software system is specified and designed in a modular fashion, it is natural to develop the system itself in this way. The resulting software system will have as many modules as there are nodes in the module hierarchy. Modularization improves the intellectual manageability of a problem. By systematically dividing a large problem into smaller relatively independent problems, we can solve the large problem with much less effort. The independent problems can be assigned to

Copyrighted material

Hidden page

of the department subtree. The remaining employees of a department could be partitioned into projects and so on.

The vice presidents are children of the president; department heads are children of their vice president, and so on. The president is the parent of the vice presidents, and each vice president is the parent of the department heads in his/her division.

In Figure 11.3 the root is the element Federal Government. Its subtrees have the roots Defense, Education, $\cdots$, and Revenue, which are the children of Federal Government. Federal Government is the parent of its children. Defense has the children Army, Navy, Air Force, and Marines. The children of Defense are siblings and are also leaves. ■

Another commonly used tree term is **level**. By definition the tree root is at level 1; its children (if any) are at level 2; their children (if any) are at level 3; and so on.[1] In the tree of Figure 11.3, Federal Government is at level 1; Defense, Education, and Revenue are at level 2; and Army, Navy, Air Force, and Marines are at level 3.

The **height** (or **depth**) of a tree is the number of levels in it. The trees of Figures 11.1, 11.3, and 11.4 have a height of 3.

The **degree of an element** is the number of children it has. The degree of a leaf is 0. The degree of Files in Figure 11.4 is 5. The **degree of a tree** is the maximum of its element degrees.

EXERCISES

1. Explain why the diagram of Figure 11.1 is a tree. Label the root node and mark each node with its level number and degree. What is the depth of this tree?

2. Do Exercise 1 for the recursion diagram of Figure 1.3.

3. Develop a tree representation for the major elements (whole book, chapters, sections, and subsections) of this text.

   (a) What is the total number of elements in your tree?

   (b) Identify the leaf elements.

   (c) Identify the elements on level 3.

   (d) List the degree of each element.

4. Access the World Wide Web home page for your department. (Alternatively, access `http://www.cise.ufl.edu`.)

   (a) Follow some of the links to lower-level pages and draw the resulting structure. In your drawing the Web pages should be represented by nodes, and the links by edges that join pairs of nodes.

[1] Some authors number tree levels beginning at 0 rather than 1. In this case the root of a tree is at level 0.

Copyrighted material

Hidden page

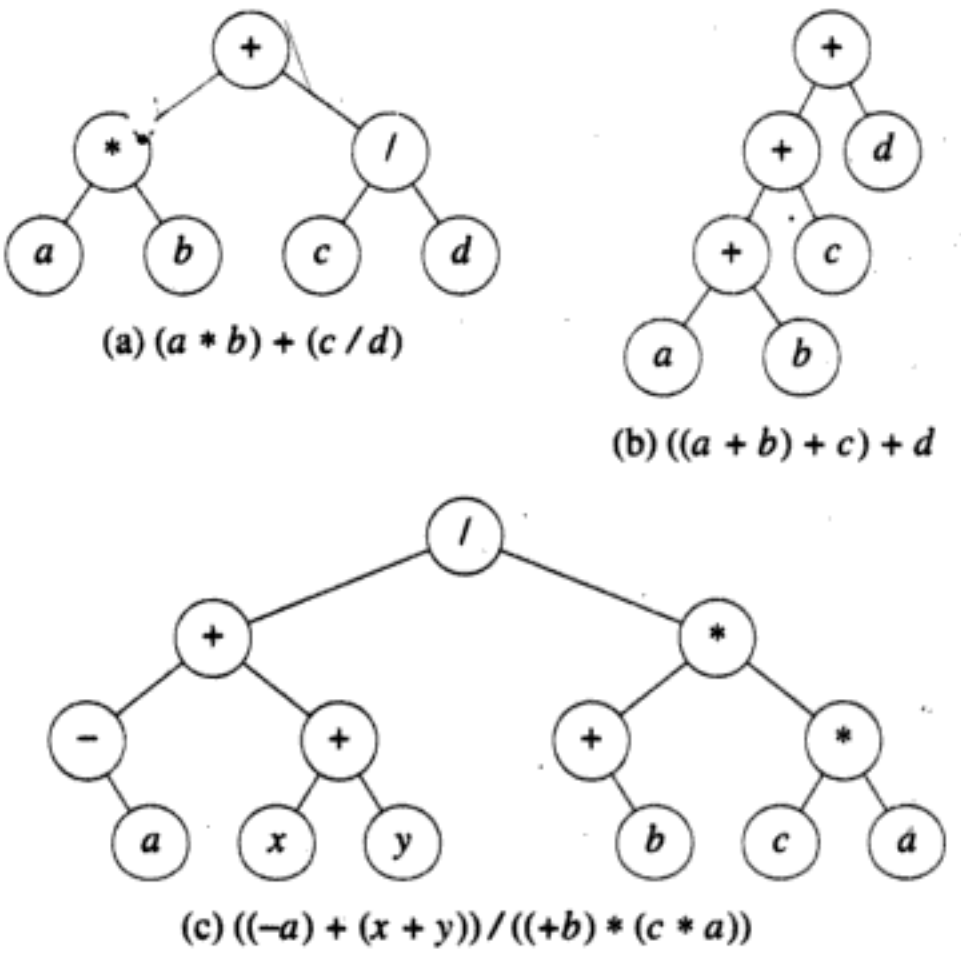


Figure 11.5 Expression trees

6. Draw the binary expression trees corresponding to each of the following expressions:

   (a) $(a+b)/(c-d)+e+g*h/a$

   (b) $-x-y*z+(a+b+c/d*e)$

   (c) $((a+b)>(c-e))||a<b\&\&(x<y||y>z)$

11.3 PROPERTIES OF BINARY TREES

Property 11.1 *The drawing of every binary tree with n elements, $n > 0$, has exactly $n-1$ edges.*

Proof Every element in a binary tree (except the root) has exactly one parent. There is exactly one edge between each child and its parent. So the number of edges is $n-1$. ■

Copyrighted material

Property 11.2 *A binary tree of height h, $h \geq 0$, has at least h and at most $2^h - 1$ elements in it.*

Proof Since each level has at least one element, the number of elements is at least h. As each element can have at most two children, the number of elements at level i is at most 2^{i-1}, $i > 0$. For $h = 0$, the total number of elements is 0, which equals $2^0 - 1$. For $h > 0$, the number of elements cannot exceed $\sum_{i=1}^{h} 2^{i-1} = 2^h - 1$. ■

Property 11.3 *The height of a binary tree that contains n, $n \geq 0$, elements is at most n and at least $\lceil \log_2(n+1) \rceil$.*

Proof Since there must be at least one element at each level, the height cannot exceed n. From Property 11.2 we know that a binary tree of height h can have no more than $2^h - 1$ elements. So $n \leq 2^h - 1$. Hence $h \geq \log_2(n+1)$. Since h is an integer, we get $h \geq \lceil \log_2(n+1) \rceil$. ■

A binary tree of height h that contains exactly $2^h - 1$ elements is called a **full binary tree**. The binary tree of Figure 11.5(a) is a full binary tree of height 3. The binary trees of Figures 11.5(b) and (c) are not full binary trees. Figure 11.6 shows a full binary tree of height 4.

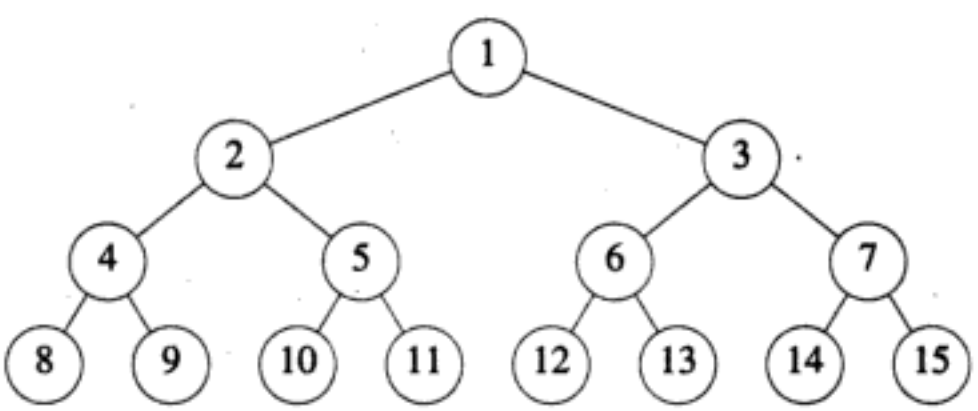


Figure 11.6 Full binary tree of height 4

Suppose we number the elements in a full binary tree of height h using the numbers 1 through $2^h - 1$. We begin at level 1 and go down to level h. Within levels the elements are numbered left to right. The elements of the full binary tree of Figure 11.6 have been numbered in this way. Now suppose we delete the k elements numbered $2^h - i$, $1 \leq i \leq k < 2^h$. The resulting binary tree is called a **complete binary tree**. Figure 11.7 gives some examples. Note that a full binary tree is a special case of a complete binary tree. Also, note that the height of a complete binary tree that contains n elements is $\lceil \log_2(n+1) \rceil$.

There is a very nice relationship among the numbers assigned to an element and its children in a complete binary tree, as given by Property 11.4.

Copyrighted material

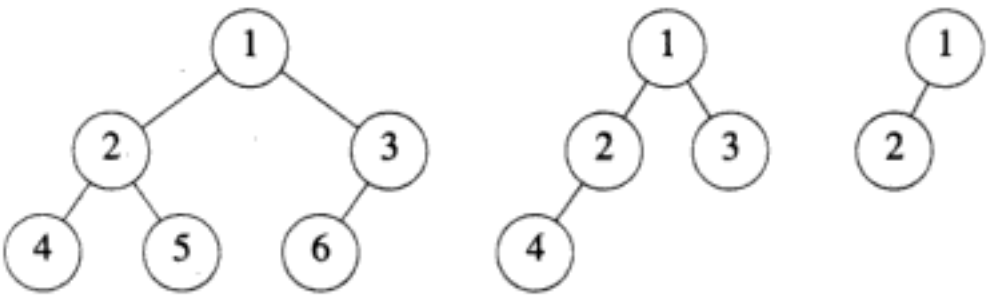


Figure 11.7 Complete binary trees

Property 11.4 *Let i, $1 \leq i \leq n$, be the number assigned to an element of a complete binary tree. The following are true:*

1. *If $i = 1$, then this element is the root of the binary tree. If $i > 1$, then the parent of this element has been assigned the number $\lfloor i/2 \rfloor$.*

2. *If $2i > n$, then this element has no left child. Otherwise, its left child has been assigned the number $2i$.*

3. *If $2i + 1 > n$, then this element has no right child. Otherwise, its right child has been assigned the number $2i + 1$.*

Proof Can be established by induction on i. ■

EXERCISES

7. Prove Property 11.4.

8. In a k-ary tree, $k > 1$, each node may have up to k children. These children are called, respectively, the first, second, $\cdots$, kth child of the node. A 2-ary tree is a binary tree.

   (a) Determine the analogue of Property 11.1 for k-ary trees.

   (b) Determine the analogue of Property 11.2 for k-ary trees.

   (c) Determine the analogue of Property 11.3 for k-ary trees.

   (d) Determine the analogue of Property 11.4 for k-ary trees.

9. What is the maximum number of nodes in a binary tree that has m leaves?

Copyrighted material

11.4 REPRESENTATION OF BINARY TREES

11.4.1 Array-Based Representation

The array representation of a binary tree utilizes Property 11.4. The binary tree to be represented is regarded as a complete binary tree with some missing elements. Figure 11.8 shows two sample binary trees. The first binary tree has three elements (A, B, and C), and the second has five elements (A, B, C, D, and E). Neither is complete. Unshaded circles represent missing elements. All elements (including the missing ones) are numbered as described in the previous section.

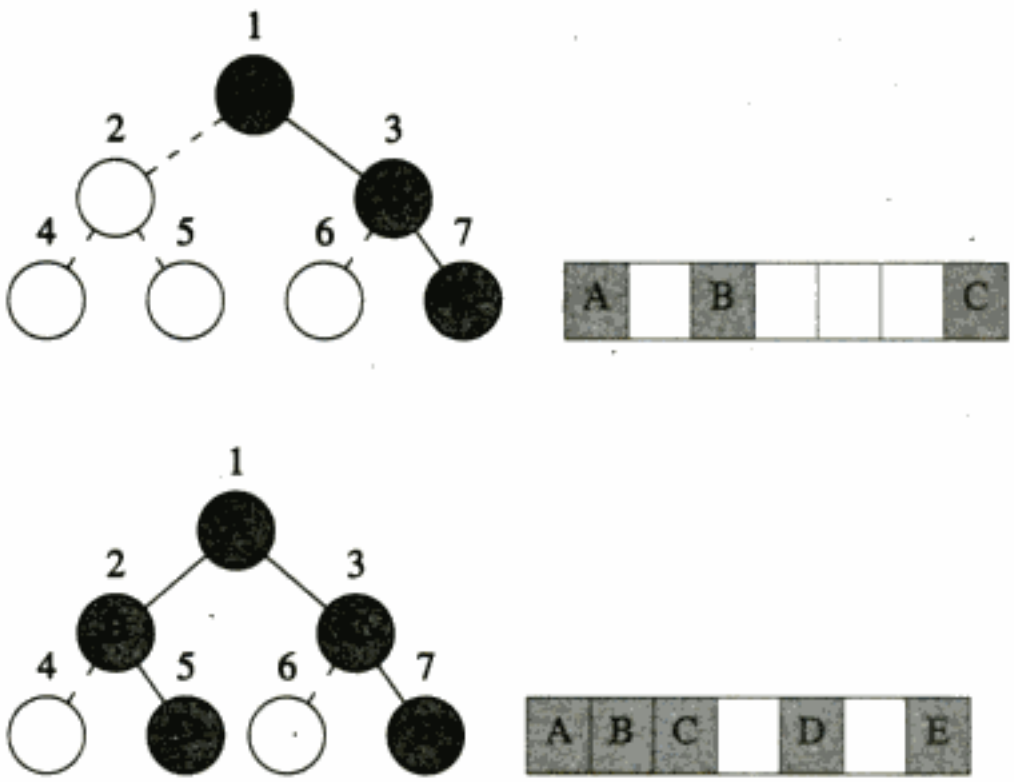
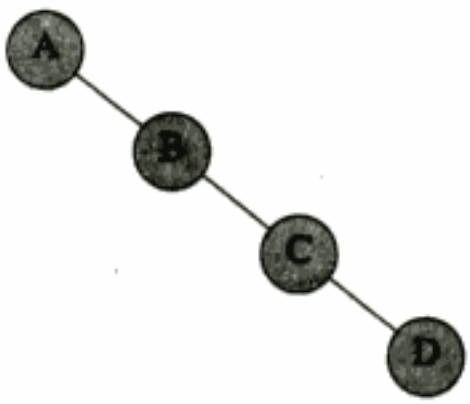


Figure 11.8 Incomplete binary trees

In an array representation, the binary tree is represented in an array by storing each element at the array position corresponding to the number assigned to it. Figure 11.8 also shows the array representations for its binary trees (position 0 of the array is not shown). Missing elements are represented by white boxes. As can be seen, this representation scheme is quite wasteful of space when many elements are missing. In fact, a binary tree that has n elements may require an array of size up to 2^n (including position 0) for its representation. This maximum size is needed when each element (except the root) of the n-element binary tree is the right child of its parent. Figure 11.9 shows such a binary tree with four elements. Binary trees of this type are called **right-skewed** binary trees. Note that the worst-case space

Copyrighted material

required may be reduced to $2^n - 1$ by numbering the binary tree nodes beginning at 0 rather than at 1.



(a) Right-skewed tree



(b) Array representation

Figure 11.9 Right-skewed binary tree

The array representation is useful only when the number of missing elements is small.

11.4.2 Linked Representation

The most popular way to represent a binary tree is by using links or pointers. A node that has exactly two pointer fields represents each element. Let us call these pointer fields `leftChild` and `rightChild`. In addition to these two pointer fields, each node has a field named `element`. This node structure is implemented by the C++ struct `binaryTreeNode` (Program 11.1). We have provided three constructors for a binary tree node. The first takes no parameters and sets the two children fields to `NULL`; the second takes one parameter and uses it to initialize `element`, and the child fields are set to `NULL`; the third takes three parameters and uses these to initialize all three fields of the node.

Each pointer from a parent node to a child node represents an edge in the drawing of a binary tree. Since an n-element binary tree has exactly $n-1$ edges, we are left with $2n - (n-1) = n+1$ pointer fields that have no value. These pointer fields are set to `NULL`. Figure 11.10 shows the linked representations of the binary trees of Figure 11.8.

We can access all nodes in a binary tree by starting at the root and following `leftChild` and `rightChild` pointers. The absence of a parent pointer from the

Copyrighted material

Hidden page

Hidden page

Hidden page

The first three traversal methods are best described recursively as in Programs 11.2, 11.3, and 11.4. These codes assume that the binary tree being traversed is represented with the linked scheme of the previous section.

In the first three traversal methods, the left subtree of a node is traversed before the right subtree. The difference among the three orders comes from the difference in the time at which a node is visited. In the case of a preorder traversal, each node is visited before its left and right subtrees are traversed. In an inorder traversal, the root of each subtree is visited after its left subtree has been traversed but before the traversal of its right subtree begins. In a postorder traversal, each root is visited after its left and right subtrees have been traversed.

Figure 11.11 shows the output generated by Programs 11.2—11.4 when `visit(t)` is as shown in Program 11.5. The input binary trees are those of Figure 11.5.

```
template <class T>
void visit(binaryTreeNode<T> *x)
{// visit node *x, just output element field.
   cout << x->element;
}
```

Program 11.5 A `visit` function

Preorder	$+*ab/cd$	$+++abcd$	$/+-a+xy*+b*ca$
Inorder	$a*b+c/d$	$a+b+c+d$	$-a+x+y/+b*c*a$
Postorder	$ab*cd/+$	$ab+c+d+$	$a-xy++b+ca**/$
	(a)	(b)	(c)

Figure 11.11 Elements of a binary tree listed in pre-, in-, and postorder

When an expression tree is output in pre-, in-, or postorder, we get the prefix, infix, and postfix forms of the expression, respectively. The **infix** form of an expression is the form in which we normally write an expression. In this form each binary operator (i.e., operator with two operands) appears just after the infix form of its left operand and just before the infix form of its right operand. An expression presented as a binary tree is unambiguous in the sense that the association between operators and operands is uniquely determined by the representation. This association is not uniquely determined by the representation when the infix form is used. For example, is $x + y * z$ to be interpreted as $(x + y) * z$ or $x + (y * z)$? To resolve this ambiguity, one assigns priorities to operators and employs priority rules. Further, delimiters such as parentheses are used to override these rules if necessary. In a *fully parenthesized* infix representation, each operator and its operands are en-

Copyrighted material

closed in a pair of parentheses. Furthermore, each of the operands of the operator are in fully parenthesized form. Some representations of this type are $((x)+(y))$, $((x)+((y)*(z)))$, and $((((x)+(y))*((y)+(z)))*(w))$. This form of the expression is obtained by modifying inorder traversal as in Program 11.6.

```
template <class T>
void infix(binaryTreeNode<T> *t)
{// Output infix form of expression.
   if (t != NULL)
   {
      cout << '(';
      infix(t->leftChild);   // left operand
      cout << t->element;    // operator
      infix(t->rightChild); // right operand
      cout << ')';
   }
}
```

Program 11.6 Output fully parenthesized infix form

In the **postfix form** each operator comes immediately after the postfix form of its operands. The operands themselves appear in left-to-right order. In the **prefix form** each operator comes immediately before the prefix form of its operands. The operands themselves appear in left-to-right order. Like the binary tree representation, the prefix and postfix representations are unambiguous. As a result, neither the prefix nor the postfix representation employs parentheses or operator priorities. The association between operators and operands is easily determined by scanning the expression from right to left or from left to right and by employing a stack of operands. If an operand is encountered during this scan, it is stacked. If an operator is encountered, it is associated with the correct number of operands from the top of the stack. These operands are deleted from the stack and replaced by an operand that represents the result produced by the operator.

In a level-order traversal, elements are visited by level from top to bottom. Within levels, elements are visited from left to right. It is quite difficult to write a recursive function for level-order traversal, as the correct data structure to use here is a queue and not a stack. Program 11.7 traverses a binary tree in level order.

Program 11.7 enters the `while` loop only if the tree is not empty. The root is visited, and its children, if any, are added to the queue. Following the addition of the children of `t` to the queue, we attempt to access the front element of the queue. When the queue is empty, `front()` throws an exception of type `queueEmpty`; and when the queue is not empty, `front` returns a pointer to the front element. This element is the next node that is to be visited.

Copyrighted material

```
template <class T>
void levelOrder(binaryTreeNode<T> *t)
{// Level-order traversal of *t.
   arrayQueue<binaryTreeNode<T>*> q;
   while (t != NULL)
   {
      visit(t);  // visit t

      // put t's children on queue
      if (t->leftChild != NULL)
         q.push(t->leftChild);
      if (t->rightChild != NULL)
         q.push(t->rightChild);

      // get next node to visit
      try {t = q.front();}
      catch (queueEmpty) {return;}
      q.pop();
   }
}
```

Program 11.7 Level-order traversal

Let n be the number of nodes in a binary tree. The space and time complexity of each of the four traversal programs is $O(n)$. To verify this claim, observe that the recursion stack space needed by pre-, in-, and postorder traversal is $\Theta(n)$ when the tree height is n (as is the case for a right-skewed binary tree (Figure 11.9)); the queue space needed by level-order traversal is $\Theta(n)$ when the tree is a full binary tree. For the time complexity observe that each of the traversal methods spends $\Theta(1)$ time at each node of the tree (assuming the time needed to visit a node is $\Theta(1)$).

EXERCISES

12. List the nodes of the binary trees of Figure 11.10 in pre-, in-, post-, and level order.

13. Do Exercise 12 for the full binary tree of Figure 11.6.

14. List the nodes of the binary trees for the expressions of Exercise 6 in pre-, in-, post-, and level order.

Copyrighted material

15. The nodes of a binary tree are labeled $a - h$. The preorder listing is *abcdefgh*, and the inorder listing is *cdbagfeh*. Draw the binary tree. Also, list the nodes of your binary tree in postorder and level order.

16. Do Exercise 15 for a binary tree with node labels $a - l$, preorder listing *abcdefghijkl*, and inorder listing *aefdcgihjklb*.

17. The nodes of a binary tree are labeled $a - h$. The postorder listing is *abcdefgh*, and the inorder listing is *aedbchgf*. Draw the binary tree. Also, list the nodes of your binary tree in preorder and level order.

18. Do Exercise 17 for a binary tree with node labels $a - l$, postorder listing *abcdefghijkl*, and inorder listing *backdejifghl*.

19. Draw two binary trees whose preorder listing is *abcdefgh* and whose postorder listing is *dcbgfhea*. Also, list the nodes of your binary trees in inorder and level order.

20. Write a function to perform a preorder traversal on a binary tree represented as an array. Assume that the elements of the binary tree are stored in array `a` and that `last` is the position of the last element of the tree. The array is of type `pair<bool, T>`, where `a[i].first` is `true` iff there is an element (`a[i].second`) at position `i`. What is the time complexity of your function?

21. Do Exercise 20 for inorder.

22. Do Exercise 20 for postorder.

23. Do Exercise 20 for level order.

24. Write a C++ function to make a copy of a binary tree represented as an array.

25. Write two C++ functions to make a copy of a binary tree that is represented using nodes of type `binaryTreeNode`. The first function should traverse the tree in postorder, and the second in preorder. What is the difference (if any) in the recursion stack space needed by these two functions?

26. Write a function to evaluate an expression tree that is represented using nodes of type `binaryTreeNode`. Develop a suitable data type for the elements in the tree.

27. Write a function to determine the height of a linked binary tree. What is the time complexity of your function?

28. Write a function to determine the number of nodes in a linked binary tree. What is the time complexity of your function?

Copyrighted material

Hidden page

11.7 THE ADT *BinaryTree*

Now that we have some understanding of what a binary tree is, we can specify it as an abstract data type (ADT 11.1). Since the number of operations we may wish to perform on a binary tree is quite large, we list only some of the commonly performed ones.

AbstractDataType *binaryTree*
{
instances
collection of elements; if not empty, the collection is partitioned into a root, left subtree, and right subtree; each subtree is also a binary tree;

operations

empty() : return **true** if empty, return **false** otherwise;

size() : return number of elements/nodes in the tree;

preOrder(*visit*) : preorder traversal of binary tree; *visit* is the visit function to use;

inOrder(*visit*) : inorder traversal of binary tree;

postOrder(*visit*) : postorder traversal of binary tree;

levelOrder(*visit*) : level-order traversal of binary tree;
}

ADT 11.1 The abstract data type binary tree

Program 11.8 gives the C++ abstract class `binaryTree` that corresponds to the ADT *binaryTree*. The class `T` refers to the data type of the nodes in the binary tree. The data type

```
void (*) (T *)
```

used to specify the data type of the single parameter of the binary tree traversal methods specifies a function whose return type is `void` and which takes a single parameter of type `T*`.

11.8 THE CLASS `linkedBinaryTree`

The class `linkedBinaryTree` derives from the abstract class `binaryTree` and uses nodes of the type `binaryTreeNode` (Program 11.1). Program 11.9 gives the data members and some of the public and private methods of of `linkedBinaryTree`.

Copyrighted material

```
template<class T>
class binaryTree
{
   public:
      virtual ~binaryTree() {}
      virtual bool empty() const = 0;
      virtual int size() const = 0;
      virtual void preOrder(void (*) (T *)) = 0;
      virtual void inOrder(void (*) (T *)) = 0;
      virtual void postOrder(void (*) (T *)) = 0;
      virtual void levelOrder(void (*) (T *)) = 0;
};
```

Program 11.8 Binary tree abstract class

linkedBinaryTree has two instance data members **root** and **treeSize**. **root** is a pointer to the root node of the binary tre and **treeSize** is the number of nodes in the binary treee. The class **linkedBinaryTree** has a static data member **visit**, which is (a pointer to) a function whose return type is **void** and which has a single parameter that is a pointer to a **binaryTreeNode**. The static method **dispose** deletes a single node, and the method **erase** uses the static method **dispose** as the visit method for a postorder traversal to delete all nodes in a binary tree.

Program 11.10 gives the code for the preorder traversal method. The public method **preOrder** sets the class data member **visit** so that the desired function is used during the visit step. After setting the class data member **visit**, **preOrder** invokes the private recursive method **preOrder**, which does the actual preorder traversal. The corresponding inorder and postorder methods are similar.

The code for level-order traversal is quite similar to that given in Program 11.7. We may add a method to output a binary tree in preorder by adding the line

```
void preOrderOutput() {preOrder(output); cout << endl;}
```

to the **public** part of Program 11.9 and the lines

```
static void output(binaryTreeNode<E> *t)
            {cout << t->element << ' ';}
```

to the private part. Methods for inorder, postorder, and level-order output may be defined similarly.

Program 11.11 gives an additional public member method of **linkedBinaryTree** together with its accompanying private static recursive method. This recursive method determines the height of a binary tree by performing a postorder traversal of the binary tree. The method first determines the height of the left subtree, then

Copyrighted material

Hidden page

```
template<class E>
void linkedBinaryTree<E>::preOrder(binaryTreeNode<E> *t)
{// Preorder traversal.
   if (t != NULL)
   {
      linkedBinaryTree<E>::visit(t);
      preOrder(t->leftChild);
      preOrder(t->rightChild);
   }
}
```

Program 11.10 Private preorder method of `linkedBinaryTree`

```
int height() const {return height(root);}

template <class E>
int linkedBinaryTree<E>::height(binaryTreeNode<E> *t)
{// Return height of tree rooted at *t.
   if (t == NULL)
      return 0;                          // empty tree
   int hl = height(t->leftChild);  // height of left
   int hr = height(t->rightChild); // height of right
   if (hl > hr)
      return ++hl;
   else
      return ++hr;
}
```

Program 11.11 Determining the height of a binary tree

binary tree `*this` with the binary tree `x`. The method returns `true` iff the two binary trees are identical. Test your code. What is the complexity of your code?

46. Write the method `linkedBinaryTree<E>::swapTrees()`, which swaps the left and right subtrees of each node of a binary tree. Test your code. What is the complexity of your code?

47. Let $heightDifference(x)$ be the absolute value of the difference in heights of the left and right subtrees of node x. Let $maxHeightDifference(t)$ be $max\{heightDifference(x) | x$ is a node of the binary tree $t\}$. Write the method

Copyrighted material

Hidden page

of pipes from the source of the petroleum/natural gas to the consumption sites. Similarly, electrical power may be distributed through a network of wires from the power plant to the points of consumption. We will use the term **signal** to refer to the resource (petroleum, natural gas, power, etc.) that is to be distributed. While the signal is being transported through the distribution network, it may experience a loss in or degradation of one or more of its characteristics. For example, there may be a pressure drop along a natural gas pipeline or a voltage drop along an electrical transmission line. In other situations noise may enter the signal as it moves along the network. Between the signal source and point of consumption, we can tolerate only a certain amount, *tolerance*, of signal degradation. To guarantee a degradation that does not exceed this amount, **signal boosters** are placed at strategic places in the network. A signal booster might, for example, increase the signal pressure or voltage so that it equals that at the source or may enhance the signal so that the signal-to-noise ratio is the same as that at the source. In this section we develop an algorithm to determine where to place signal boosters. Our objective is to minimize the number of boosters in use while ensuring that the degradation in signal (relative to that at the source) does not exceed the given tolerance.

To simplify the problem, we assume that the distribution network is a tree with the source as the root. Each node in the tree (other than the root) represents a substation where we can place a booster. Some of these nodes also represent points of consumption. The signal flows from a node to its children. Figure 11.12 shows a distribution network that is a tree. Each edge is labeled by the amount of signal degradation that takes place when a signal flows between the corresponding parent and child. The units of degradation are assumed to be additive. That is, when a signal flows from node p to node v in Figure 11.12, the degradation is 5; the degradation from node q to node x is 3. When a signal booster is placed at node r, the strength of the signal that arrives at r is three units less than that of the signal that leaves source p; however, the signal that leaves r has the same strength as the signal that left p. Therefore, the signal that arrives at v has degraded by two units relative to the signal at the source; and the signal that arrives at z is four units weaker than the signal that left p. Without the booster at r, the arriving signal at z will be seven units weaker than the signal that left p.

Solution Strategy

Let $degradeFromParent(i)$ denote the degradation between node i and its parent. Therefore, in Figure 11.12, $degradeFromParent(w) = 2$, $degradeFromParent(p) = 0$, and $degradeFromParent(r) = 3$. Since signal boosters can be placed only at nodes of the distribution tree, the presence of a node i with $degradeFromParent(i) > tolerance$ implies that no placement of boosters can prevent signal degradation from exceeding *tolerance*. For example, if $tolerance = 2$, then there is no way to place signal boosters so that the degradation between p and r is $\leq tolerance = 2$ in Figure 11.12.

For any node i let $degradeToLeaf(i)$ denote the maximum signal degradation from node i to any leaf in the subtree rooted at i. If i is a leaf node, then

Copyrighted material

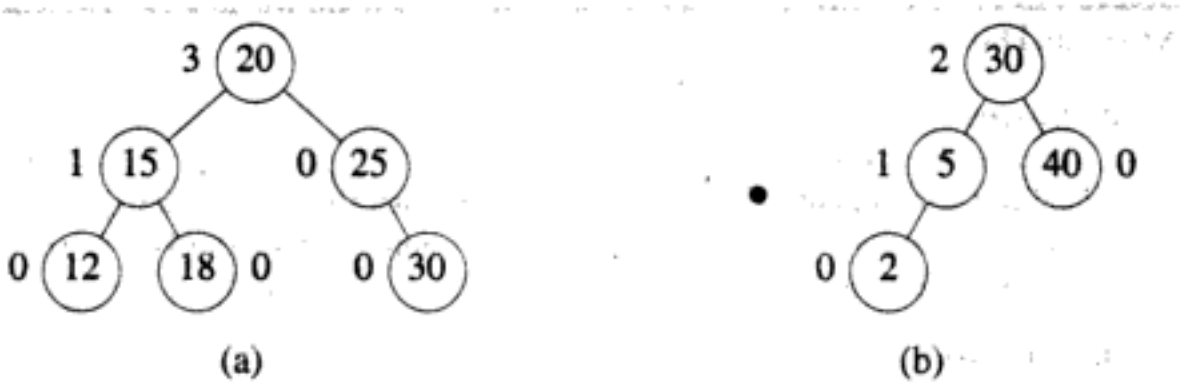


Figure 14.2 Indexed binary search trees

EXERCISES

1. Start with a complete binary tree that has 10 nodes. Place the keys [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], one key per node, so that the result is a binary search tree. Label each node with its `leftSize` value.

2. Start with a complete binary tree that has 13 nodes. Place the keys [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13], one key per node, so that the result is a binary search tree. Label each node with its `leftSize` value.

14.2 ABSTRACT DATA TYPES

ADT 14.1 gives the abstract data type specification for a binary search tree. An indexed binary search tree supports all the binary search tree operations. In addition, it supports search and deletion by rank. Its abstract data type specification is given in ADT 14.2. The abstract data types *dBSTree* (binary search trees with duplicates) and *dIndexedBSTree* may be specified in a similar way.

Programs 14.1 and 14.2 give C++ abstract classes for the ADTs *bsTree* and *indexedBSTree*, respectively.

EXERCISES

3. How much (expected) time does it take to do the *bsTree* operations of ADT 14.1 using skip lists?

4. Provide a specification for the abstract data type *dBSTree* (binary search tree with duplicates). Define a C++ abstract class that corresponds to this ADT.

Copyrighted material

Hidden page

Hidden page

method **inOrderOutput** as is done below.

```
void ascend() {inOrderOutput();}
```

The method **inOrderOutput** first outputs the elements in the left subtree (i.e., smaller elements), then the root element, and finally those in the right subtree (i.e., larger elements). The time complexity is $O(n)$ for an n-element tree.

```
void ascend() {inOrderOutput();}
```

Program 14.3 The method `binarySearchTree<K,E>::ascend`

14.3.2 Searching

Suppose we wish to search for a pair with key **theKey**. We begin at the root. If the root is **NULL**, the search tree contains no pairs and the search is unsuccessful. Otherwise, we compare **theKey** with the key in the root. If **theKey** is less than the key in the root, then no pair in the right subtree can have key value **theKey** and only the left subtree is to be searched. If **theKey** is larger than the key in the root, only the right subtree needs to be searched. If **theKey** equals the key in the root, then the search terminates successfully. The subtrees may be searched similarly. Program 14.4 gives the code. The time complexity is $O(h)$ where h is the height of the tree being searched.

14.3.3 Inserting an Element

To insert a new pair **thePair** into a binary search tree, we must first determine whether its key **thePair.first** is different from those of existing pairs by performing a search for **thePair.first**. If the search is successful, we must replace the old value associated with **thePair.first** with the value **thePair.second**. If the search is unsuccessful, then the new pair is inserted as a child of the last node examined during the search. For instance, to put a pair with key 80 into the tree of Figure 14.1(b), we first search for 80. This search terminates unsuccessfully, and the last node examined is the one with key 40. The new pair is inserted as the right child of this node. The resulting search tree appears in Figure 14.3(a). Figure 14.3(b) shows the result of inserting a pair with key 35 into the search tree of Figure 14.3(a). Program 14.5 implements this strategy to insert a pair into a binary search tree.

14.3.4 Deleting an Element

For deletion we consider the three possibilities for the node **p** that contains the pair that is to be removed: (1) **p** is a leaf, (2) **p** has exactly one nonempty subtree, and (3) **p** has exactly two nonempty subtrees.

Copyrighted material

Hidden page

```
template<class K, class E>
void binarySearchTree<K,E>::insert(const pair<const K, E>& thePair)
{// Insert thePair into the tree. Overwrite existing
 // pair, if any, with same key.
   // find place to insert
   binaryTreeNode<pair<const K, E> > *p = root,
                                     *pp = NULL;
   while (p != NULL)
   {// examine p->element
      pp = p;
      // move p to a child
      if (thePair.first < p->element.first)
         p = p->leftChild;
      else
         if (thePair.first > p->element.first)
            p = p->rightChild;
         else
         {// replace old value
            p->element.second = thePair.second;
            return;
         }
   }

   // get a node for thePair and attach to pp
   binaryTreeNode<pair<const K, E> > *newNode
                = new binaryTreeNode<pair<const K, E> > (thePair);
   if (root != NULL) // the tree is not empty
      if (thePair.first < pp->element.first)
         pp->leftChild = newNode;
      else
         pp->rightChild = newNode;
   else
      root = newNode; // insertion into empty tree
   treeSize++;
}
```

Program 14.5 Insert an element into a binary search tree

80 is discarded.

Next consider the case when the pair to be removed is in a node p that has only one nonempty subtree. If p has no parent (i.e., it is the root), the root of its single

Copyrighted material

subtree becomes the new search tree root. If p has a parent pp, then we change the pointer from pp so that it points to p's only child. For instance, if we wish to delete the pair with key 5 from the tree of Figure 14.3(b), we change the left-child field of its parent (i.e., the node containing 30) to point to the node containing the 2.

Finally, to remove a pair in a node that has two nonempty subtrees, we replace this pair with either the largest pair in its left subtree or the smallest pair in its right subtree. Following this replacement, the replacing pair is removed from its original node. Suppose we wish to remove the pair with key 40 from the tree of Figure 14.4(a). Either the largest pair (35) from its left subtree or the smallest (60) from its right subtree can replace this pair. If we opt for the smallest pair in the right subtree, we move the pair with key 60 to the node from which the 40 was removed; in addition, the leaf from which the 60 is moved is removed. The resulting tree appears in Figure 14.4(b).

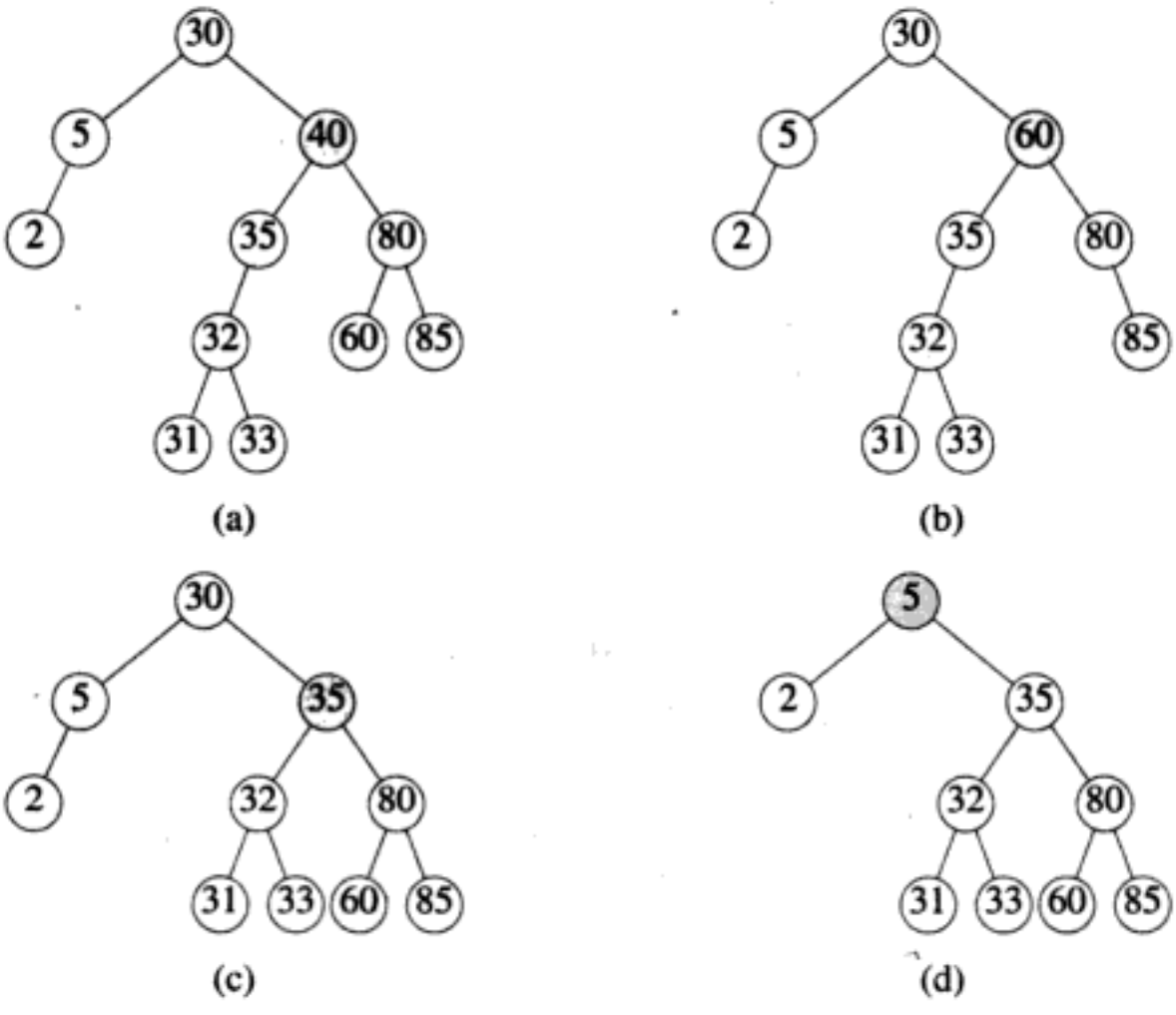


Figure 14.4 Deletion from a binary search tree

Suppose, instead, that when removing 40 from the search tree of Figure 14.4(a),

Copyrighted material

Hidden page

Hidden page

Hidden page

Hidden page

operations in the order specified by the random permutation. Measure the height of the resulting search tree. Repeat this experiment for several random permutations and compute the average of the measured heights. Compare this figure with $2\lceil \log_2(n+1) \rceil$. For n use the values 100; 500; 1000; 10,000; 20,000; and 50,000.

14. Extend the class `binarySearchTree` by including an iterator that allows you to examine the pairs in ascending order of the key. The time taken to enumerate all pairs of an n-element binary tree should be $O(n)$, the complexity of no method should exceed $O(h)$ where h is the tree height, and the space requirements should be $O(h)$. Show that this is the case. Test your code.

15. Write a method to delete the pair with largest key from a binary search tree. Your method must have time complexity $O(h)$ where h is the height of the binary search tree. Show that your code has this complexity.

    (a) Use suitable test data to test the correctness of your delete max method.

    (b) Create a random list of n pairs and a random sequence of insert and delete max operations of length m. Create the latter sequence so that the probability of an insert operation is approximately 0.5 (therefore, the probability of a delete max operation is also 0.5). Initialize a max heap and a binary search tree with duplicates to contain the n pairs in the first random list. Now measure the time to perform the m operations, using the max heap as well as the binary search tree. Divide this time by m to get the average time per operation. Do this experiment for n = 100, 500, 1000, 2000, $\cdots$, and 5000. Let m be 5000. Tabulate your computing times.

    (c) Based on your experiments, what can you say about the relative merits of the two priority queue schemes?

14.4 BINARY SEARCH TREES WITH DUPLICATES

The class for the case when the binary search tree is permitted to contain two or more pairs that have the same key is called `dBinarySearchTree`. We can implement this class by changing the `while` loop of `binarySearchTree<K,E>::insert` (Program 14.5) to that shown in Program 14.7 and by changing the second occurrence of the line

```
if (thePair.first < pp->element.first)
```

to

```
if (thePair.first <= pp->element.first)
```

Copyrighted material

```
while (p != NULL)
{// examine p->element
   pp = p;
   // move p to a child
   if (thePair.first <= p->element.first)
      p = p->leftChild;
   else
      p = p->rightChild;
}
```

Program 14.7 New `while` loop for Program 14.5 to permit duplicates

If we insert n pairs, all of which have the same key, into an an initially empty binary search tree, the result is a left-skewed tree whose height is n. An alternative approach that uses random numbers and results in trees with better height properties is considered in Exercise 17.

EXERCISES

16. Start with an empty binary search tree with duplicates and insert the keys 2, 2, 2, and 2 using the `insert` method described in this section. Draw your tree following each insert. What is the tree height following the insertion of n 2s?

17. Develop a new implementation of the C++ class `dBinarySearchTree`. During an insert, instead of consistently moving to the left subtree of a node that contains a key equal to `thePair.first` (see Program 14.7), use a random number generator to move to the left or right subtree with equal probability. Test the correctness of your implementation.

14.5 INDEXED BINARY SEARCH TREES

The class `indexedBinarySearchTree` may also be defined as a derived class of `linkedBinaryTree` (see Exercise 19). For the class `indexedBinarySearchTree`, the `element` field of a node is a triple `leftSize`, `key`, and `value`.

We can perform an indexed search in a manner similar to that used to search for a pair (a pair is now defined by the fields `key` and `value`). Consider the tree of Figure 14.2(a). Suppose we are looking for the pair whose index is 2. The `leftSize` field (and hence index) of the root is 3. So the pair we desire is in the left subtree. Furthermore, the pair we desire has index 2 in the left subtree. The root, 15, of

Copyrighted material

the left subtree has `leftSize` = 1. So the pair we desire is in the right subtree of 15. However, the index, in the right subtree of 15, of the pair we desire is no longer 2 because all pairs in the right subtree of 15 follow the pairs in the left subtree of 15 as well as the pair 15. To determine the index of the desired pair in the right subtree, we subtract `leftSize + 1`. Since the `leftSize` value of 15 is 1, the index of the desired pair in its right subtree is $2-(1+1)=0$. Since `leftSize` = 0 (which equals the index of the pair we seek) for the root of the right subtree of 15, we have found the desired pair 18.

When inserting a pair into an indexed binary search tree, we use a procedure similar to Program 14.5. This time, though, we also need to update `leftSize` fields on the path from the root to the newly inserted node.

A delete by index is done by first performing an indexed search to locate the pair to be deleted. Next we delete the pair as outlined in Section 14.3.4 and update `leftSize` fields on the path from the root to the physically deleted node as necessary.

The time required to find, insert, and delete is $O(h)$ time where h is the height of the indexed search tree.

EXERCISES

18. Start with an empty indexed binary search tree.

    (a) Insert the keys 4, 12, 8, 16, 6, 18, 24, 2, 14, 3 in this order. Draw the tree following each insert. Show `leftSize` values. Use the insert method described in this section.

    (b) Use the method of this section to find the keys with index 3, 6, and 8. Describe the search process used in each case.

    (c) Start with the tree of (a) and remove the keys whose index is 7, 5, and 0 in this order. Draw the search tree following each deletion. Use the deletion method of this section.

19. Develop the C++ class `indexedBinarySearchTree`. You should derive from the abstract class `indexedBSTree`. Test the correctness of your code. Express the time complexity of each method in terms of the number of elements and/or the height of the tree.

20. Do Exercise 19 for the class `dIndexedBinarySearchTree`, which represents an indexed binary search tree with duplicates.

21. You are to represent a linear list as an indexed binary tree which is like an indexed binary search tree except that no node has a key value. Each node of the indexed binary tree contains exactly one element of the linear list. When an indexed binary tree is traversed in inorder, we visit the elements in linear list order from left to right.

Copyrighted material

Hidden page

Hidden page

```
void main(void)
{// Histogram of nonnegative integer values.
   int n,  // number of elements
       r;  // values between 0 and r
   cout << "Enter number of elements and range"
        << endl;
   cin >> n >> r;

   // create histogram array h
   int *h = new int[r+1];

   // initialize array h to zero
   for (int i = 0; i <= r; i++)
      h[i] = 0;

   // input data and compute histogram
   for (i = 1; i <= n; i++)
   {// assume input values are between 0 and r
      int key;  // input value
      cout << "Enter element " << i << endl;
      cin >> key;
      h[key]++;
   }

   // output histogram
   cout << "Distinct elements and frequencies are"
        << endl;
   for (i = 0; i <= r; i++)
      if (h[i] != 0)
         cout << i << "   " << h[i] << endl;
}
```

Program 14.8 Simple histogramming program

Histogramming with a Binary Search Tree

Program 14.8 becomes infeasible when the key range is very large as well as when the key type is not integral (for example, when the keys are real numbers). Suppose we are determining the frequency with which different words occur in a text. The number of possible different words is very large compared to the number that might actually appear in the text. In such a situation we can use hashing to arrive at a solution whose expected complexity is $O(n)$ (Exercise 24). Alternatively, we may

Copyrighted material

$$\begin{aligned} F &= (3+4)5 = 35 \\ G &= (1+2)8 = 24 \\ H &= (3-1)(5+6) = 22 \\ I &= (2-4)(7+8) = -30 \\ J &= (1+4)(5+8) = 65 \end{aligned}$$

From these values the components of the answer are computed as follows:

$$\begin{aligned} C_1 &= 8-30+65-24 = 19 \\ C_2 &= -2+24 = 22 \\ C_3 &= 8+35 = 43 \\ C_4 &= -2+22+65-35 = 50 \end{aligned}$$

For our 2×2 instance, the divide-and-conquer algorithm has done seven multiplications and 18 add/subtracts. We could have computed C by performing eight multiplications and four add/subtracts by using Equation 18.1 directly. For the divide-and-conquer scheme to be faster, the time cost of a multiplication must be more than the time cost of 14 add/subtracts.

If Strassen's instance-dividing scheme is used only when $n \geq 8$ and smaller instances are solved using Equation 18.1, then the case $n = 8$ requires seven multiplications of 4×4 matrices and 18 add/subtracts of matrices of this size. The multiplications take $64m + 48a$ operations each, and each matrix addition or subtraction takes $16a$ operations. The total operation count is $7(64m+48a) + 18(16a) = 448m + 624$. The direct method has an operation count of $512m + 448a$. A minimum requirement for Strassen's method to be faster is that the cost of $512 - 448$ multiplications be more than that of $624-448$ add/subtracts. Or one multiplication should cost more than approximately 2.75 add/subtracts.

If we consider a "small" instance to be one with $n < 16$, the Strassen's decomposition scheme is used only for matrices with $n \geq 16$; smaller matrices are multiplied using Equation 18.1. The operation count for the divide-and-conquer algorithm becomes $7(512m + 448a) + 18(64a) = 3584m + 4288a$ when $n = 16$. The direct method has an operation count of $4096m + 3840a$. If the cost of a multiplication is the same as that of an add/subtract, then Strassen's method needs time for 7872 operations plus overhead time to do the problem division. The direct method needs time for 7936 operations plus time for the `for` loops and other overhead items in the program. Even though the operation count is less for Strassen's method, it is not expected to run faster because of its larger overhead.

For larger values of n, the difference between the operation counts of Strassen's method and the direct method becomes larger and larger. So for suitably large n,

Copyrighted material

Strassen's method will be faster. Let $t(n)$ denote the time required by Strassen's divide-and-conquer method. Since large instances are recursively divided into smaller ones until each instance becomes of size k or less (k is at least eight and may be larger depending on the implementation and computer characteristics), the recurrence for t is

$$t(n) = \begin{cases} d & n \leq k \\ 7t(n/2) + cn^2 & n > k \end{cases} \tag{18.6}$$

where cn^2 represents the time to perform 18 add/subtracts of $n/2 \times n/2$ matrices as well as the time to divide the instance of size n into the smaller instances. Using the substitution method, this recurrence may be solved to obtain $t(n) = \Theta(n^{\log_2 7})$. Since $\log_2 7 \approx 2.81$, the divide-and-conquer matrix multiplication algorithm is asymptotically faster than Program 2.22. ■

Implementation Note

The divide-and-conquer methodology naturally leads to recursive algorithms. In many instances, these recursive algorithms are best implemented as recursive programs. In fact, in many cases all attempts to obtain nonrecursive programs result in the use of a stack to simulate the recursion stack. However, in some instances we can implement the divide-and-conquer algorithm as a nonrecursive program without the use of such a stack and in such a way that the resulting program is faster, by a constant factor, than the natural recursive implementation. The divide-and-conquer algorithms for both the gold nuggets problem (Example 18.2) and the merge sort method (Section 18.2.2) can be implemented without the use of recursion so as to obtain fast programs that do not directly simulate the recursion.

Example 18.4 [Gold Nuggets] The work done by the algorithm of Example 18.2 to find the lightest and heaviest of eight nuggets is described by the binary tree of Figure 18.3. The leaves of this tree denote the eight nuggets (a, b, ···, h). Each shaded node represents an instance containing all the leaves in its subtree. Therefore, root A represents the problem of finding the lightest and heaviest of all eight nuggets, while node B represents the problem of finding the lightest and heaviest of the nuggets a, b, c, and d. The algorithm begins at the root. The eight-nugget instance represented by the root is divided into 2 four-nugget instances represented by the nodes B and C. At B the four-nugget instance is divided into the two-nugget instances D and E. We solve the two-nugget instance at node D by comparing the nuggets a and b to determine which is heavier. After we solve the problems at D and E, we solve the problem at B by comparing the lighter nuggets from D and E, as well as the heavier nuggets at D and E. We repeat this process at F, G, and C and then at A.

We can classify the work of the recursive divide-and-conquer algorithm as follows:

Copyrighted material

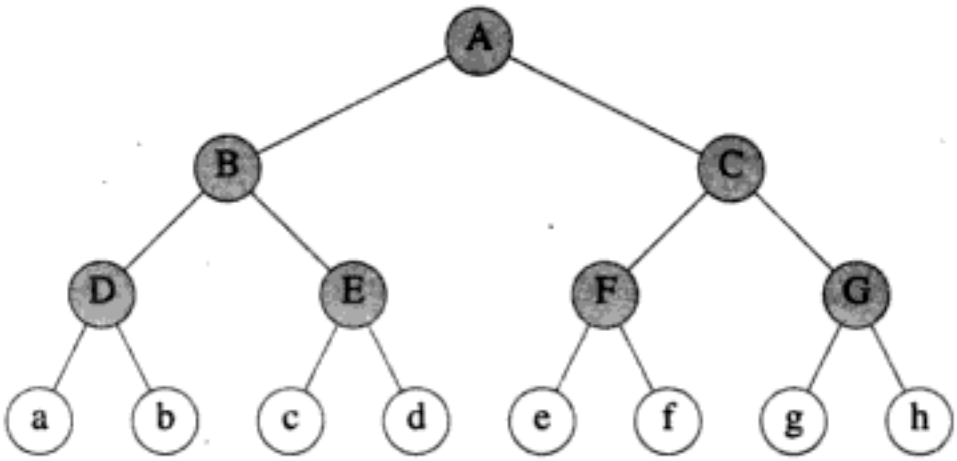


Figure 18.3 Finding the lightest and heaviest of eight nuggets

1. Divide a large instance into many smaller ones, each of size 1 or 2, during a downward root-to-leaf pass of the binary tree of Figure 18.3.

2. Compare the nuggets in each smaller size 2 instance to determine which nugget is heavier and which is lighter. This comparison is done at nodes D, E, F, and G. For size 1 instances the single nugget is both the smaller and lighter nugget.

3. Compare the lighter nuggets to determine which is lightest. Compare the heavier nuggets to determine which is heaviest. These comparisons are done at nodes A through C.

This classification of the work leads to the nonrecursive code of Program 18.1 to find the locations of the minimum and maximum of the `n` weights `a[0:n-1]`. These locations are returned in the variables `indexOfMin` and `indexOfMax`.

The cases $n < 1$ and $n = 1$ are handled first. If $n > 1$ and odd, the first weight `a[0]` becomes the candidate for minimum and maximum, and we are left with an even number of weights `a[1:n-1]` to account for in the `for` loop. When `n` is even, the first two weights are compared outside the `for` loop and `indexOfMin` and `indexOfMax` are set to the location of the smaller and larger weight, respectively. Again, we are left with an even number of weights `a[2:n-1]` to account for in the `for` loop.

In the `for` loop the outer `if` finds the larger and smaller of the pair `(a[i],a[i+1])` being compared. This work corresponds to the category 2 work in our classification of the work of our divide-and-conquer algorithm. The embedded `if`s find the smallest of the smaller weights and the largest of the larger weights. This work is the category 3 work.

The `for` loop compares the smaller of each pair to the current minimum weight

Copyrighted material

Hidden page

Hidden page

Hidden page

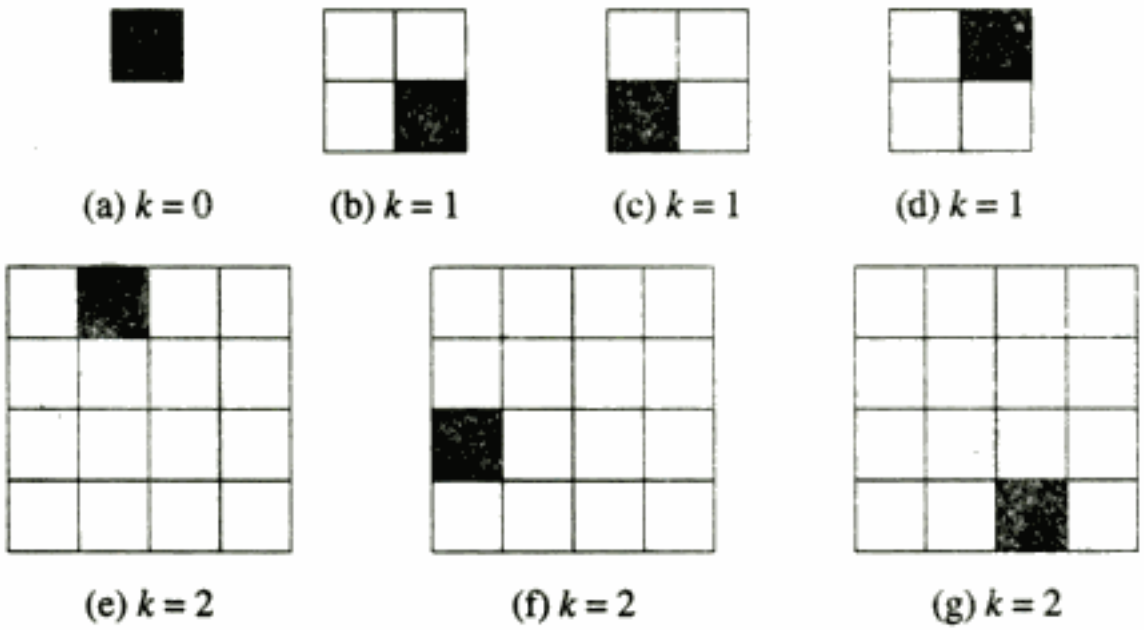


Figure 18.4 Defective chessboards

squares and these squares are covered using a triomino in one of the orientations of Figure 18.5.

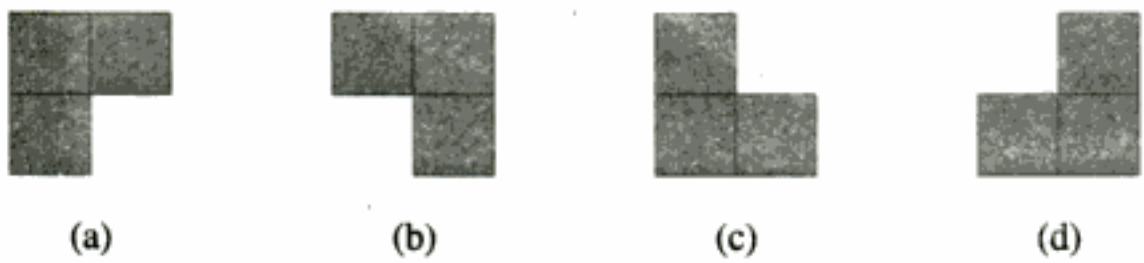


Figure 18.5 Triominoes with different orientations

Solution Strategy

The divide-and-conquer method leads to an elegant solution to the defective-chessboard problem. The method suggests reducing the problem of tiling a $2^k \times 2^k$ defective chessboard to that of tiling smaller defective chessboards. A natural partitioning of a $2^k \times 2^k$ chessboard would be into four $2^{k-1} \times 2^{k-1}$ chessboards as in Figure 18.6(a). Notice that when such a partitioning is done, only one of the four smaller boards has a defect (as the original $2^k \times 2^k$ board had exactly one defective square). Tiling one of the four smaller boards corresponds to tiling a defective

Copyrighted material

Hidden page

Hidden page

$$t(k) = \begin{cases} d & k = 0 \\ 4t(k-1) + c & k > 0 \end{cases} \tag{18.7}$$

We may solve this recurrence using the substitution method (see Example 2.20) to obtain $t(k) = \Theta(4^k) = \Theta(\text{number of tiles needed})$. Since we must spend at least $\Theta(1)$ time placing each tile, we cannot obtain an asymptotically faster algorithm than divide and conquer.

18.2.2 Merge Sort

The Sort Method

We can apply the divide-and-conquer method to the sorting problem. In this problem we must sort n elements into nondecreasing order. The divide-and-conquer method suggests sorting algorithms with the following general structure: If n is 1, terminate; otherwise, partition the collection of elements into two or more subcollections; sort each; combine the sorted subcollections into a single sorted collection.

Suppose we limit ourselves to partitioning the n elements into two subcollections. Now we need to decide how to perform this partitioning. One possibility is to put the first $n-1$ elements into the first subcollection (say, A) and the last element into the second subcollection (say, B). A is sorted using this partitioning scheme recursively. Since B has only one element, it is already sorted. Following the sort of A, we need to combine A and B, using the method `insert` of Program 2.10. Comparing the resulting sort algorithm with `insertionSort` (Program 2.15), we see that we have really discovered the recursive version of insertion sort. The complexity of this sort algorithm is $O(n^2)$.

Another possibility for the two-way partitioning of n elements is to put the element with largest key in B and the remaining elements in A. Then A is sorted recursively. To combine the sorted A and B, we need merely append B to the sorted A. If we find the element with largest key using the function `Max` of Program 1.37, the resulting sort algorithm is a recursive formulation of `selectionSort` (Program 2.7). If we use a bubbling process (Program 2.8) to locate and move the element with largest key to the right-most position, the resulting algorithm is a recursive version of `bubbleSort` (Program 2.9). In either case the sort algorithm has complexity $\Theta(n^2)$. This complexity can be made $O(n^2)$ by terminating the recursive partitioning of A as soon as A is known to be in sorted order (see Examples 2.16 and 2.17).

The partitioning schemes used to arrive at the three preceding sort algorithms partitioned the n elements into two very unbalanced collections A and B. A has $n-1$ elements, while B has only 1 element. Let us see what happens when this partitioning is done in a more balanced way, that is, when A gets a fraction n/k of the elements and B gets the rest. Now both A and B are to be sorted by recursive

Copyrighted material